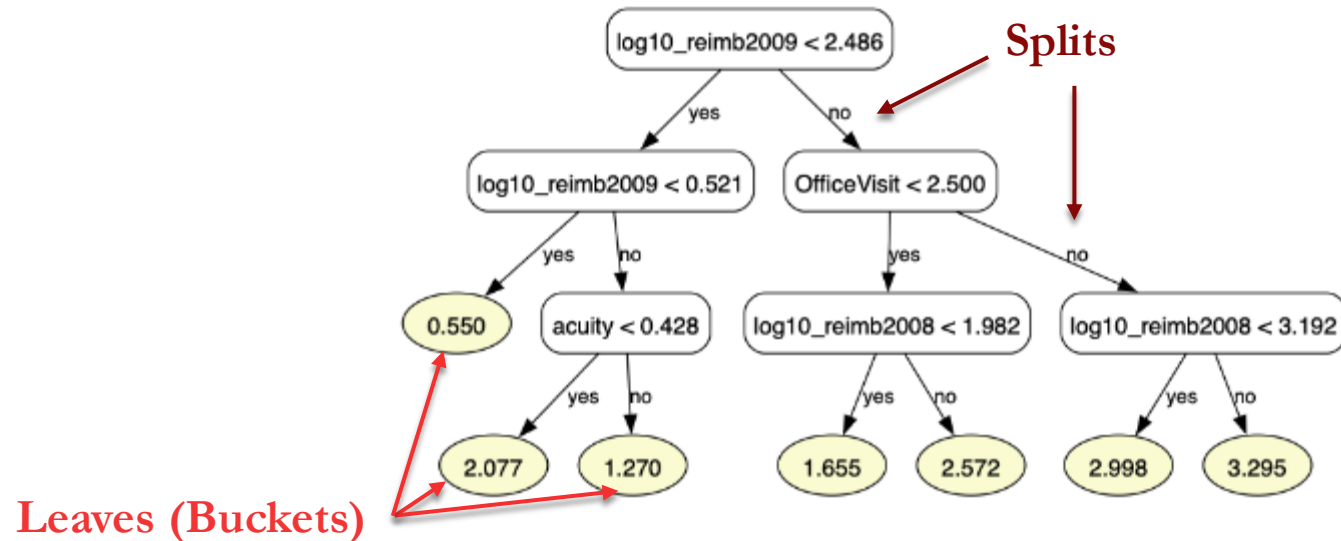




TREE-BASED ENSEMBLE LEARNING

Predicting Click-Through Rates of Ad Impressions

Review: What is a CART Model?



- CART is a “tree”-based model used to make predictions.
- Given the observed values of the independent variables, just “follow the splits” to the final node
 - At the split, go left if “yes” and go right if “no”
- Follow the splits to the end/bottom of the tree and read off the value. That’s it!

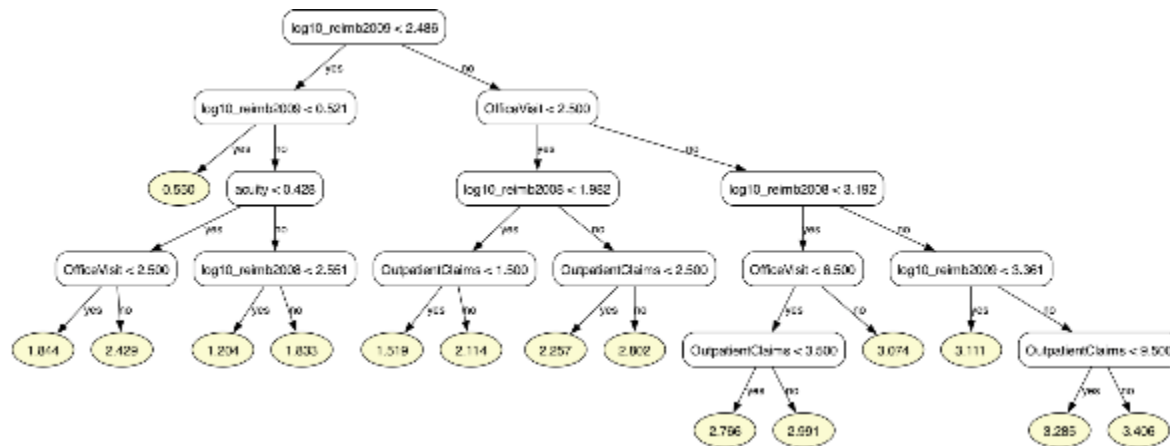
(Hyper)-Parameters

- When we build a CART model, we can specify some parameters which control the complexity of the resulting tree.

Parameter Name in Python	English Name	Description	Default Value	Higher = Simpler Model?
<code>min_samples_split</code>	Minimum Split Size	CART will only consider splitting a node at all if its size is at least <code>min_samples_split</code> .	2	Yes
<code>min_samples_leaf</code>	Minimum Bucket Size	CART only considers splits if each leaf has <code>min_samples_leaf</code> data points at least.	1	Yes
<code>max_depth</code>	Maximum Depth	CART will build a tree no deeper than <code>max_depth</code> .	None	No
<code>ccp_alpha</code>	Complexity Parameter	CART removes a branch if it reduces impurity (MSE or SSE) by less than <code>ccp_alpha</code> $\times$ (# leaves - 1).	0.0	Yes

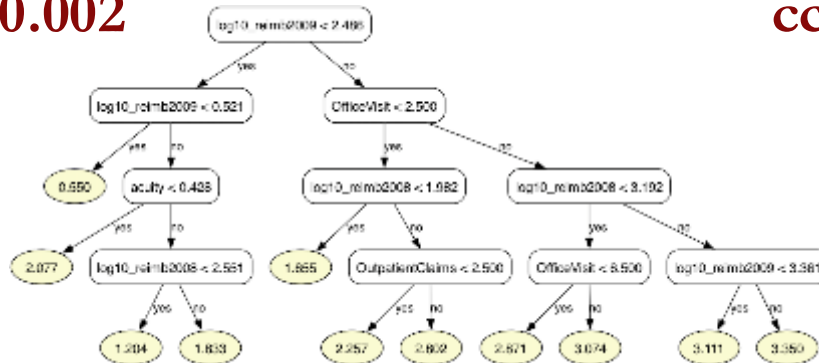
Impact of `ccp_alpha` value on the tree size

ccp=0.001



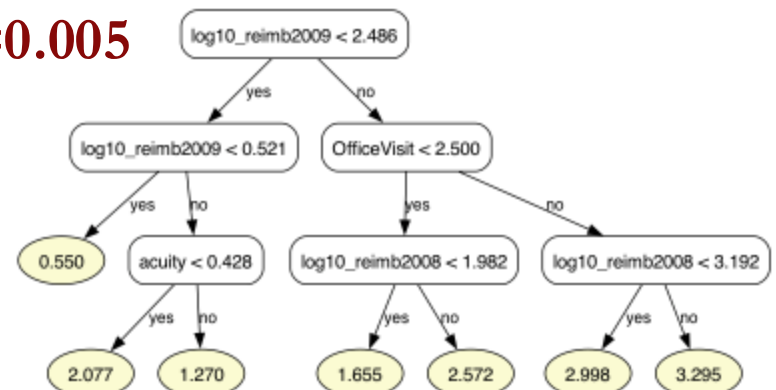
15 leaves

ccp=0.002



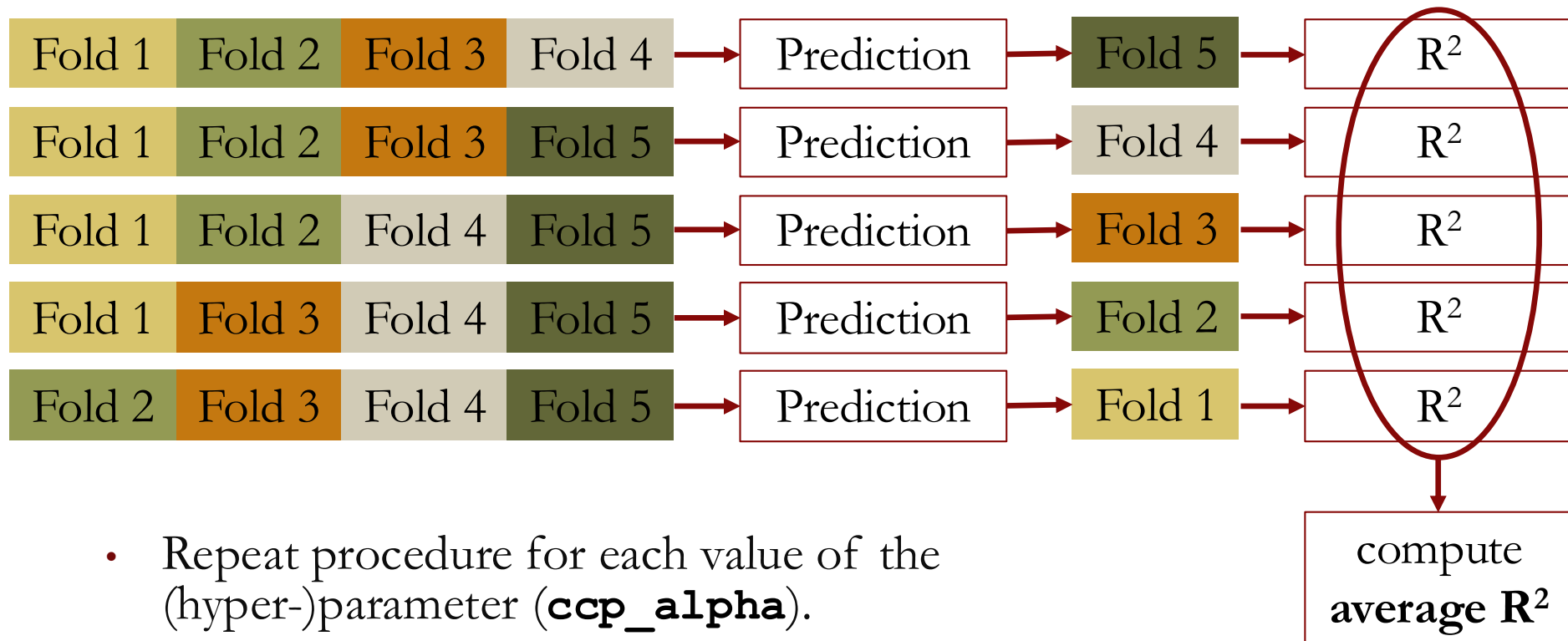
11 leaves

ccp=0.005



7 leaves

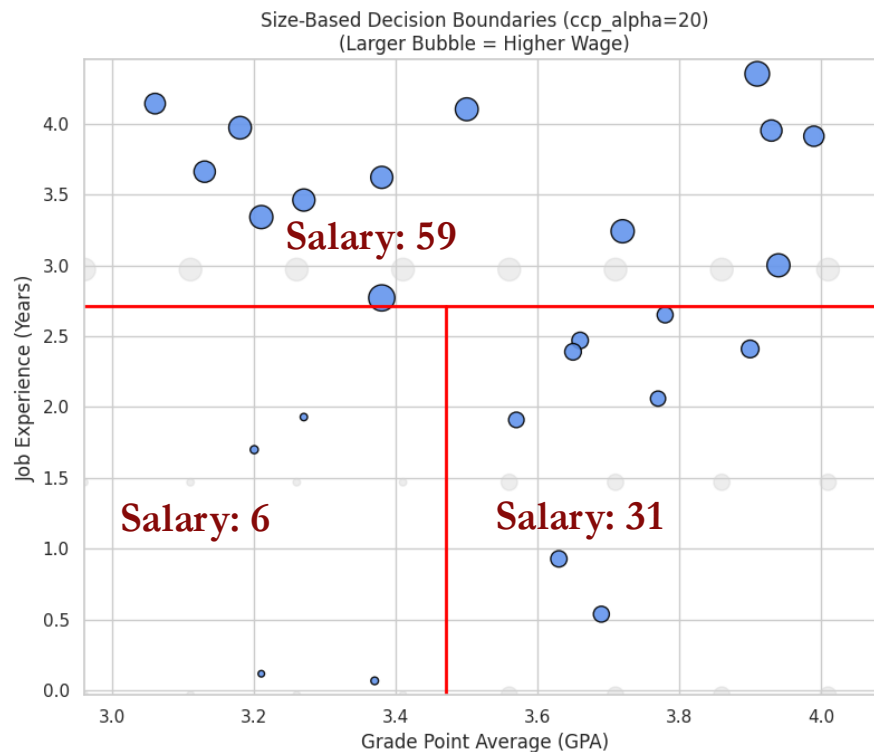
5-Fold Cross-Validation



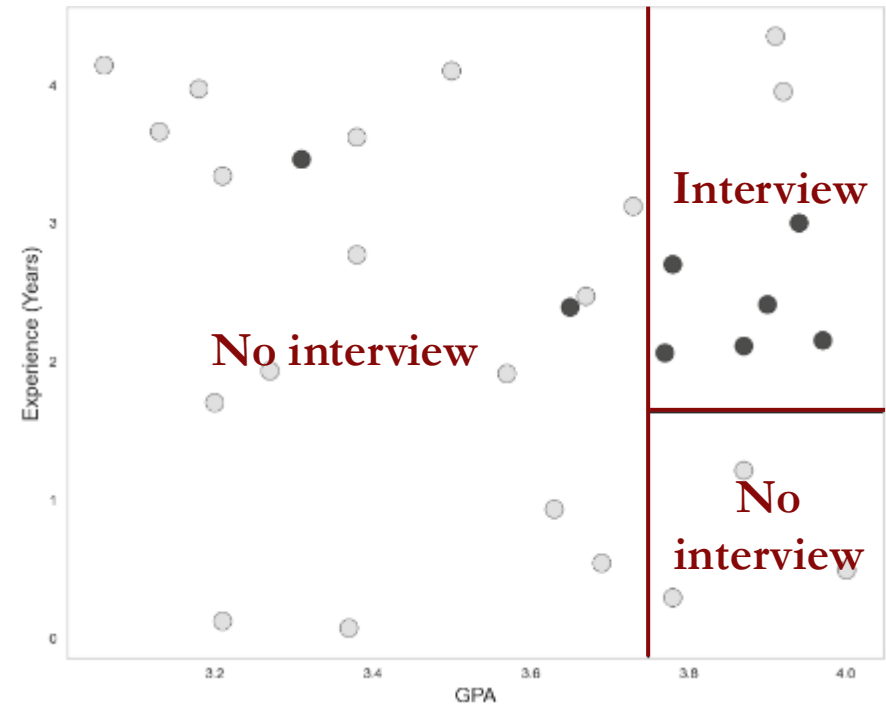
- Repeat procedure for each value of the (hyper-)parameter (**ccp_alpha**).
- Choose the hyper-parameter value that maximizes the **average R^2**

CART for Regression vs. Classification

Regression: Salary (\$/hour)

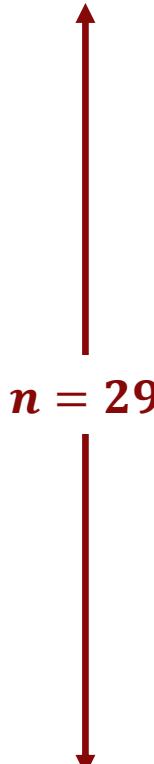


Classification: Interview or not



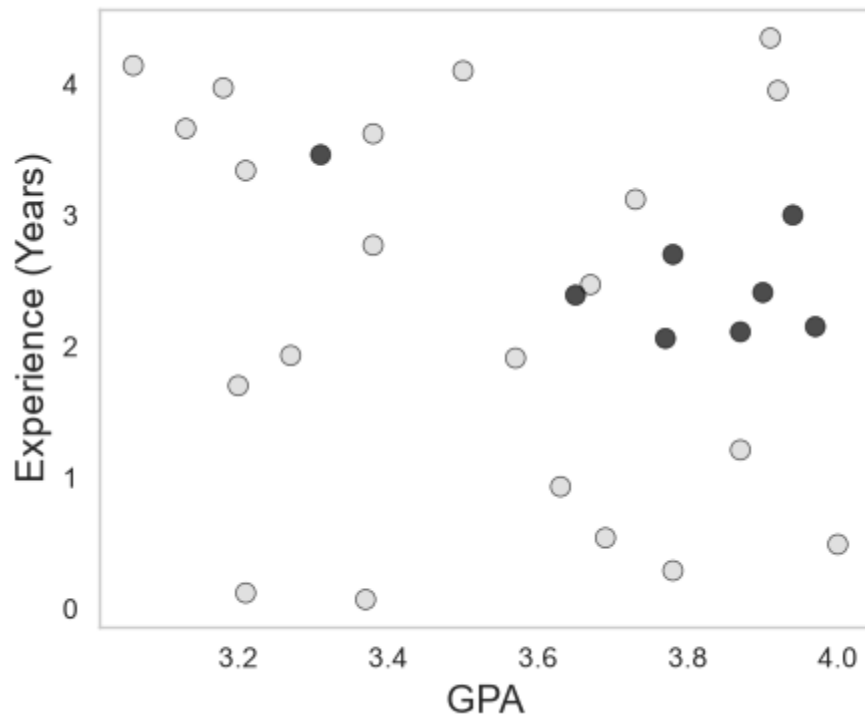
Toy Example: Job Interviews

k : # of independent variables ($k = 2$); n : # of observations ($n = 29$)



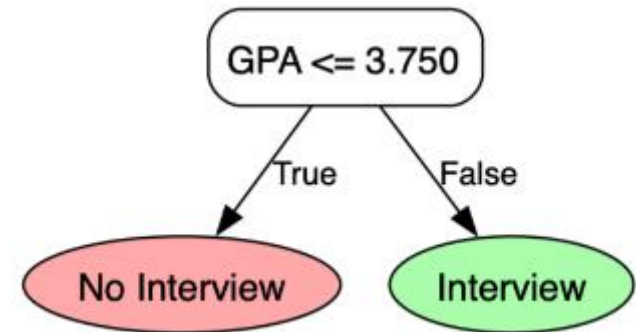
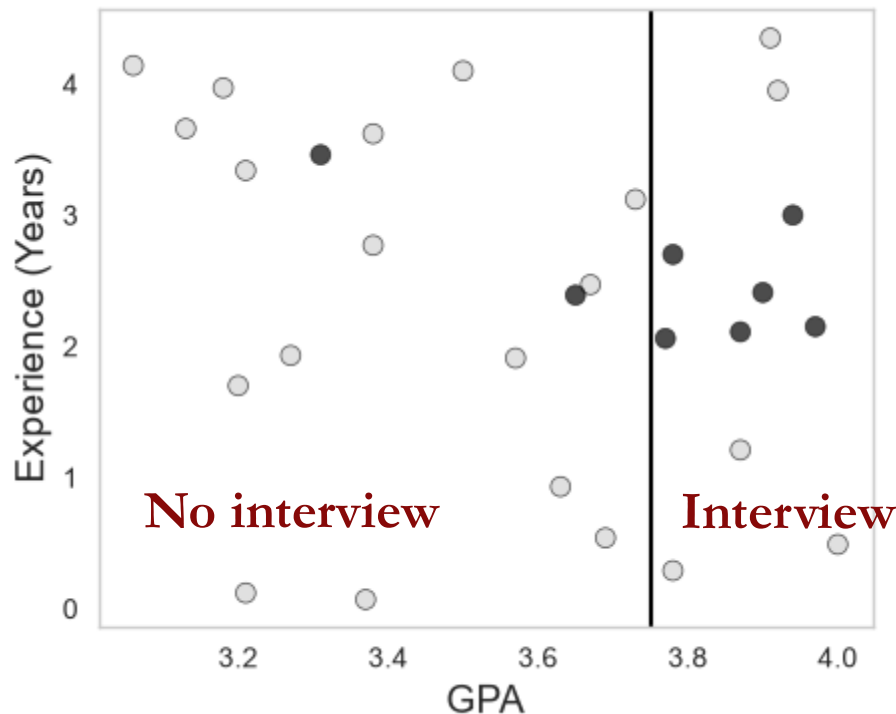
	Interview	GPA	Experience
0	0	3.27	1.93
1	0	3.37	0.07
2	0	3.57	1.91
3	0	3.91	4.35
4	0	3.2	1.7
5	1	3.9	2.41
6	1	3.94	3.0
...	...	...	...
26	1	3.97	2.15
27	1	3.78	2.7
28	0	3.73	3.12

Who Will Receive a Job Interview?

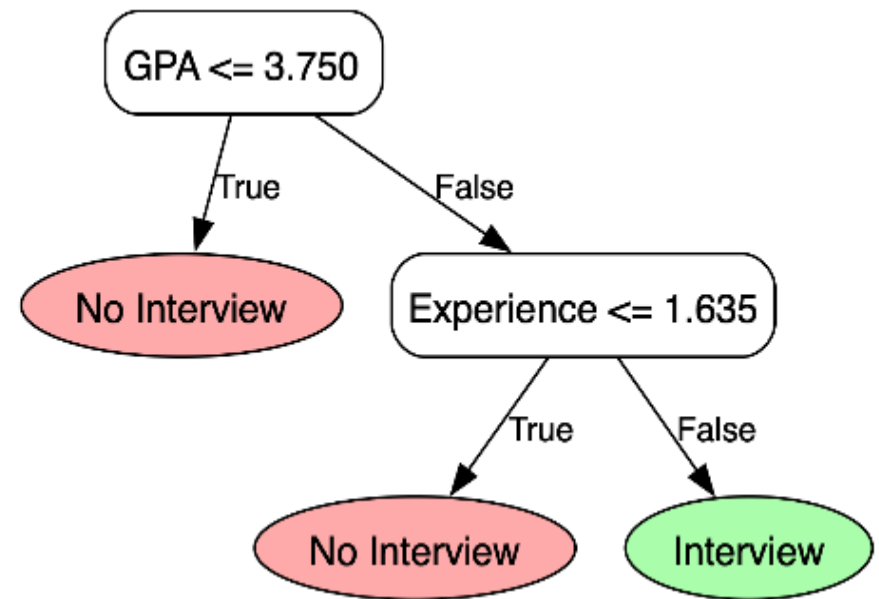
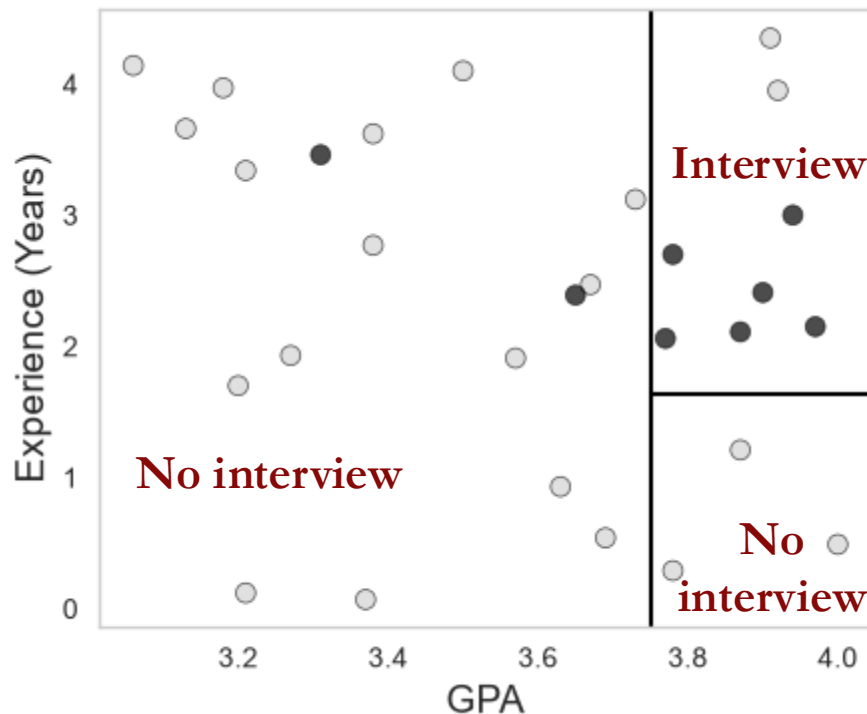


No Interview

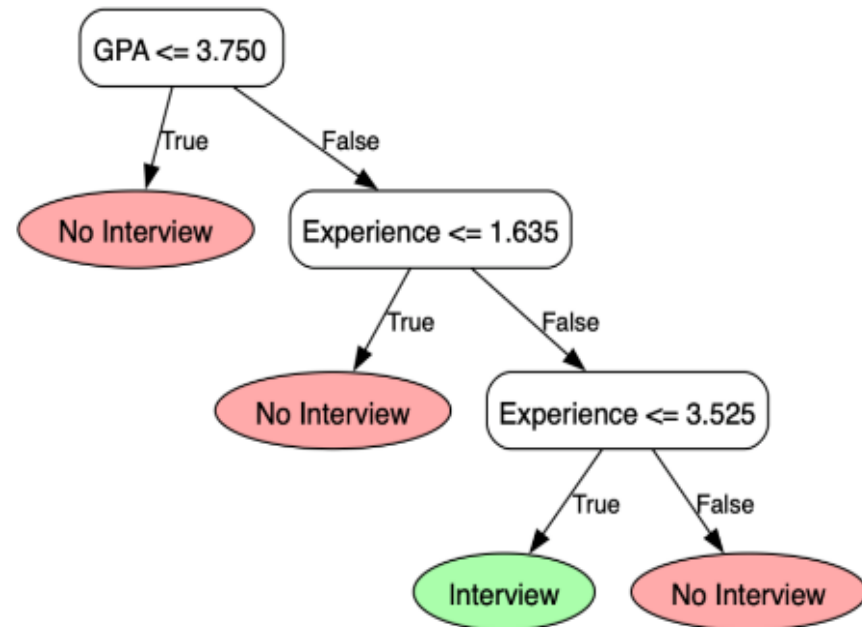
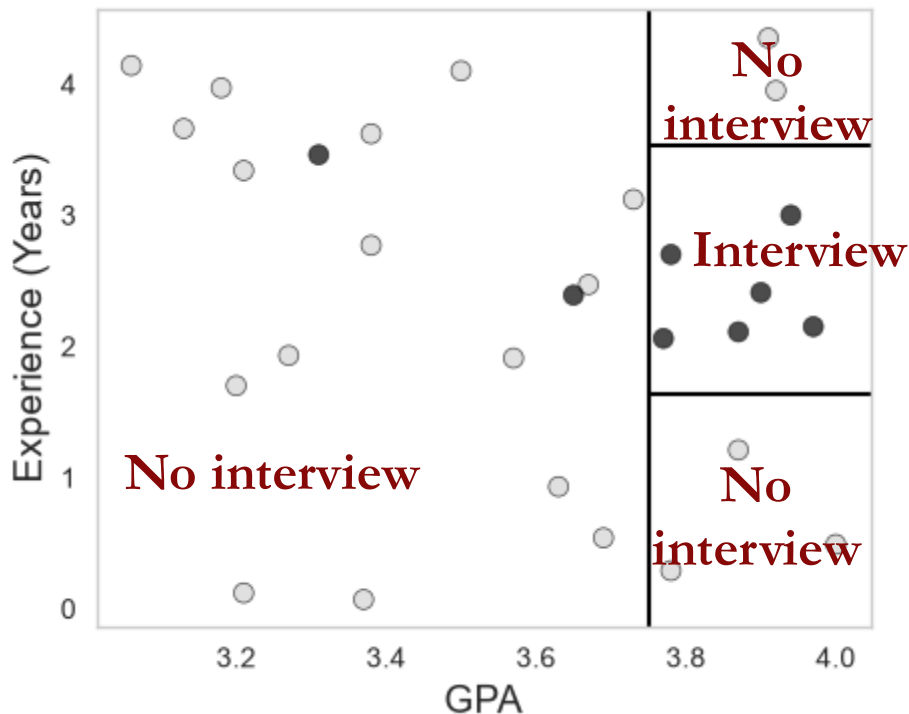
Who Will Receive a Job Interview?



Who Will Receive a Job Interview?



Who Will Receive a Job Interview?



How to predict: Total Loss (Cost)

		Prediction/Decision	
		No Violation (Grant Parole)	Violation (Deny Parole)
Reality	No Violation	0	L_{FP}
	Violation	L_{FN}	0

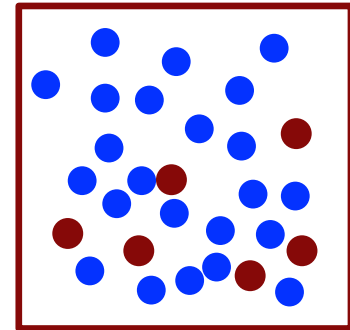
- In CART for Classification, we measure the quality of the model using:

$$\text{Total Loss} = (\text{number of FN}) * \overset{\text{Cost of a single false negative (FN)}}{L_{FN}} + (\text{number of FP}) * \overset{\text{Cost of a single false positive (FP)}}{L_{FP}}$$

$$\text{Average Loss} = \frac{\text{Total Loss}}{\text{Number of Data Points}}$$

Prediction at a Leaf

- For each leaf bucket, CART selects the class that has the lowest **Total Loss** across all the training observations in that leaf bucket.
- From the training set observations, suppose the bucket has:
 - 6 parole violators ●
 - 23 non-violators ●
- $L_{FN} = 20$ and $L_{FP} = 1$
- If predict this leaf as “**Violator**” Total Loss is
 $23 * 1 = 23$
- Total Loss from predicting “**Non-violator**” for this leaf
 $6 * 20 = 120$
- “**Violator**” has the lower loss



Gini index: leaf gini

- Assume a CART tree has L leaves numbered from 1 to L .
- For each $l \in \{1, \dots, L\}$, let p_l and q_l respectively denote the *proportions* of loss-weighted samples sizes of violators and non-violators at leaf l . Note that $p_l + q_l = 1$.
 - E.g. Leaf l has 6 violators and 23 non-violators. Then:

$$p_l = \frac{20 \times 6}{20 \times 6 + 1 \times 23} = 0.84, \quad q_l = \frac{1 \times 23}{20 \times 6 + 1 \times 23} = 0.16.$$

- The expression of the *Gini index* of leaf l is:

$$\text{Gini (Leaf } l) = 1 - p_l^2 - q_l^2.$$

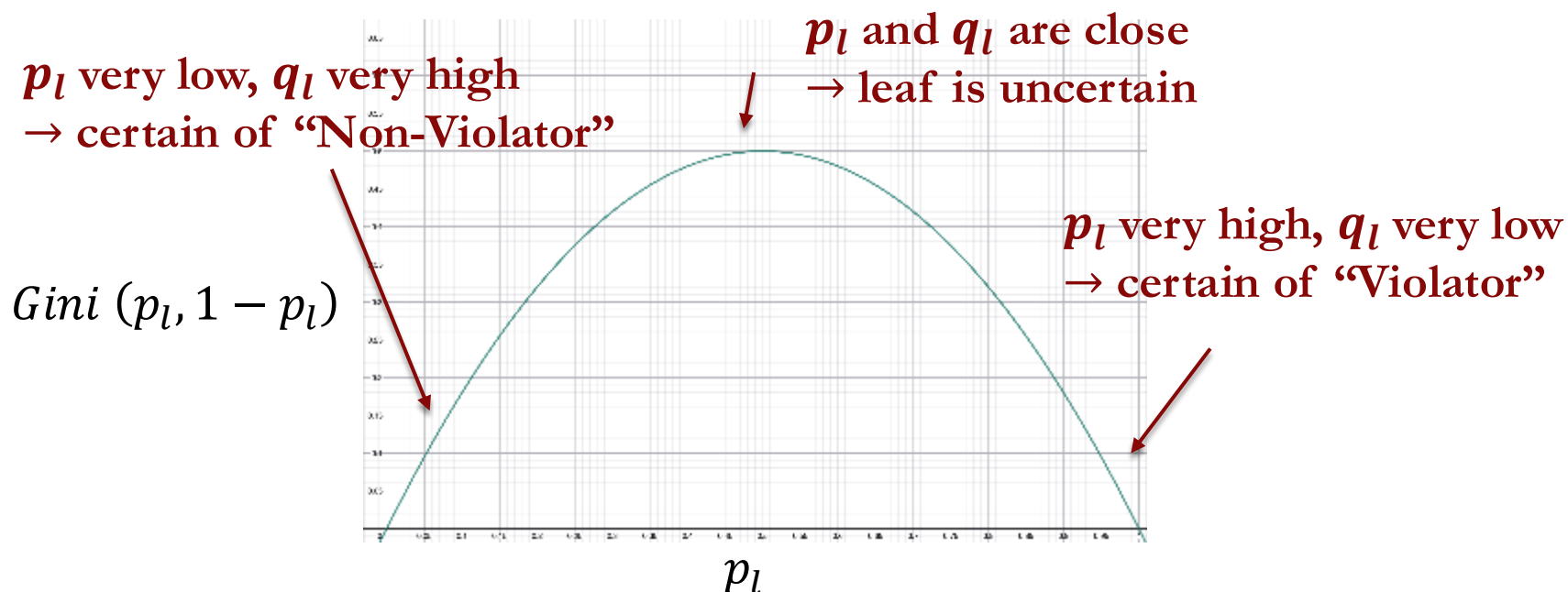
- It quantifies how *uncertain* the leaf is of its decision.

Gini index: Leaf Gini

- The expression of the *Gini index* of leaf l is:

$$\text{Gini}(p_l, q_l) = 1 - p_l^2 - q_l^2.$$

- It quantifies how *uncertain* the leaf is of its label.

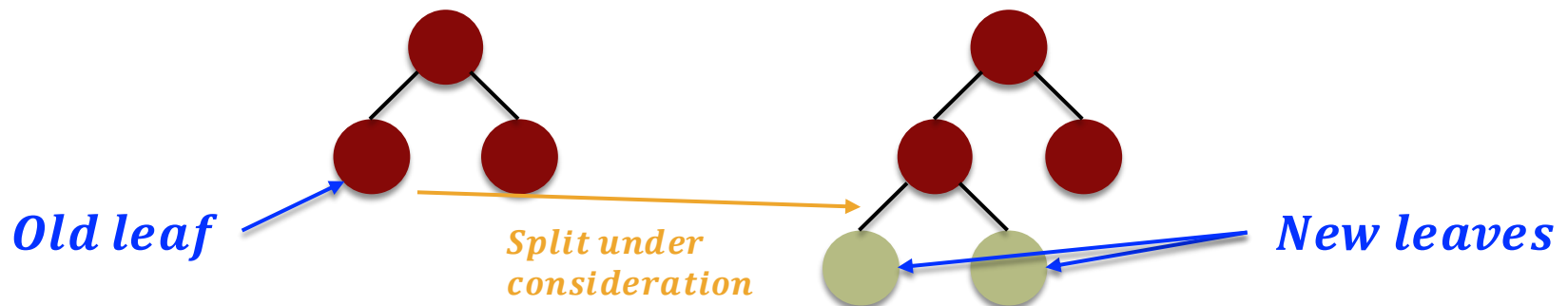


Split criterion: Gini reduction

- CART decides which split to make by computing the *Gini reduction* of each potential split:

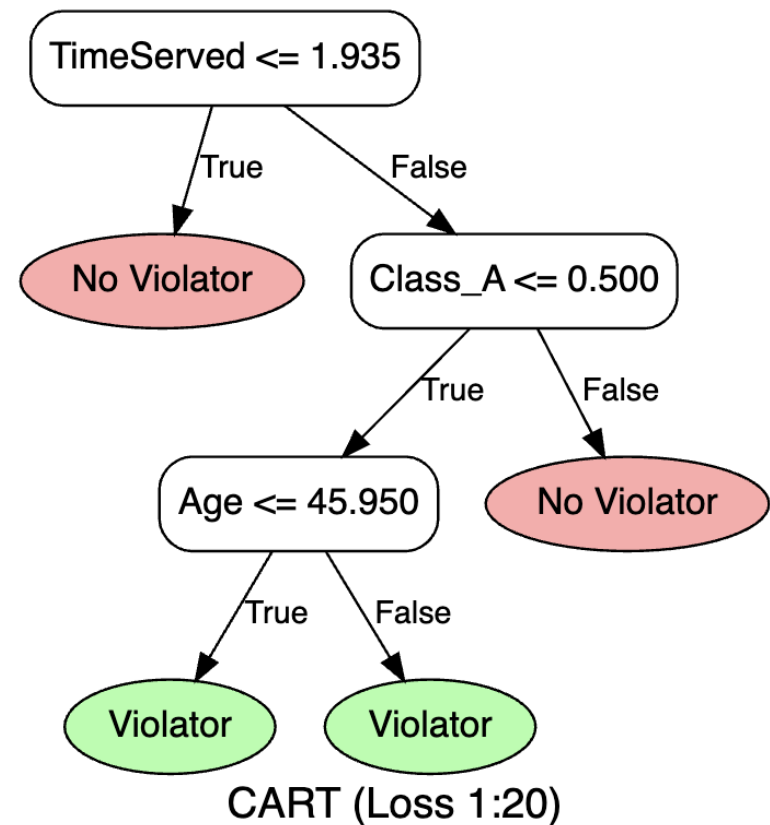
$$\begin{aligned} &\text{Gini reduction} \\ &= \\ &\text{weighted sum of Ginis of leaves} - \text{Gini of original leaf} \end{aligned}$$

- Iteratively, CART chooses the split with the *highest Gini Reduction*.

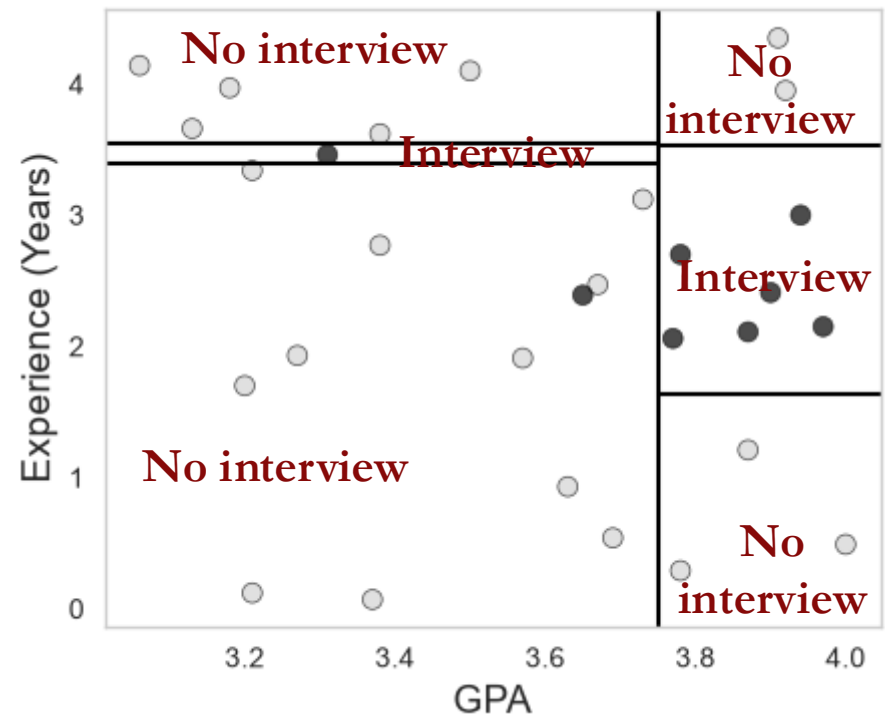
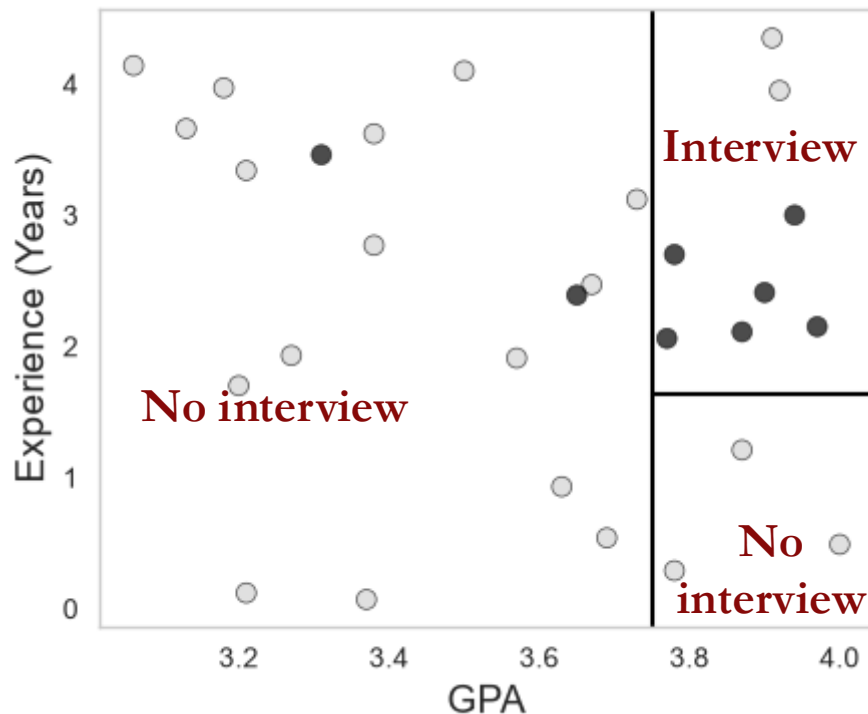


CART Model for Parole Data

- Here is the CART model we built for this classification problem
- $L_{FP} = 1$ and $L_{FN} = 20$
- `min_samples_leaf` = 25
- `ccp_alpha` = 0.01

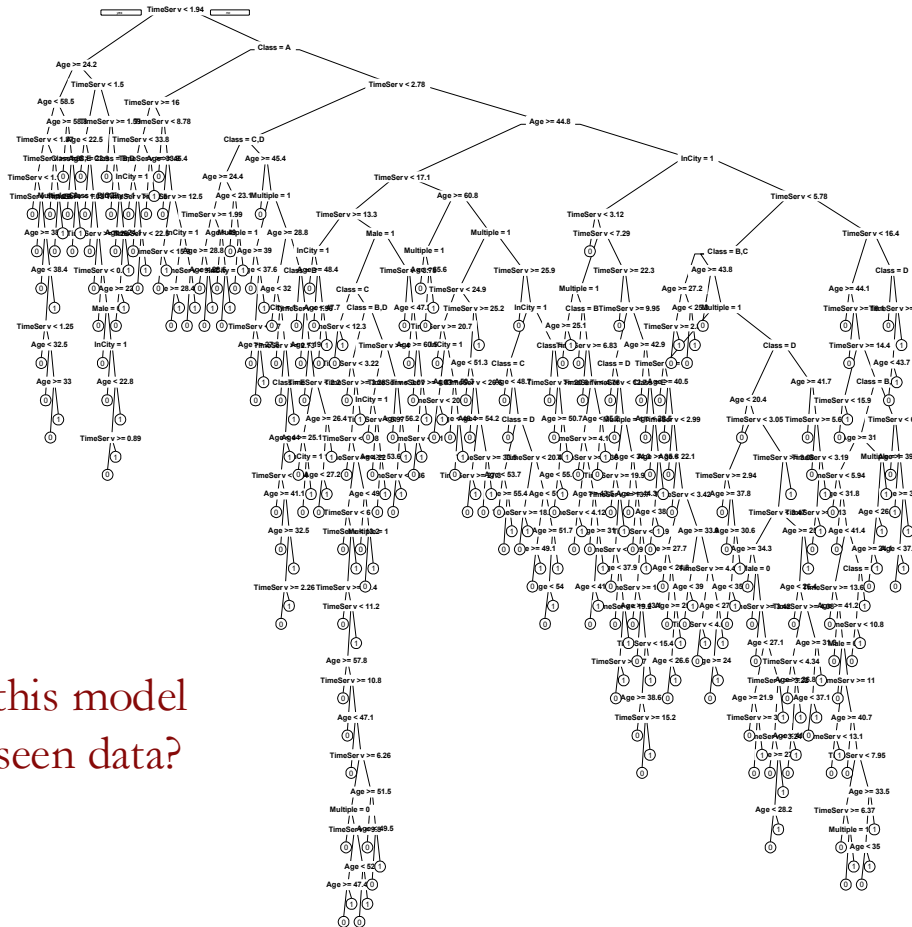


Two different CART models



- Which model is more accurate on the training data?
- Which model will be more accurate on the test data?

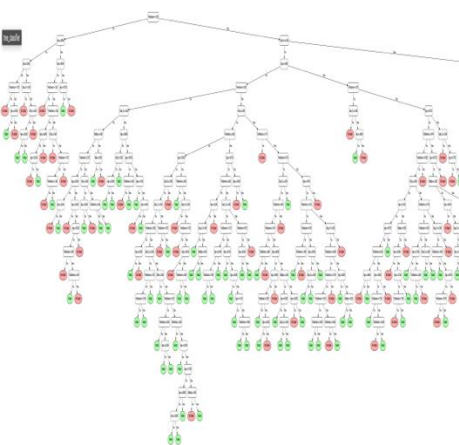
CART that perfectly fits the Training data



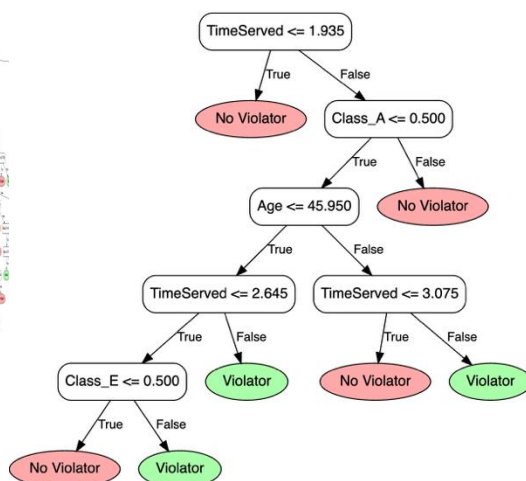
How do you think this model will perform on unseen data?

How do we find the best ccp_alpha?

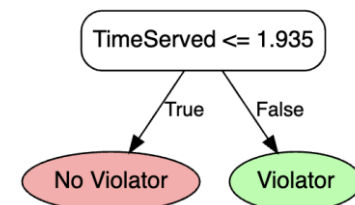
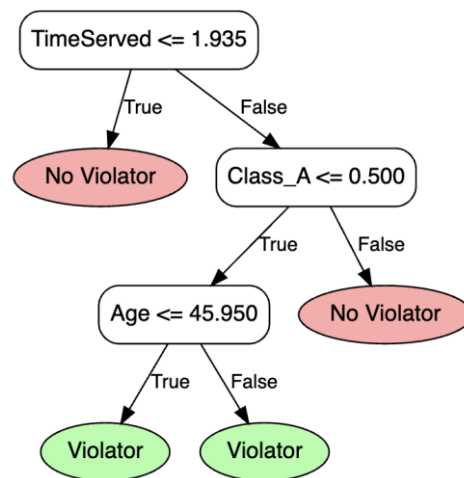
ccp_alpha = 0 ccp_alpha = 0.005 ccp_alpha = 0.01 ccp_alpha = 0.1



overfitting
the training
data?



sweet spot?



underfitting
the training
data?

How to predict: Total Loss (Cost)

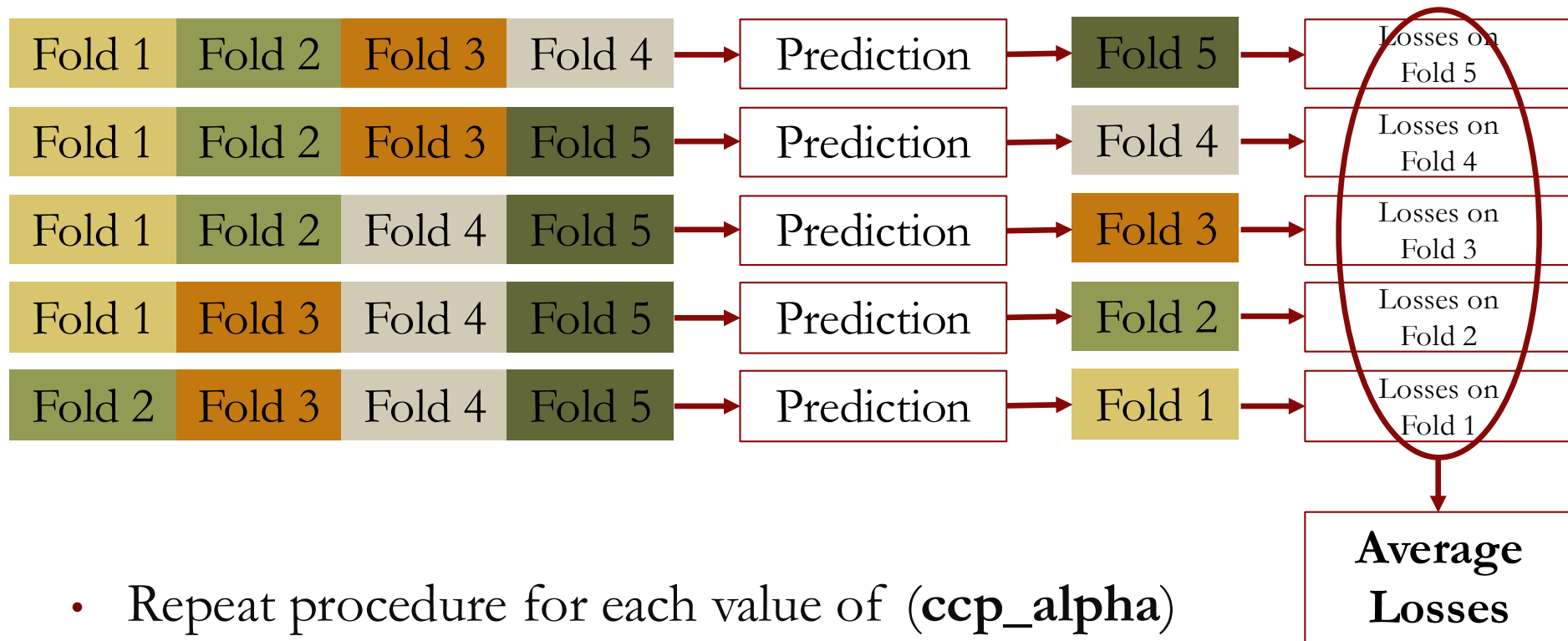
		Prediction/Decision	
		No Violation (Grant Parole)	Violation (Deny Parole)
Reality	No Violation	0	L_{FP}
	Violation	L_{FN}	0

- In CART for Classification, we measure the quality of the model using:

$$\text{Total Loss} = (\text{number of FN}) * \overset{\text{Cost of a single false negative (FN)}}{L_{FN}} + (\text{number of FP}) * \overset{\text{Cost of a single false positive (FP)}}{L_{FP}}$$

$$\text{Average Loss} = \frac{\text{Total Loss}}{\text{Number of Data Points}}$$

5-Fold Cross-Validation

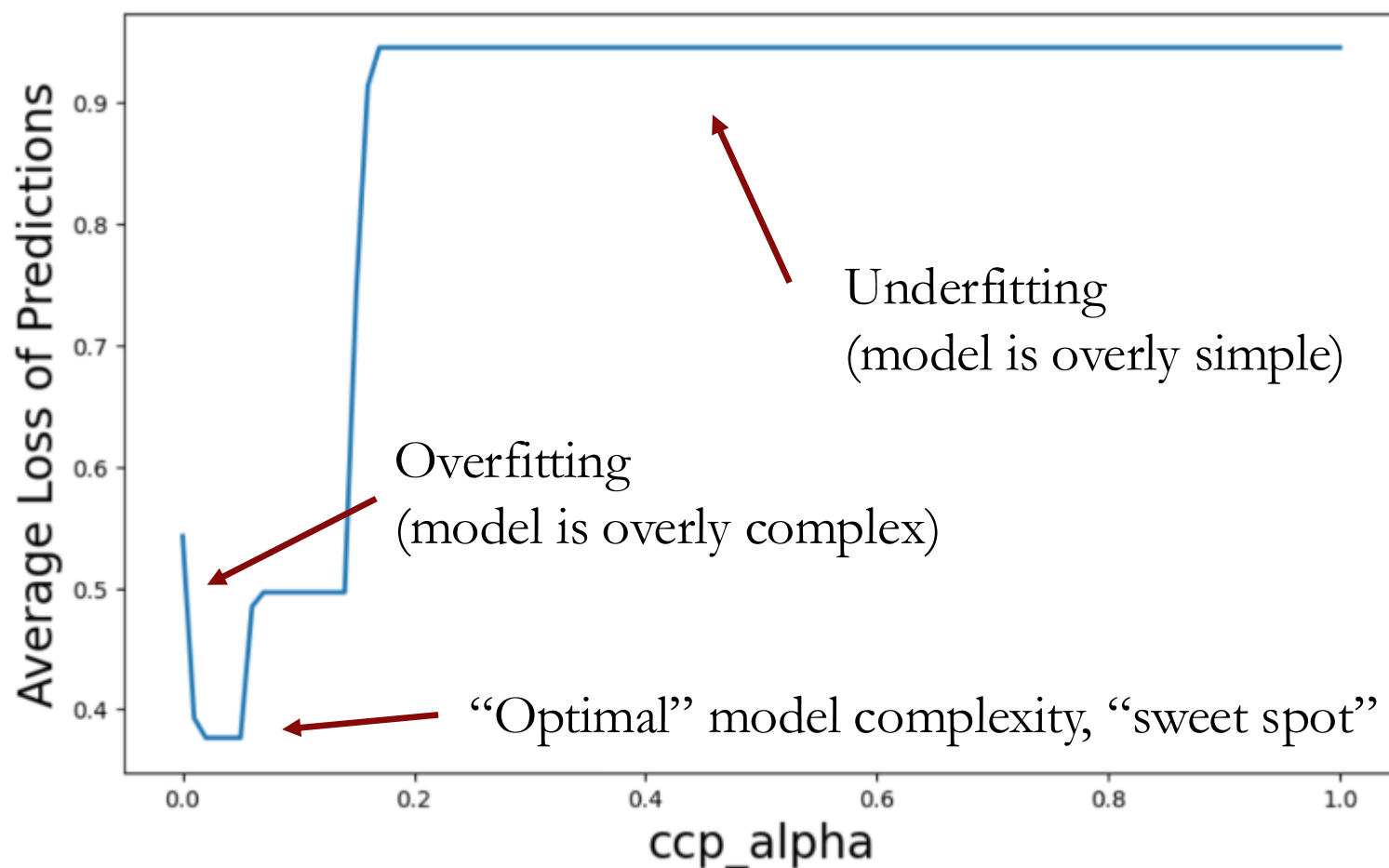


- Repeat procedure for each value of (**ccp_alpha**)
- Select the value of **ccp_alpha** that maximizes the **average loss**

Cross-Validation to choose **ccp_alpha**

- It is common to perform 5-fold or 10-fold cross-validation (we will perform 10-fold cross-validation here)
- One parameter is varied (**ccp_alpha**). **min_samples_leaf** is fixed throughout.
- We use our losses of $L_{FN} = 20$ and $L_{FP} = 1$
- For **ccp_alpha** = 0, 0.01, 0.02, 0.03, ..., 1 :
 - We will do cross-validation with the value of **ccp_alpha**.
 - We will record the average loss value over all folds. Call it **AvgLoss**.
- Finally, we will choose that value of **ccp_alpha** that achieves the lowest value of **AvgLoss**.
- Then fit a (final) tree on the full training data with this value of **ccp_alpha**.

Cross-Validation to select the best choice of `ccp_alpha`



Cross-Validation Results from R

Tested values Average performance

Select the best value

	ccp_alpha	AvgLoss (L=20)
0	0.00	0.542710
1	0.01	0.392636
2	0.02	0.376477
3	0.03	0.376477
4	0.04	0.376477
5	0.05	0.376477
6	0.06	0.484674
7	0.07	0.496364
8	0.08	0.496364
9	0.09	0.496364
10	0.10	0.496364
11	0.11	0.496364
12	0.12	0.496364
13	0.13	0.496364
14	0.14	0.496364
15	0.15	0.742734
16	0.16	0.913596
17	0.17	0.945212
18	0.18	0.945212
19	0.19	0.945212
20	0.20	0.945212
21	0.21	0.945212
22	0.22	0.945212
23	0.23	0.945212
24	0.24	0.945212

Python Code for Cross Validation

1. Define the estimator with the loss matrix:

```
dt_cv = DecisionTreeClassifier(  
    min_samples_leaf=5,  
    class_weight={0: 1, 1: L}  
)
```

2. Define the scoring function:

```
loss_scorer = make_scorer(lambda y_true, y_pred: avg_loss(y_true,  
    y_pred, L), greater_is_better=False)
```

3. Do cross validation on ccp_alpha:

```
cv.trees = GridSearchCV(  
    dt_cv,  
    param_grid = {'ccp_alpha': np.linspace(0, 1, 101)},  
    cv=StratifiedKFold(10),  
    scoring=loss_scorer,  
    n_jobs=-1  
)
```



PART I: RANDOM FORESTS

Predicting Click-Through Rates of Ad Impressions

Lecture Outline

1. Predicting Click-Through Rate in Online Advertisements
 - Why would an advertisement platform like Google want to predict the click-through rates of its ads?
2. Intro to Ensemble Learning and Random Forest
 - **2.1 Building a Random Forest**
 - How does a random forest model use randomness in its construction?
 - **2.2 Results on Real Data with Parameter Selection**
 - What parameters of a random forest model can we control?
 - **2.3 Trade-offs for Performance: Computation and Interpretability**
 - Why are random forest models computationally intensive to train? What are some ways to remedy this?
 - What information on feature importance does random forest provide?

1. Online Advertisements

Sponsored Search Advertising

Google

flowers

AI Mode All Images Shopping Short videos Videos Maps More Tools

Chapel Hill, NC Use precise location

Sponsored Stores

Venus et Fleur
4.8 ★ (3.6K) · 14-day returns



Jules Pink Eternity
Peonies in Fleura
Porcelain Vase | Flower...
\$649.00
By 9/24

Red Eternity Rose | Large
Round In Black Classic
Box | Flowers And Gifts ...
\$399.00
★★★★★ (583)

1800Flowers.com
4.2 ★ (7.4K)



Send Flowers | Same Day
Flower Delivery | Floral
Embrace | Large | 1-800...
\$74.99
★★★★★ (112)

Send Flowers | Same Day
Flower Delivery | Splendid
Beauty Bouquet | Large ...
\$74.99
By 9/16

UrbanStems
3.5 ★ (9.4K) · 3-day returns



Peach Anemones
Bouquet - Double the
Margot | Next-Day Flow...
\$162.00 \$160

Pink Roses Bouquet -
Triple The Verona | Next-
Day Flower Delivery wit...
\$183.00 \$218

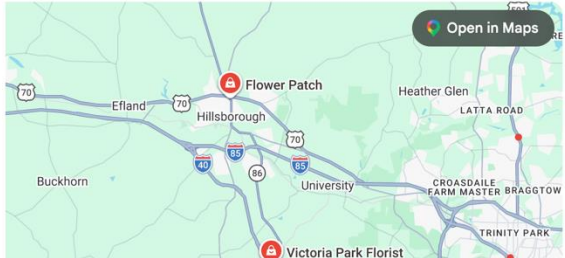
Places



Victoria Park Florist
5.0 ★★★★★ (180) · Florist
6113 NC-86 · (919) 967-9394
Open · Closes 6 PM
Sold here: **flowers**

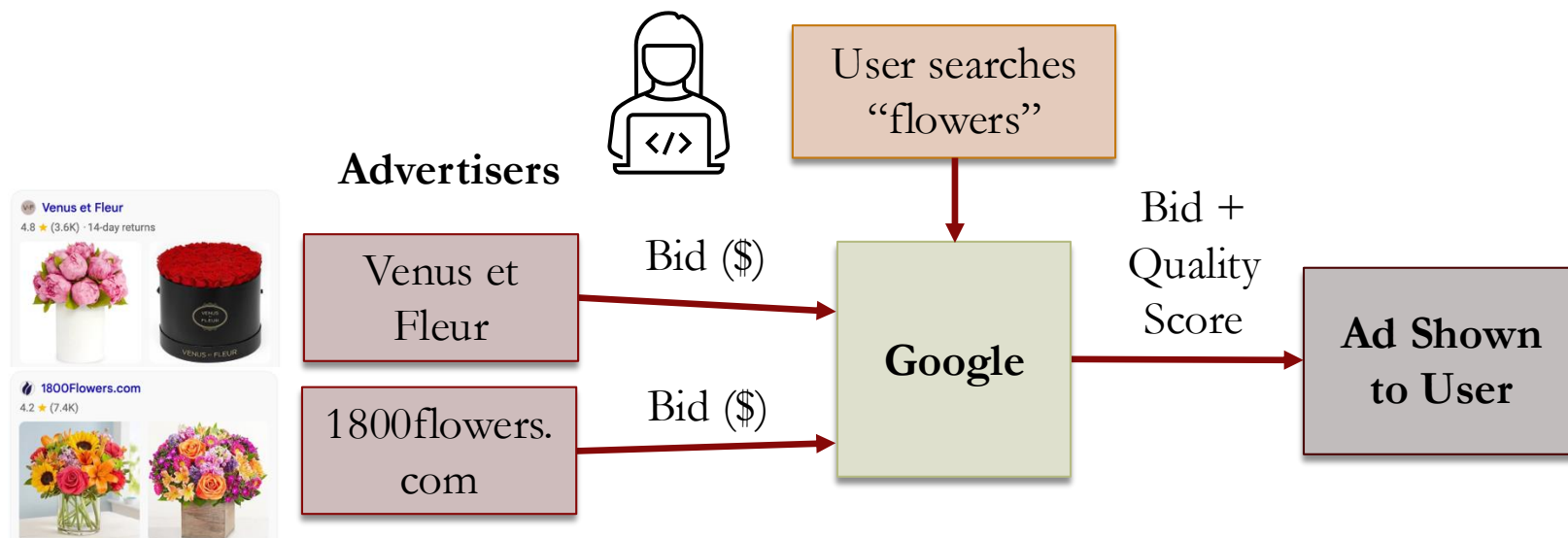


Chapel Hill Florist
4.9 ★★★★★ (929) · Florist
200 W Franklin St Ste 120 · (919) 929-2903
Closes soon · 4 PM
Seen by shoppers: **flowers**



Open in Maps

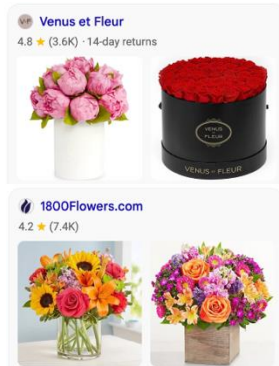
Sponsored Search Overview



- Advertisers create ads and submit to Google bids (\$), typically for a particular keyword search (e.g., “flowers”).
- Google determines which ads to show and in what order based on the bid and a **quality score**.
- Crucially, advertisers only pay if their ad is **clicked**.

Deciding which Ad to Show

Advertisers



Venus et
Fleur

1800flowers.
com



User searches
“flowers”

- Influenced by bid (\$) and quality score.
- The bid is how much an advertiser is willing to pay for a click.
- The quality score is dominated by the **click-through rate (CTR)** of the ad, an estimate of how likely the user will click this ad.
 - Other factors: advertiser targeting (e.g., user demographic, geography).
- **Final Decision:** Google shows the ad with the highest expected revenue (bid * CTR).

Example



Advertiser Name	Bid	CTR Estimate	Expected Revenue
Loreto's business "Petals"	\$0.10	0.10	\$0.0100
Clyde's business "Peonies"	\$0.11	0.08	\$0.0088

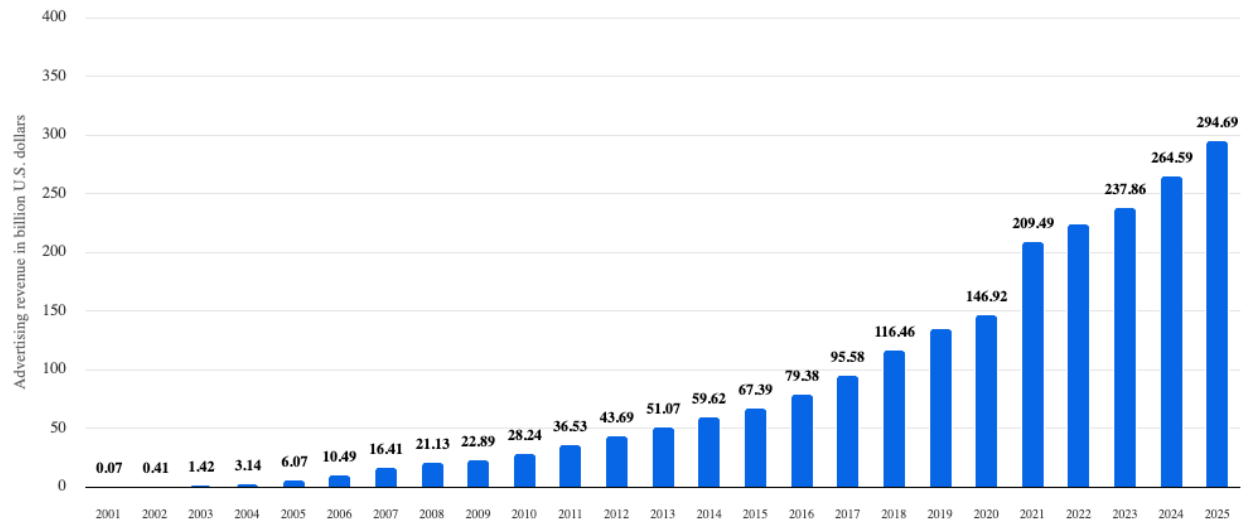
*In practice, there are multiple ads shown, so the position of the ad will also be factored in. Loreto may win position 1, and Clyde wins position 2. Google may find this more optimal than Loreto in position 2 and Clyde in position 1. We'll have a lecture on this!

Question: What are some negative consequences for Google if CTR's are not estimated correctly?

Google Advertising Revenue

- Google launched its advertisement platform (AdWords) in 2000.
- ~75% of Google's revenue

Advertising revenue of Google from 2001 to 2025 (in billion U.S. dollars)



Sources:
Alphabet, Google Inc.
@ Statista 2026

Additional information:
Worldwide; From 01/01/2001 to
31/12/2025

statista

Click-Through Rate Dataset

Click-Through Rate Dataset

- 6,057 new advertisements shown to specific groups of users in a specified position on the (former) Tencent search engine soso.com
- **Dependent Variable:** Click-through rate (CTR)
- **Independent Variables**
 - Number of words in the title and text of the advertisement
 - Depth (i.e., number of ads on the page) and the position of this ad
 - Average CTR of this advertiser's ads and of this query—both across all ads and across all ads in this position (4 metrics)
 - Gender and age of the users viewing the advertisement

Click-Through Rate (CTR) Data

k : # of independent variables ($k = 10$); n : # of observations ($n = 6,507$)

	CTR	titleWords	adWords	depth	position	advCTR	advCTRInPos	queryCTR	queryCTRInPos	gender	age
0	0.0	10	17	3	2	0.0136	0.0146	0.0	0.0	male	41+
1	0.0761	13	30	2	1	0.0373	0.0465	0.0382	0.0581	male	13-18
2	0.0426	12	14	1	1	0.0254	0.031	0.0255	0.0323	male	13-18
3	0.0	5	19	3	2	0.0178	0.0076	0.0035	0.0017	female	25-30
4	0.0068	11	17	2	2	0.0096	0.0069	0.0294	0.0171	female	19-24
5	0.0	15	28	2	2	0.0307	0.0157	0.0324	0.0137	female	31-40
6	0.0383	10	20	3	1	0.0283	0.0338	0.0219	0.0353	female	13-18
...	...	...	...	...	...	...	...	...	...	...	...
6052	0.0182	8	16	1	1	0.0273	0.0356	0.004	0.0	unknown	unknown
6053	0.0536	6	20	1	1	0.0919	0.0982	0.0	0.0	female	19-24
6054	0.0141	9	26	3	1	0.0496	0.0523	0.0467	0.0668	unknown	unknown
6055	0.1339	9	16	2	1	0.0566	0.061	0.0618	0.1012	unknown	unknown
6056	0.1322	12	23	2	1	0.043	0.062	0.0215	0.0287	unknown	unknown

Example Query with Ads

Google

flowers

AI Mode All Images Shopping Short videos Videos Maps More Tools

Chapel Hill, NC Use precise location

Sponsored Stores

Position 1

Position 2

Position 3

Query Depth = 3

Venus et Fleur
4.8 ★ (3.6K) · 14-day returns

Jules Pink Eternity Peonies in Fleura Porcelain Vase | Flower...
\$649.00
By 9/24

Red Eternity Rose | Large Round In Black Classic Box | Flowers And Gifts ...
\$399.00
★★★★★ (583)

1800Flowers.com
4.2 ★ (7.4K)

Send Flowers | Same Day Flower Delivery | Floral Embrace | Large | 1-800...
\$74.99
★★★★★ (112)

Send Flowers | Same Day Flower Delivery | Splendid Beauty Bouquet | Large ...
\$74.99
By 9/16

UrbanStems
3.5 ★ (9.4K) · 3-day returns

Peach Anemones Bouquet - Double the Margot | Next-Day Flow...
\$162.00 \$180

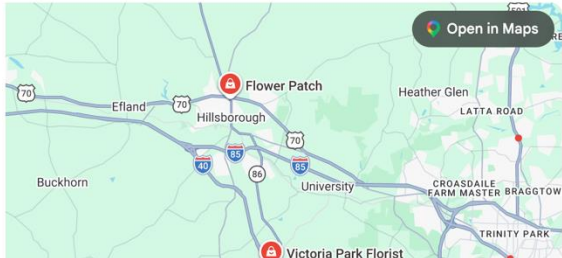
Pink Roses Bouquet - Triple The Verona | Next-Day Flower Delivery wit...
\$183.00 \$218

Places

Victoria Park Florist
5.0 ★★★★★ (180) · Florist
6113 NC-86 · (919) 967-9394
Open · Closes 6 PM
Sold here: **flowers**

Chapel Hill Florist
4.9 ★★★★★ (929) · Florist
200 W Franklin St Ste 120 · (919) 929-2903
Closes soon · 4 PM
Seen by shoppers: **flowers**

Open in Maps



Training/Testing data split

Full Dataset ($n = 6,057$)

	CTR	titleWords	adWords	depth
0	0.0	10	17	3
1	0.0761	13	30	2
2	0.0426	12	14	1
3	0.0	5	19	3
4	0.0068	11	17	2
5	0.0	15	28	2
6	0.0383	10	20	3
7	0.0172	9	23	3
...	...	...	...	...
6055	0.1339	9	16	2
6056	0.1322	12	23	2

70% Training Set ($n = 4,239$)

	CTR	titleWords	adWords	depth
2	0.0426	12	14	1
3	0.0	5	19	3
4	0.0068	11	17	2
5	0.0	15	28	2
...	...	...	...	...
6052	0.0182	8	16	1
6054	0.0141	9	26	3

30% Testing Set ($n = 1,818$)

	CTR	titleWords	adWords	depth
0	0.0	10	17	3
1	0.0761	13	30	2
8	0.1015	8	20	2
14	0.025	9	23	2
...	...	...	...	...
6055	0.1339	9	16	2
6056	0.1322	12	23	2

Linear Regression and CART models

Variable	Linear regression
titleWords	-0.00039
adWords	0.000195
depth	0.000572
position	-0.001295
advCTR	0.272312
advCTRInPos	0.702157
queryCTR	-0.072609
queryCTRInPos	0.565682
gendermale	-0.008656
genderunknown	-0.048420
age13-18	-0.006917
age19-24	-0.006791
age25-30	-0.006079
age31-40	-0.007717
age41+	0.005114
ageunknown	0.027482

CART (for regression)



- Somewhat interpretable
- But do you think Google would prioritize interpretability of their model here?
- Or do they simply care about maximizing predictive ability?

Linear Regression and CART models, cont.

Metric	Linear Regression	CART
OSR ²	0.3961	0.3377
MAE	0.0331	0.0340
RMSE	0.0551	0.0577

Mean Absolute Error (MAE):

$$MAE = \frac{\sum_{i \in \text{Test Set}} |y_i - \hat{y}_i|}{\# \text{ observations in Test Set}}$$

Root Mean Square Error (RMSE):

$$RMSE = \sqrt{\frac{\sum_{i \in \text{Test Set}} (y_i - \hat{y}_i)^2}{\# \text{ observations in Test Set}}}$$

2. Ensemble Learning and RF

Combining multiple models

Wisdom of Crowds

- 1907: F. Galton examined 787 predictions of the weight of an ox
 - Mean prediction: 1,197 pounds
 - Median prediction: 1,207 pounds
 - Actual weight: 1,198 pounds
- *Who Wants to be a Millionaire*
 - The friend (“expert”) is correct 65% of the time
 - The audience (“crowd”) is correct 91% of the time
- **Wisdom of crowds:** aggregate predictions of a group often outperform individual predictions



Wisdom of the Crowd Activity

- How many pieces of candy are in this jar?



Wisdom of the Crowd Activity

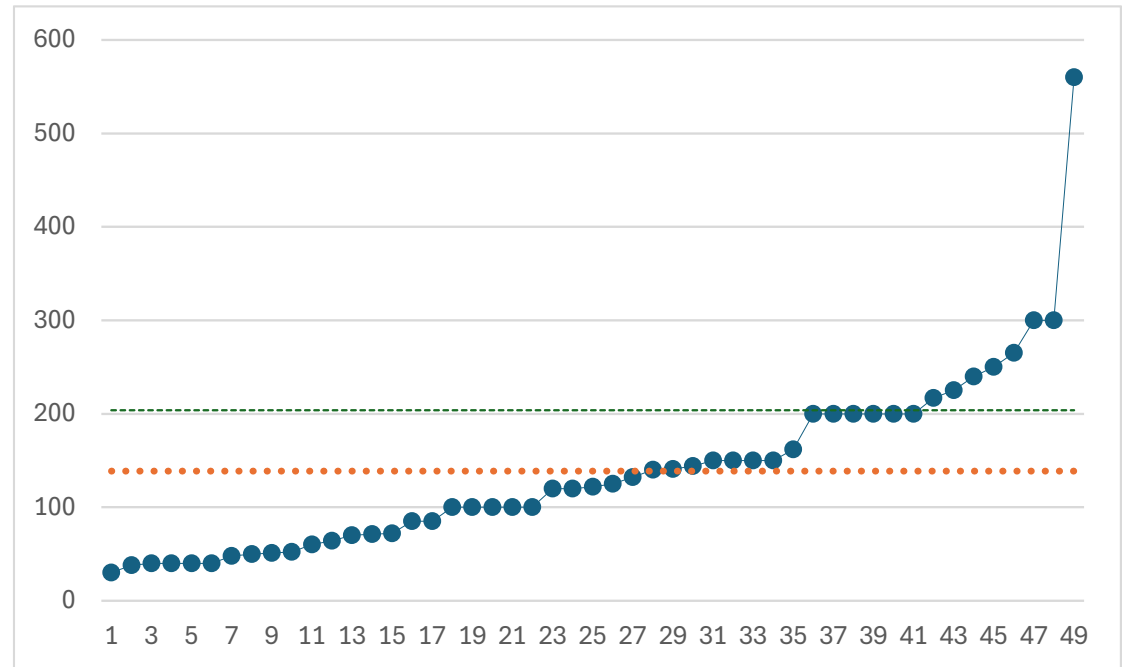


- How many pieces of candy are in this jar?

Wisdom of the Crowd Activity

- 49 responses
 - Average: 138.75
 - True: 204
 - 30 responses
- “worse” than average

- Winners:
 - Arohi Singh
 - Rajitha Penaka
 - Jake Readinger
 - Shakthi Velan Mariappan Veerashanmughamoni
 - Nicolas Aldunate
 - Fernando Bonilla



Ensemble Learning Principles

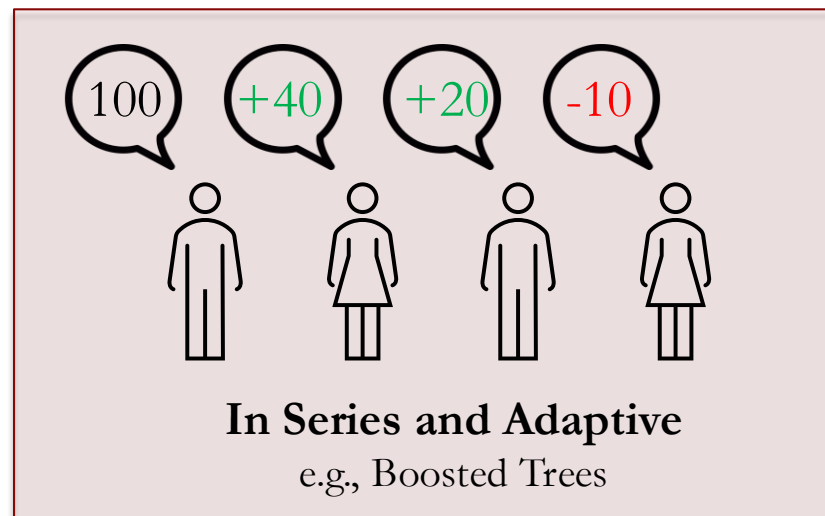
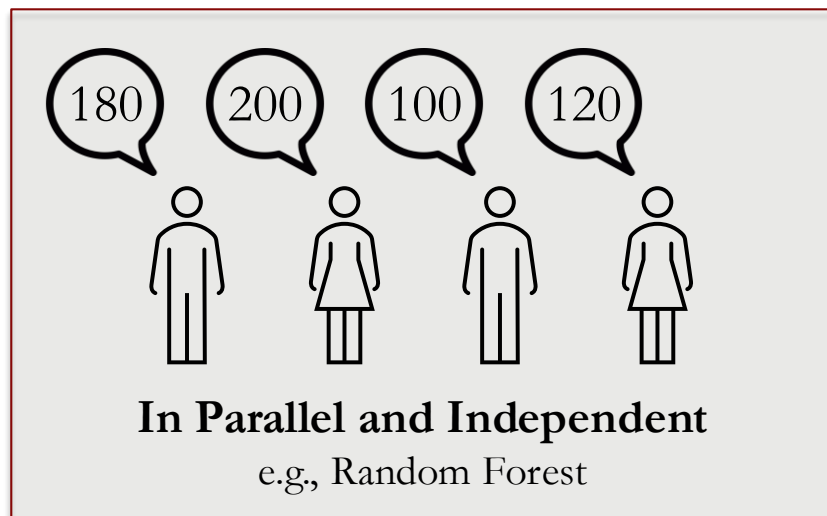
- Ensemble learning: *build* multiple prediction models and *combine* their predictions (for example by averaging)
 - **Different strengths and weaknesses.** Models complement each other and work together to make a prediction.
 - **Reduces overfitting:** a single model may overfit in one way, but not all models will overfit in the same way.
- Advantages and Disadvantages
 - **Pros: Strong performance.** Routinely used by winners of machine-learning competitions (e.g., Kaggle)
 - **Cons: Interpretability and Computation.** Even though each model may be interpretable, the ensemble typically is not.

Ensemble Learning Paradigms

- **Models in Parallel:** Build models independently, then make predictions by taking an average or majority vote.
- **Models in Series:** Build models adaptively, where each model is trained to fix the mistakes of the previous models.



Number of candies?



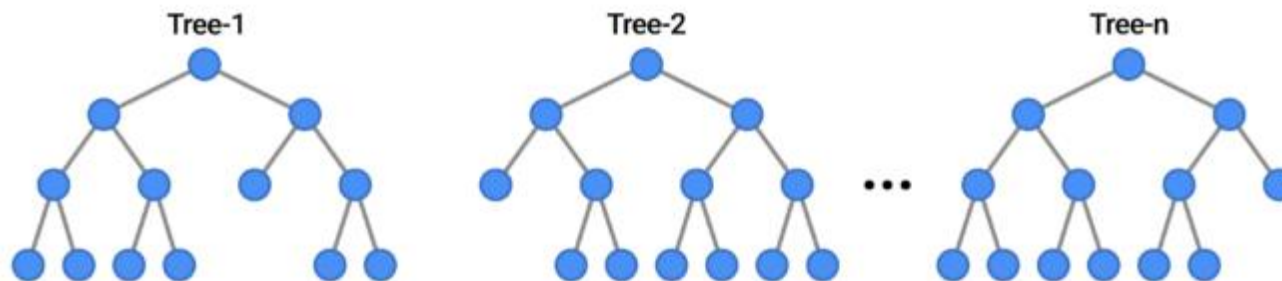
Warning! Science vs. Practice

- As we search for better and better models, we sometimes discover a model type which works amazingly in practice, but whose scientific basis is not well-understood.
 - Why does this modification work so well?
 - Why don't we do this another way? Try it!
- Have we really found an excellent new type of model, or are we just (consistently) lucky?
- After finding a model type that consistently works well, we start to “believe” it is inherently a great new discovery.

2. Intro to Random Forest

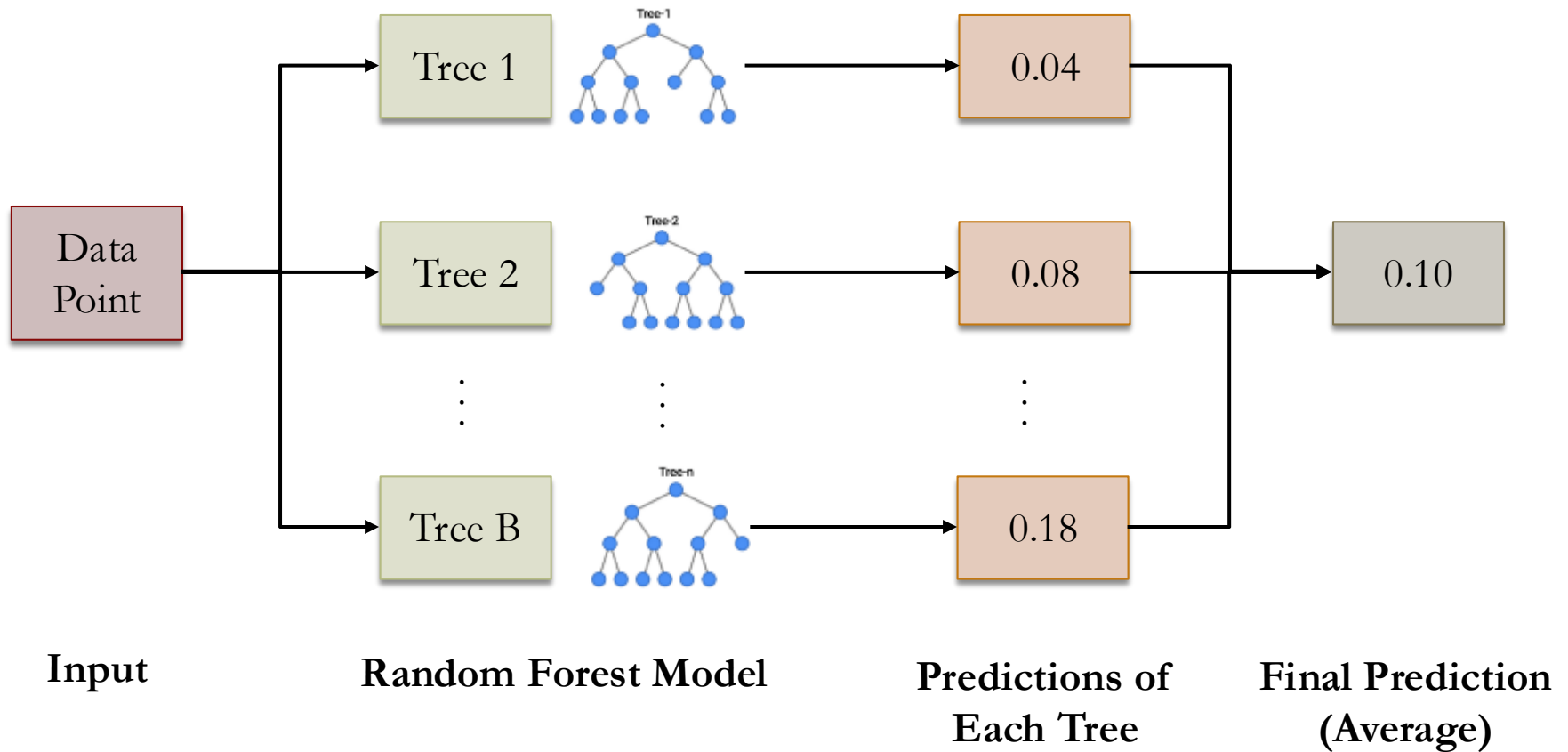
A collection of independent CART models.

Random Forest Overview

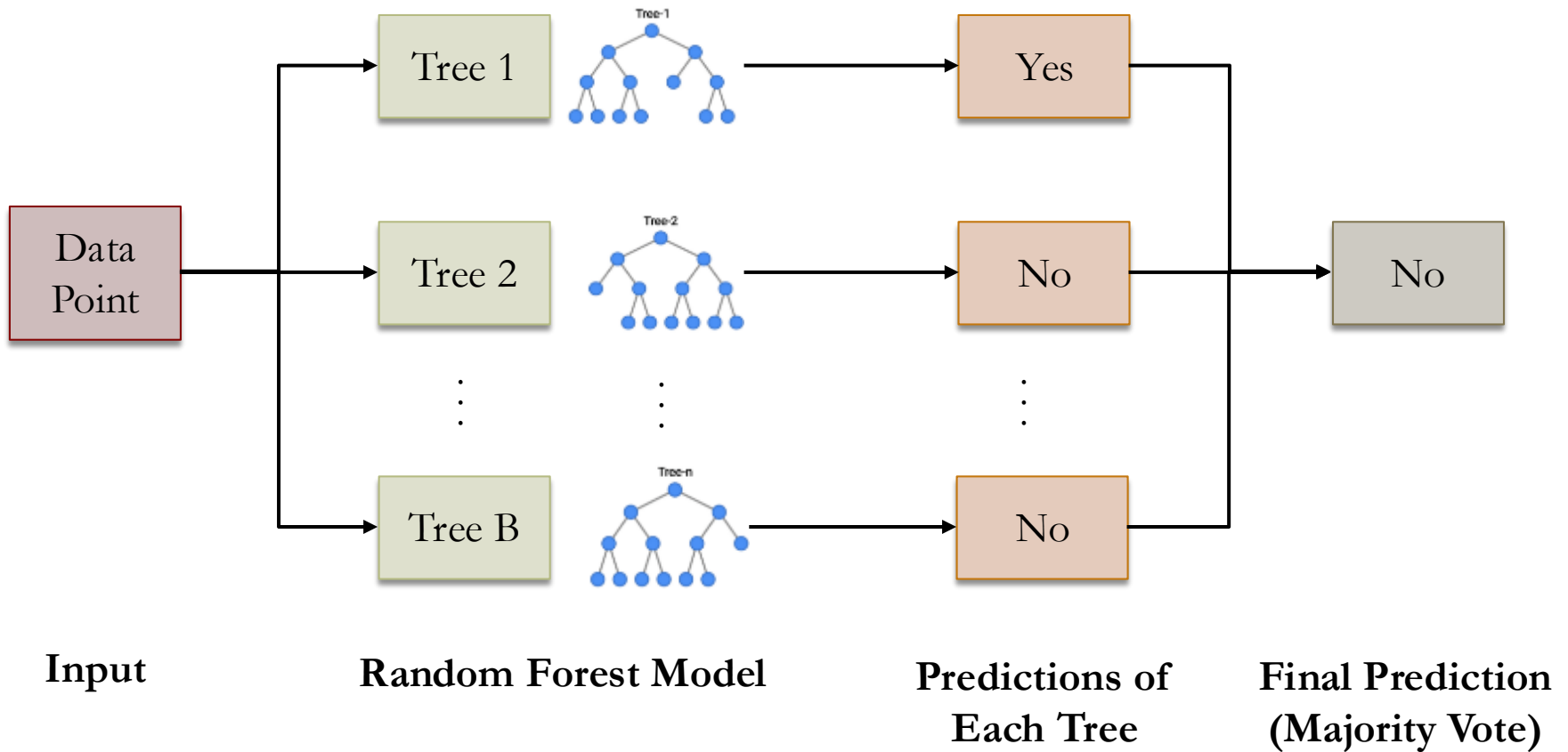


- Random Forest is a collection of CART trees (hence called a forest).
- Predictions are made by taking the average (regression) or majority vote (classification) of each tree's prediction.
- The forest of trees is built using some **randomization**, which ensures trees are diverse.
 1. **Random Training Data (Bootstrapping)** – Creating different training datasets for each tree by sampling.
 2. **Random Splits** – Restricting each split to a subset of the independent variables.

Making Predictions (Regression)



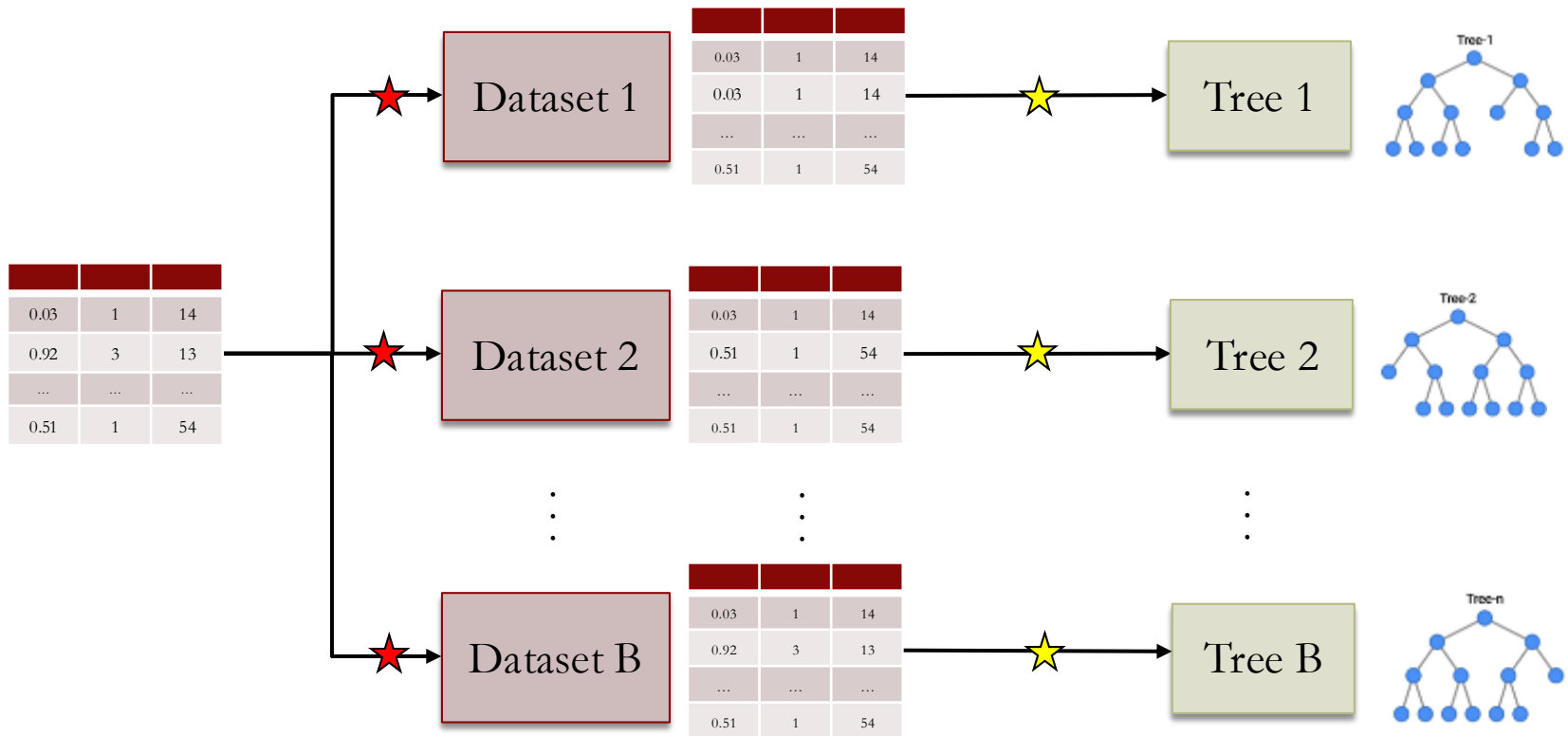
Making Predictions (Classification)



2.1 Building a Forest with Randomness

What's so “random” about random forest?

Building the Forest with Randomness



**Training
Dataset**

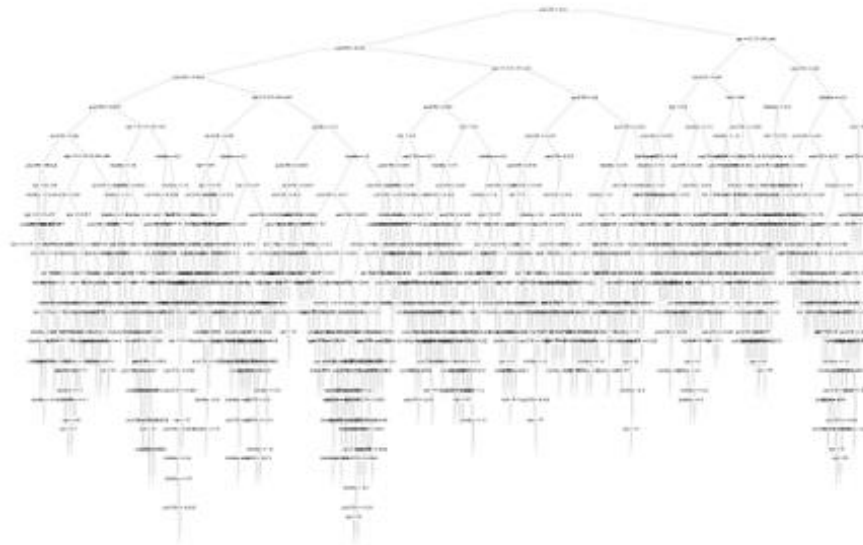
**Create Random “New”
Datasets (Bootstrapping)**

**Build CART with
Randomized Splits**

**Random Forest
Model**

Each CART Model is Very Complex

- Unlike CART, there is no pruning of trees based on a complexity parameter **ccp_alpha**
 - As a result, each fitted tree is usually very large and very complicated
 - We will see how randomization reduces the risk of overfitting



Randomizing the Training Datasets

Bootstrapping and Bagging

1. Bootstrapping

Bootstrap: Sample **with replacement** from the training set, selecting the same number of observations as in the training set

As a result, some rows may appear more than once, and other rows may not appear at all

	titleWords	adWords	depth	position	advCTR	advCTRInPos	
→ → →	2	12	14	1	1	0.0254	0.031
→	3	5	19	3	2	0.0178	0.0076
→ → →	4	11	17	2	2	0.0096	0.0069
	5	15	28	2	2	0.0307	0.0157
	6	10	20	3	1	0.0283	0.0338
	7	9	23	3	1	0.0054	0.0084
	9	11	21	2	1	0.018	0.0314
	...	...	...	...	...	...	...
→ →	6051	10	15	3	2	0.0292	0.0303
	6052	8	16	1	1	0.0273	0.0356
→	6054	9	26	3	1	0.0496	0.0523

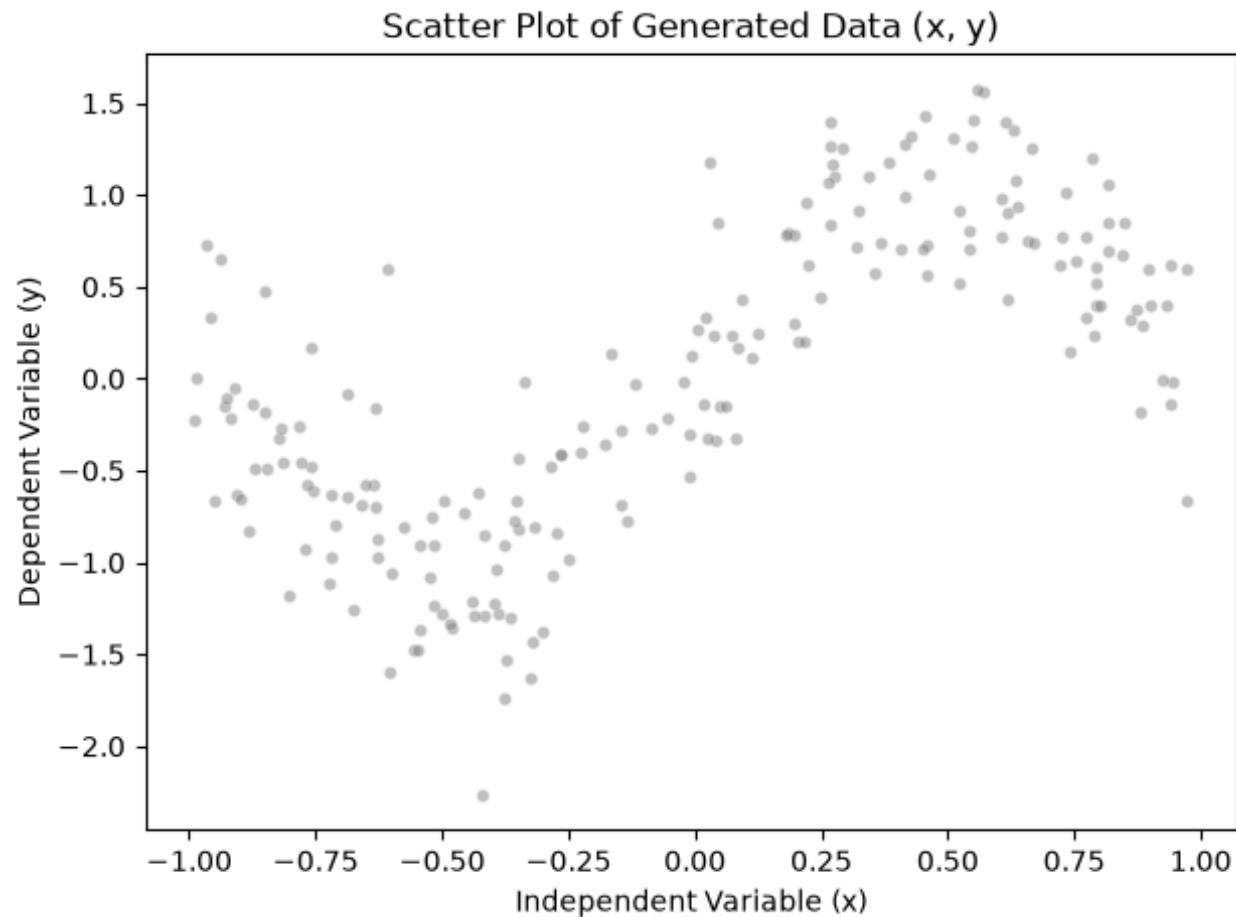
	titleWords	adWords	depth	position	advCTR	advCTRInPos	
	2	12	14	1	1	0.0254	0.031
	2	12	14	1	1	0.0254	0.031
	2	12	14	1	1	0.0254	0.031
	3	5	19	3	2	0.0178	0.0076
	4	11	17	2	2	0.0096	0.0069
	4	11	17	2	2	0.0096	0.0069
	4	11	17	2	2	0.0096	0.0069
	...	...	...	...	...	...	...
	6051	10	15	3	2	0.0292	0.0303
	6051	10	15	3	2	0.0292	0.0303
	6054	9	26	3	1	0.0496	0.0523

Bagging = Bootstrap aggregation

Bootstrap aggregation, or Bagging, is a general ensemble learning principle for creating diverse predictors

- Generate B different bootstrapped training sets (think $B = 100$)
- Train your favorite predictive analytics (i.e., machine learning) method on each of the B bootstrapped training sets
- Aggregate the results by taking the average (regression) or majority vote (classification) of the B predicted values.

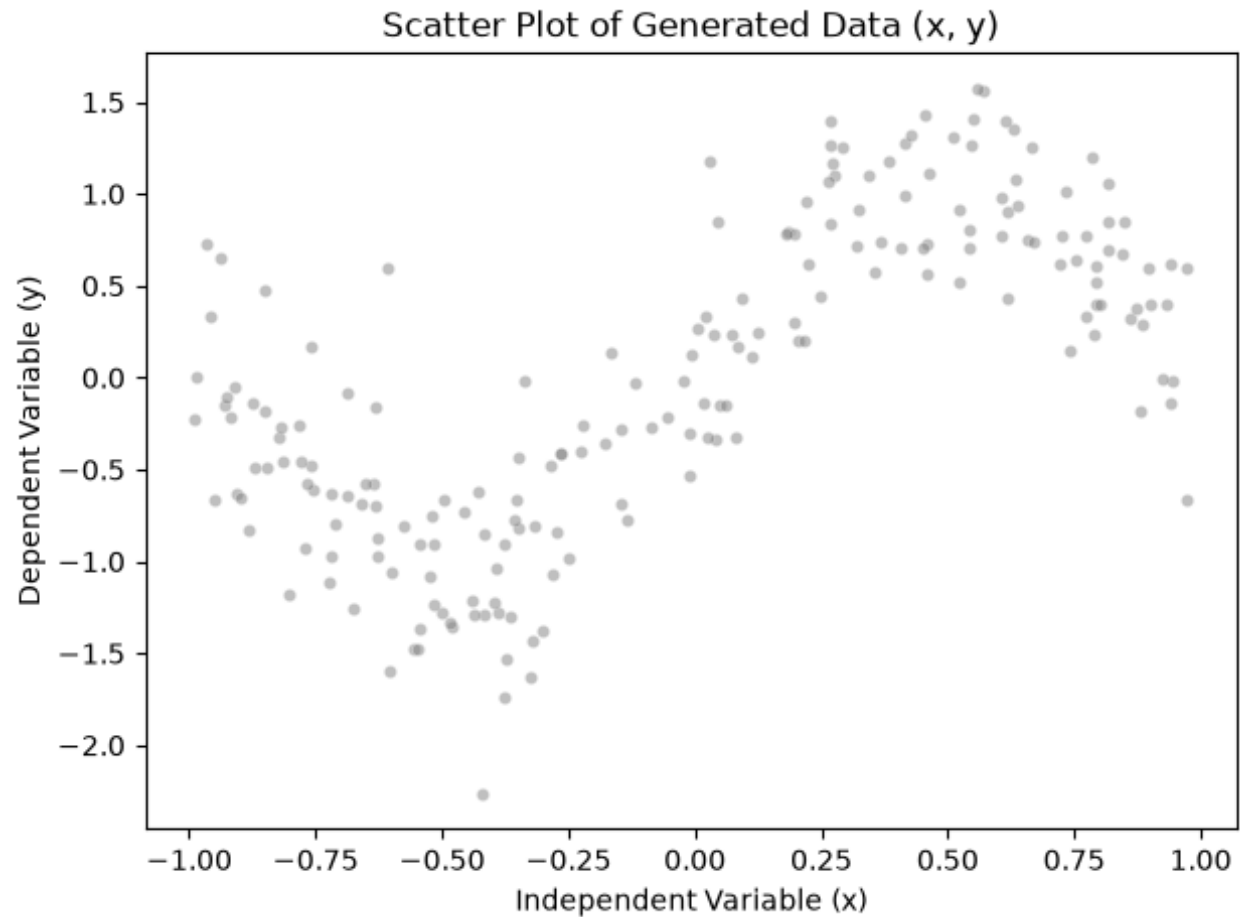
Demonstration of Bagging



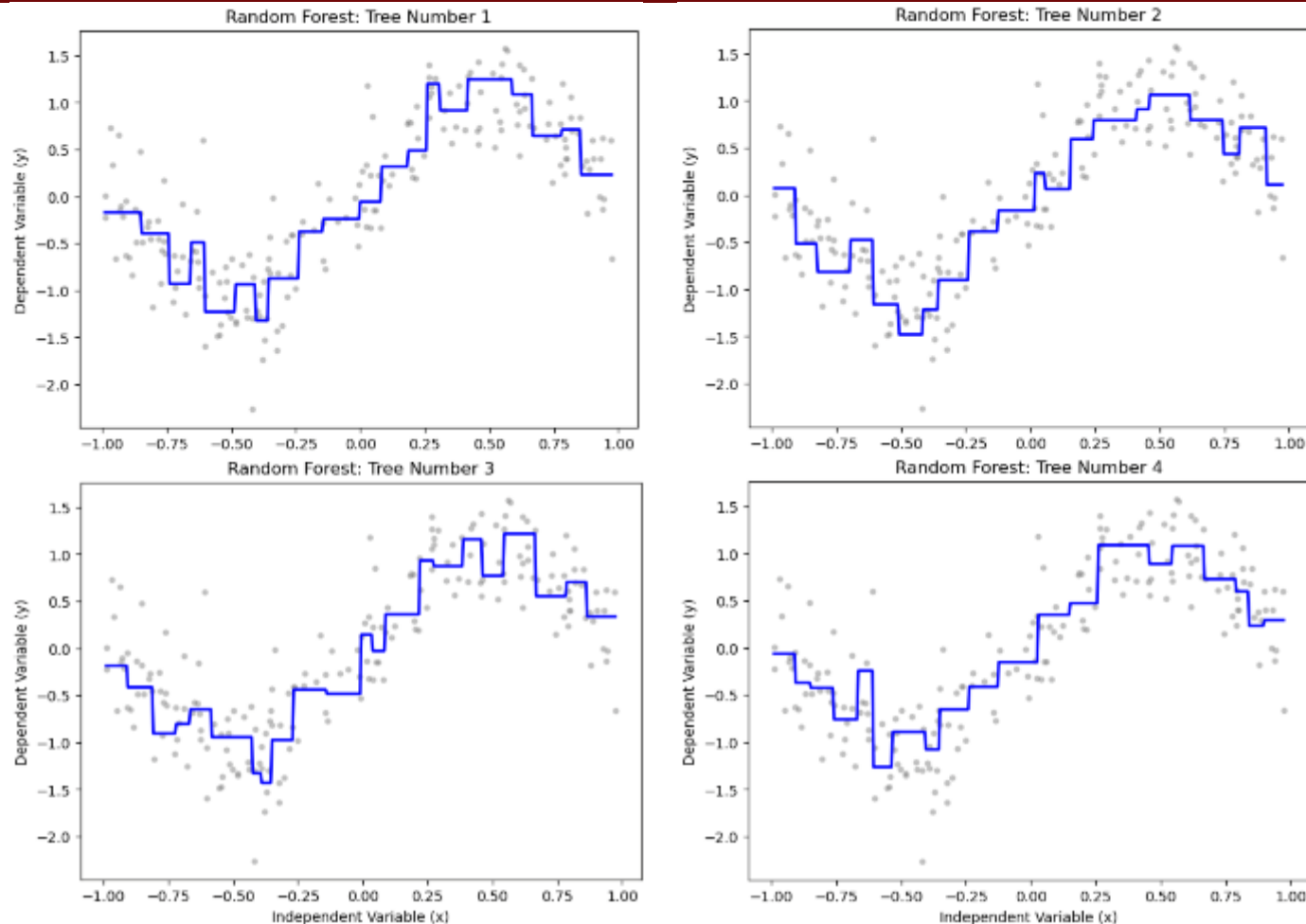
Demonstration of Bagging

- 2 Variables: x (independent variable) and y (dependent variable).
- Vary the number of trees from 1 to 100.
- Each tree is fit on a bootstrapped sample of the training data.
- The fit is generally good, but is quite “jagged” because of overfitting.
- As the number of trees increases, the curve smooths out significantly.

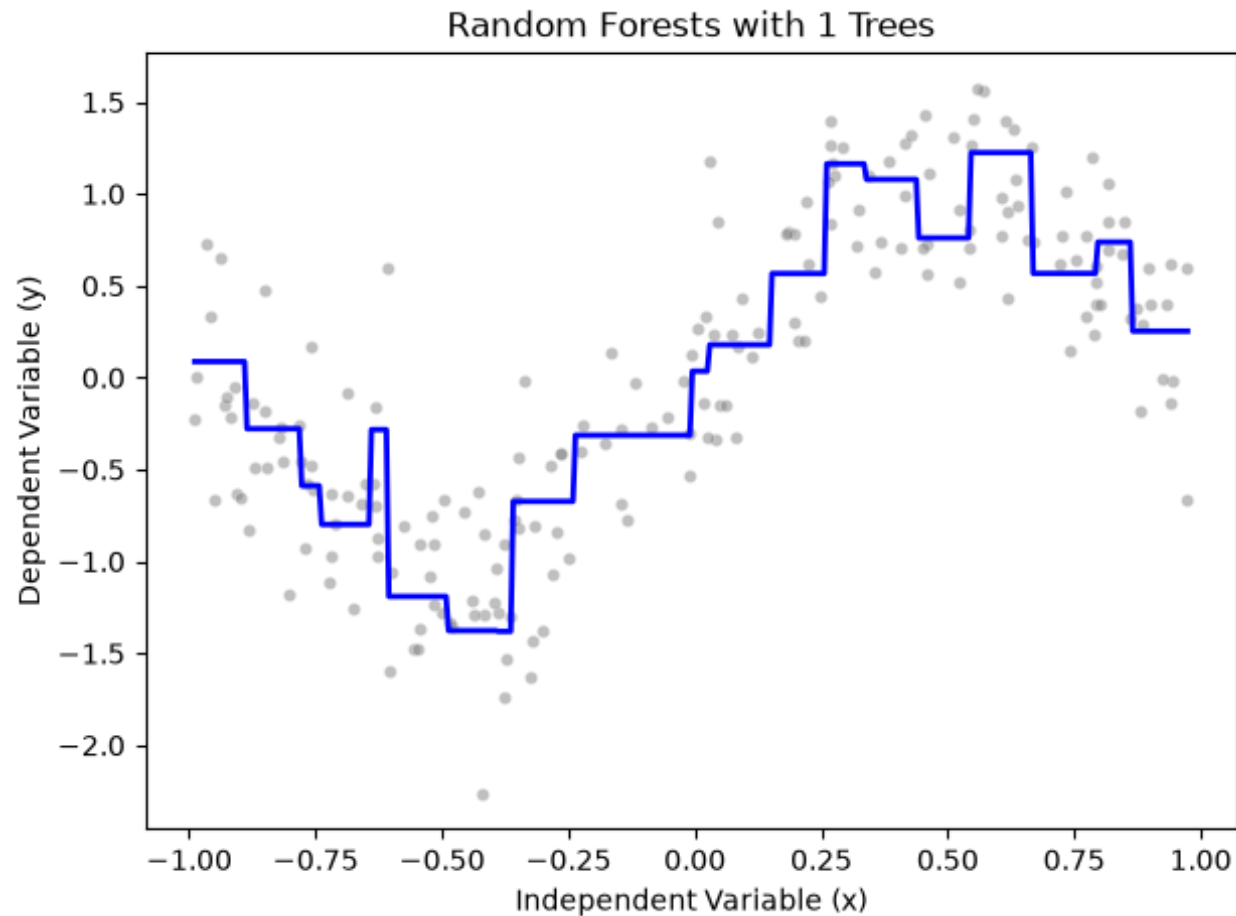
Demonstration



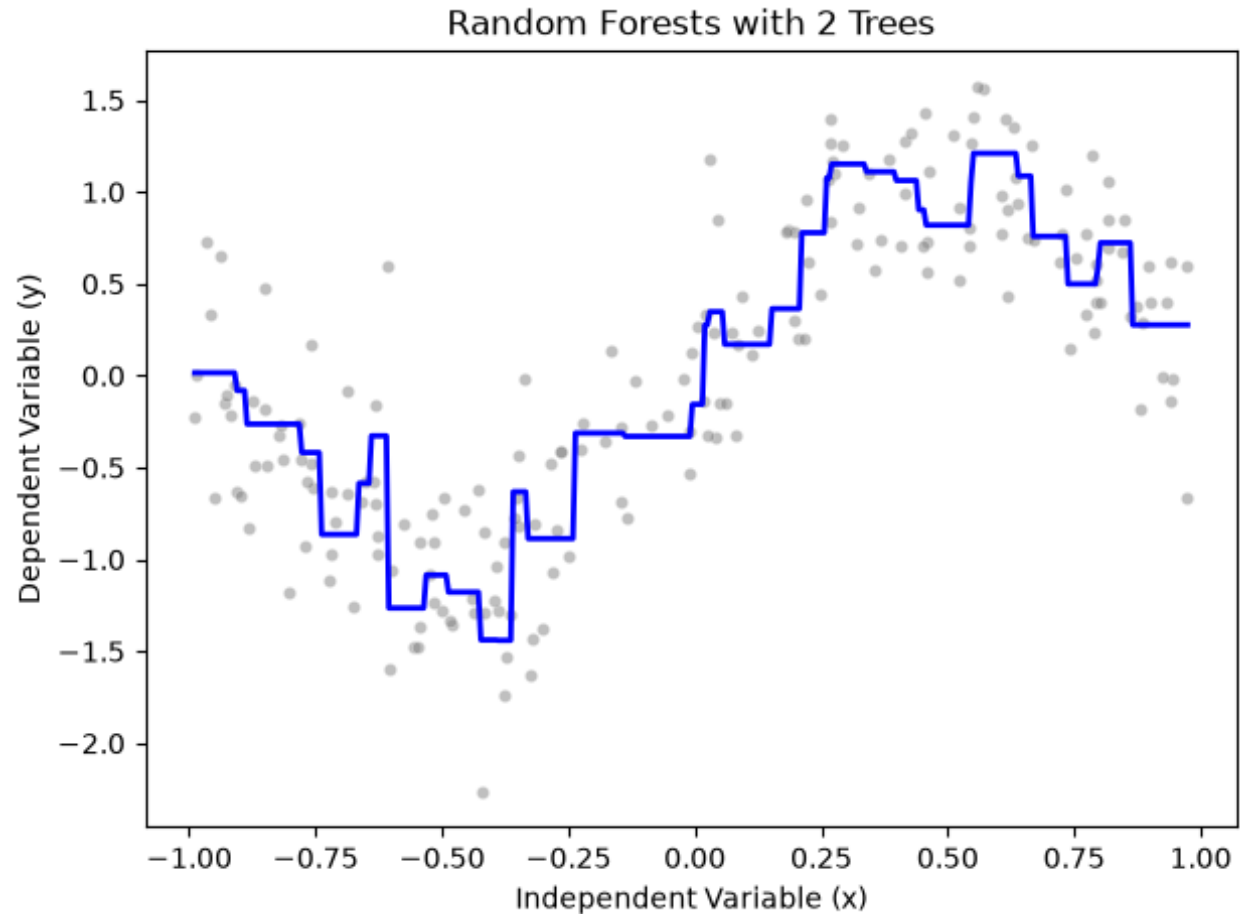
Random Forest: Individual Trees



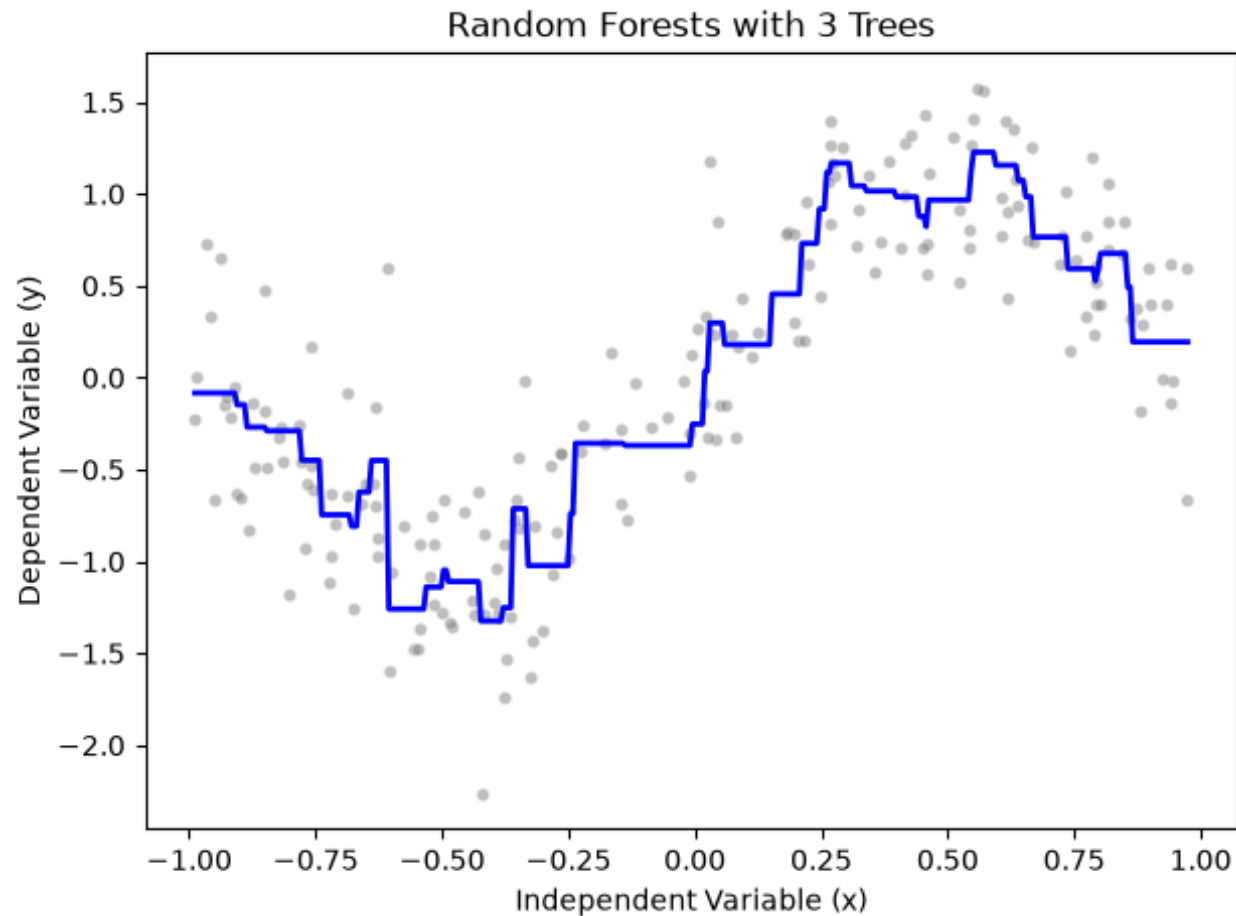
Random Forest: Tree 1 Only



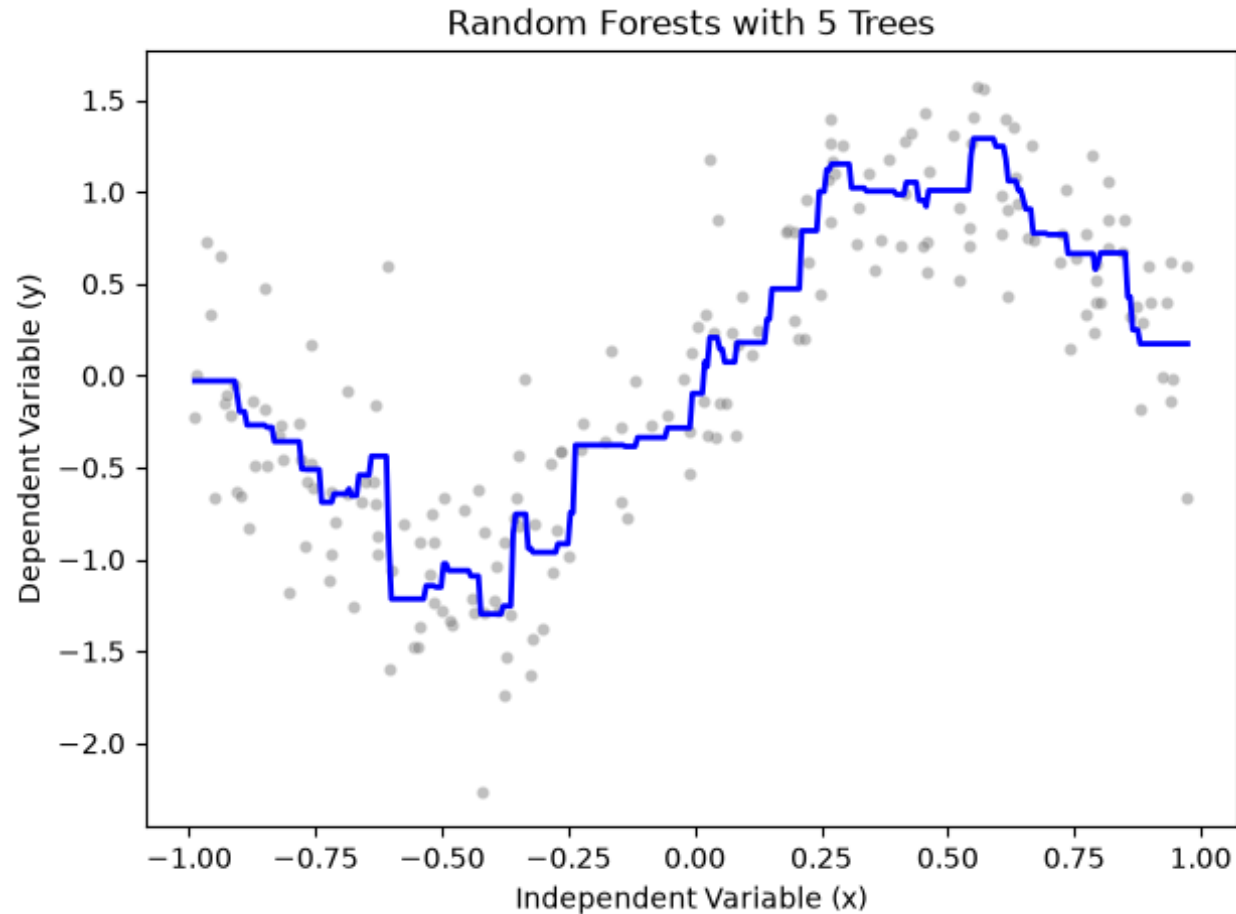
Random Forest: 2 Trees



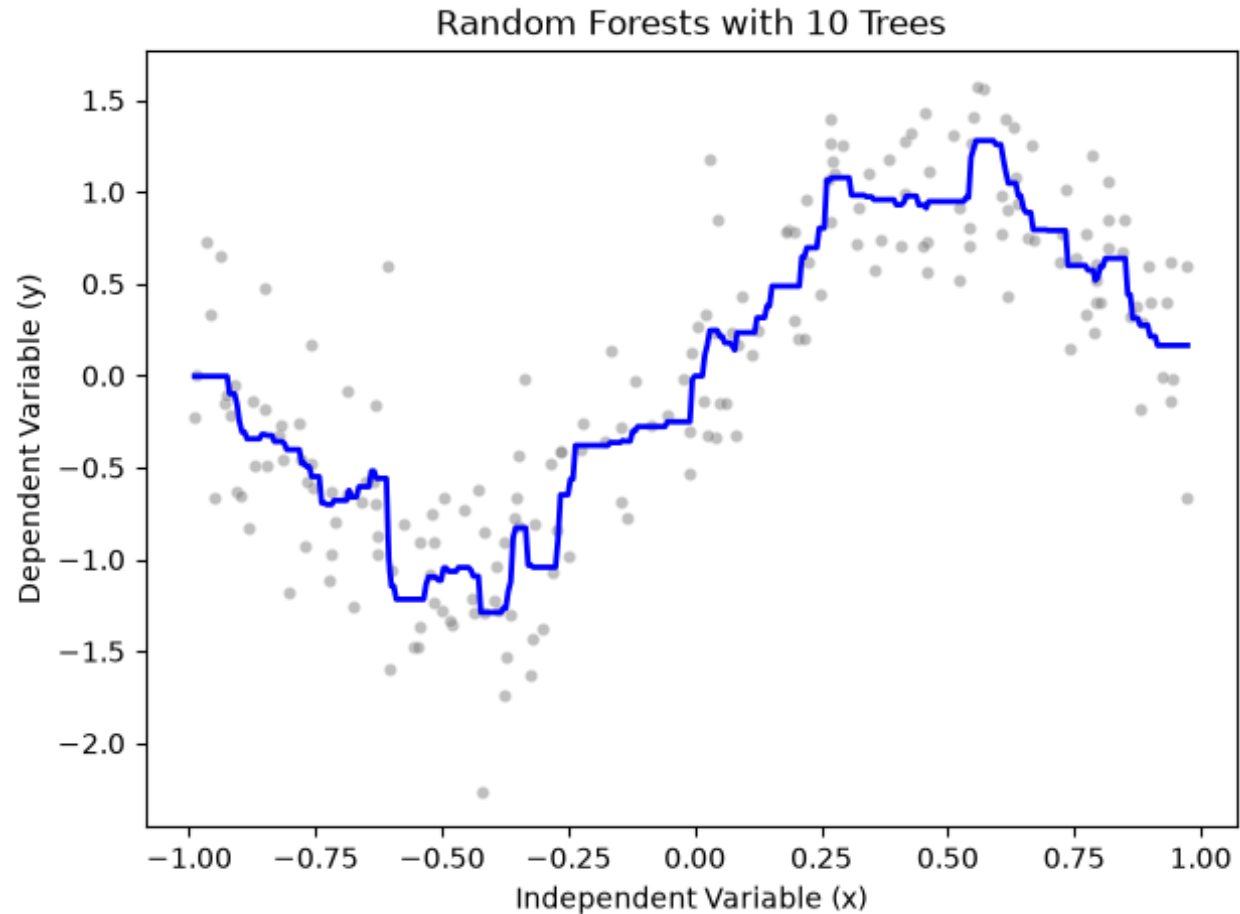
Random Forest: 3 Trees



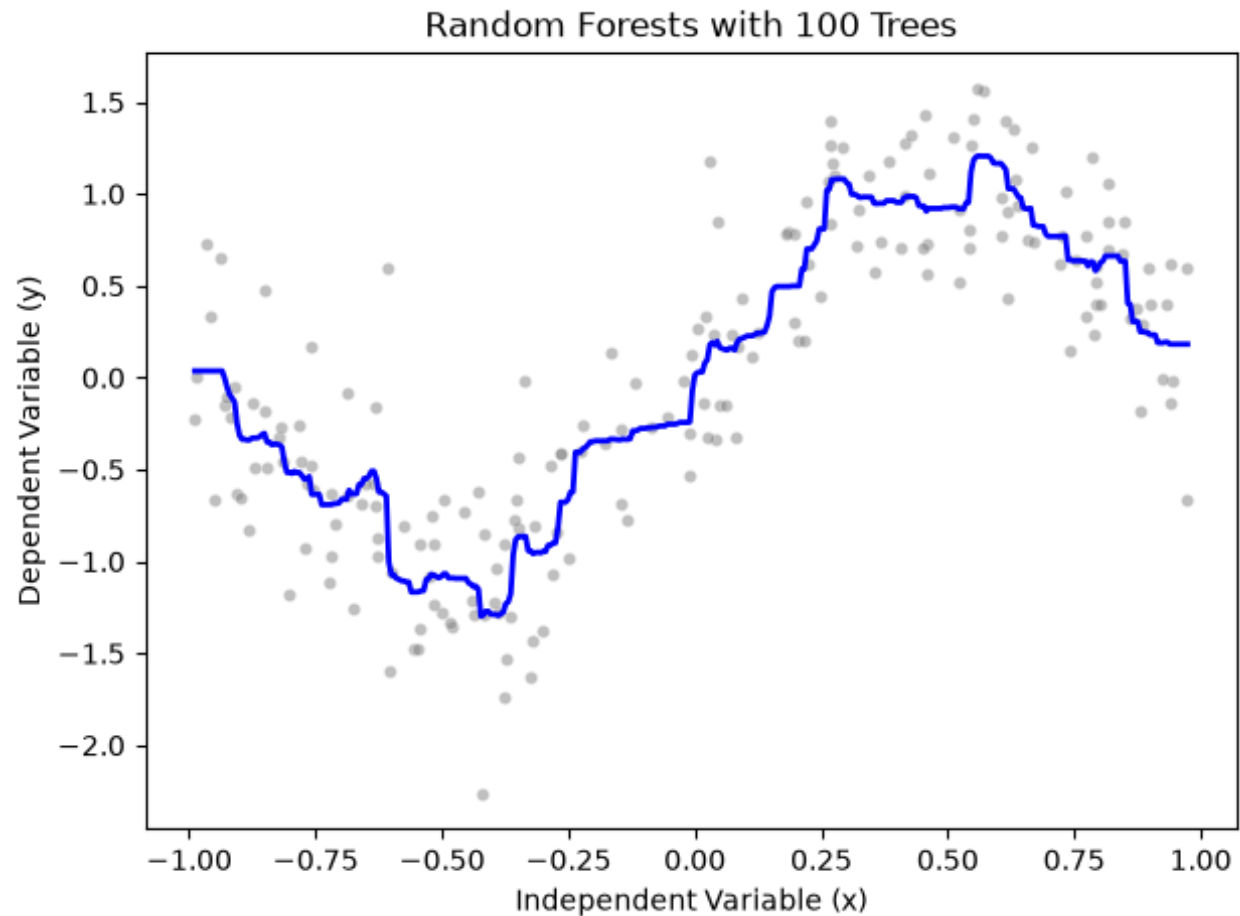
Random Forest: 5 Trees



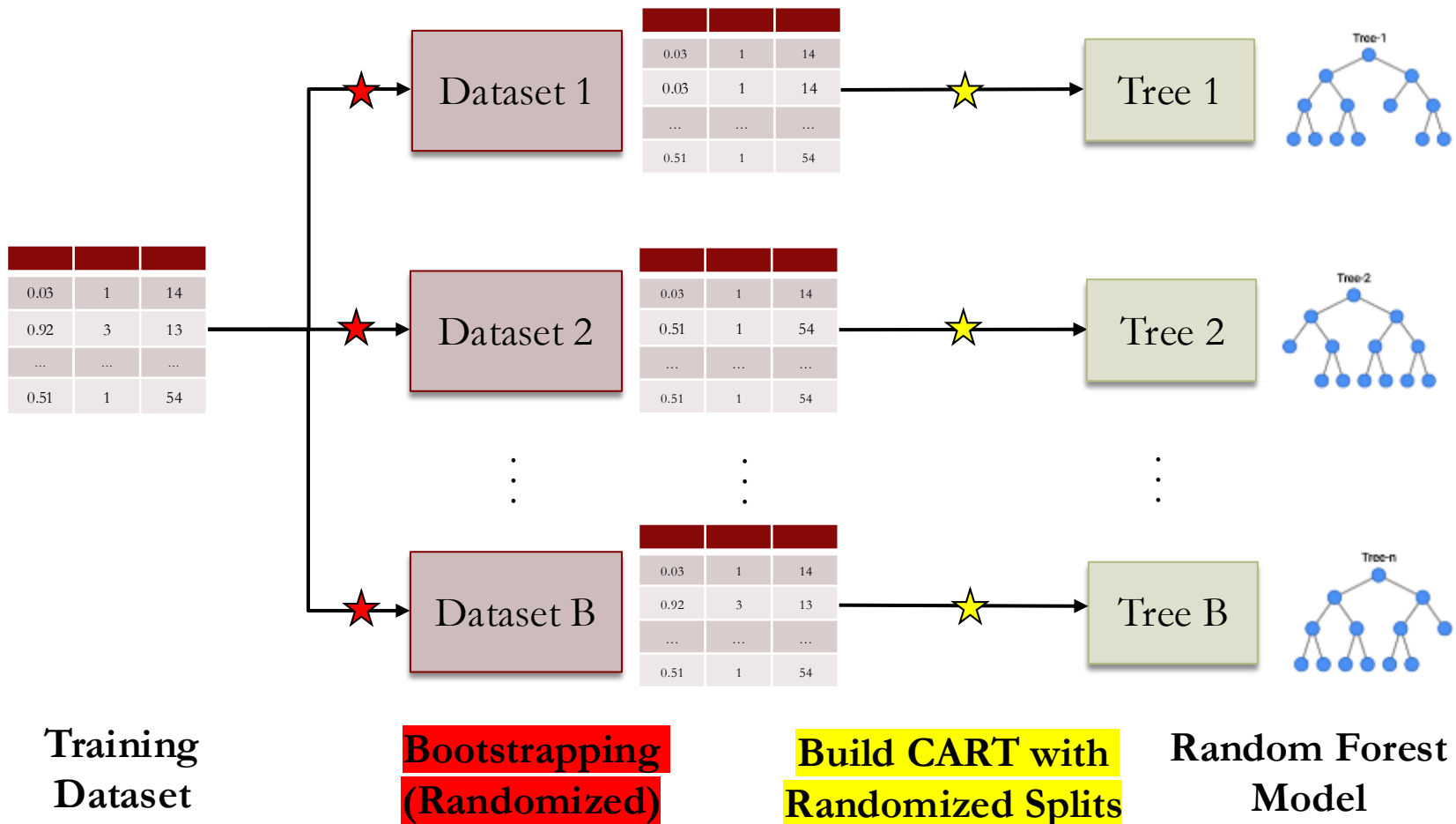
Random Forest: 10 Trees



Random Forest: 100 Trees



Building the Forest



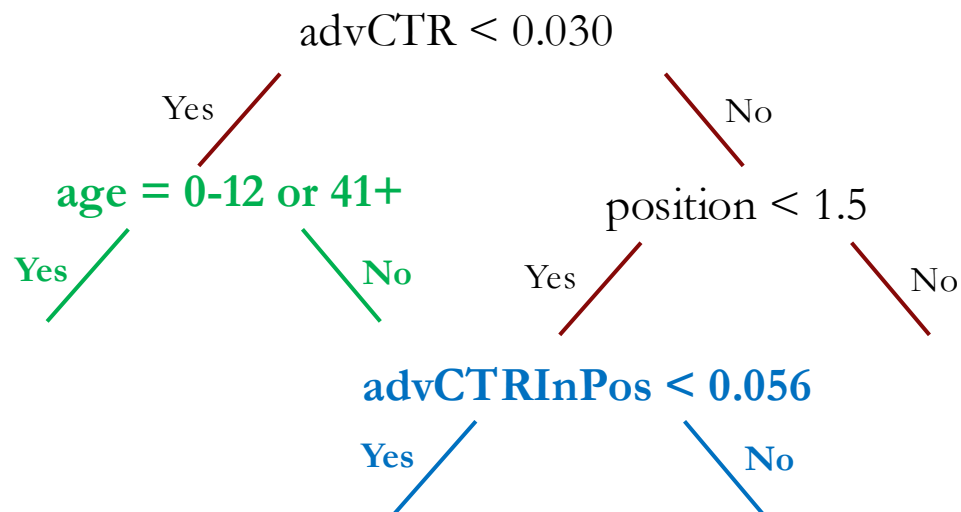
Randomizing Splits

Choosing different splits among trees

2. Randomizing Splits

titleWords	adWords	depth	position	advCTR	advCTRInPos	queryCTR	queryCTRInPos	gender	age
			↑ ↑	↑ ↑	↑				↑

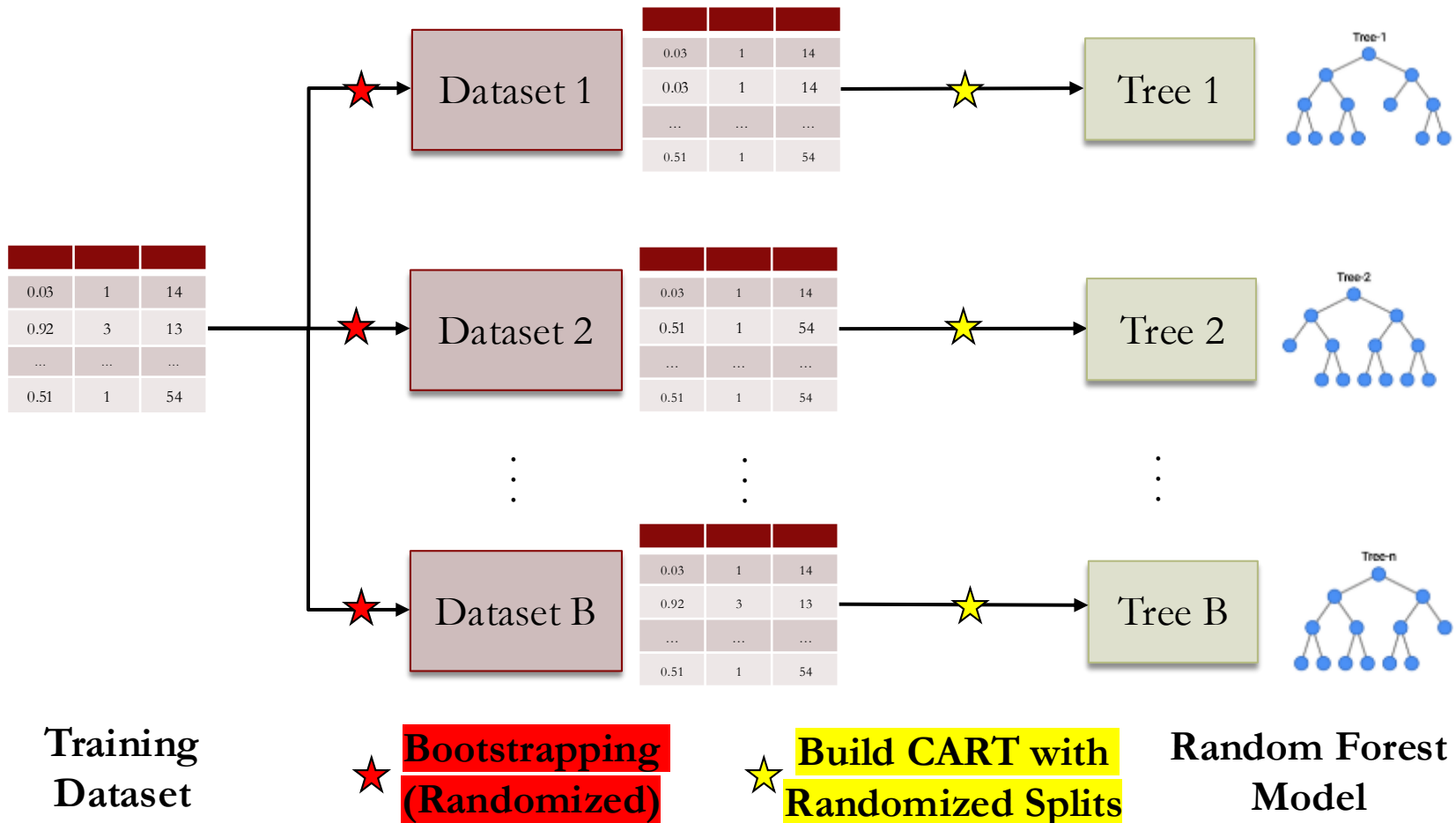
- For each split in the CART tree, random forests only consider a small random subset of m variables (here $m = 3$)
- Random forests add the best available split to the tree (among just those variables)
- The set of candidate variables to consider is different for each split



Why Does This Promote Diversity?

- Recall that CART typically chooses the split that reduces impurity (SSE or Total Loss) the most.
- Randomly selecting the candidate predictors at each split results in more diverse trees with lower correlation.
 - Consider an extreme example where there is one strong feature variable (say GPA) and other very weak variables (say citizenship)
 - In this case, CART would almost always pick the strong variable for its first split. As a result, all trees will be quite similar, and the trees' predictions will be highly correlated (hence less diverse)
- **Trade-Off.** However, sampling too few predictors at each split might result in poorly optimized splits and might worsen predictive ability.

Building the Forest Summary



2.2. Parameter Selection and Results

How do we select the best parameters?

How well does Random Forest perform?

Model Parameters: **n_estimators**, **max_features**, **min_samples_leaf**, **min_samples_split**

1. **n_estimators**: number of CART trees in the forest (B) (typical value = 100).
2. **max_features**: number of variables randomly examined at each split of the trees (m) (typical value = 1-10).
3. **min_samples_leaf**: minimum number of observations in each terminal node of the trees (same as in CART).

Default values (where k = number of independent variables)

- Regression: **n_estimators** = 100, **max_features** $\sim k$, **min_samples_leaf** = 1.
- Classification: **n_estimators** = 100, **max_features** $\sim \sqrt{k}$, **min_samples_leaf** = 1.

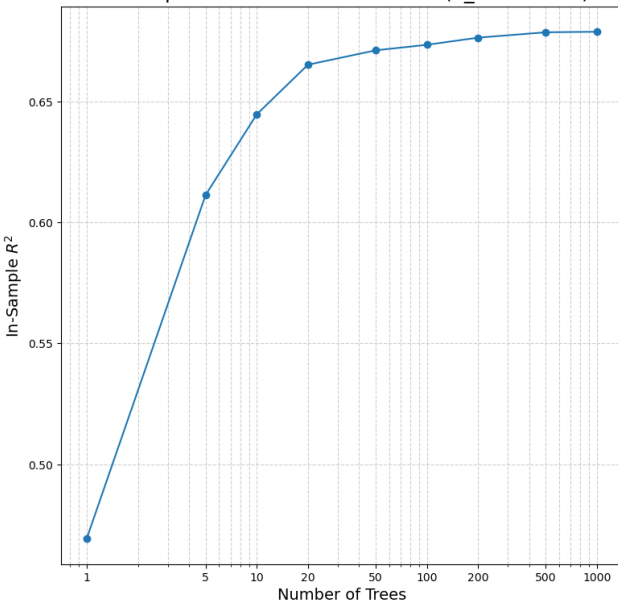
Selecting Parameters

- **n_estimators:** higher is always better, but there are diminishing returns (e.g., 50 $\rightarrow$ 500 vs. 500 $\rightarrow$ 5000 trees)
- **min_samples_leaf:** typical values (1-10).
- **max_features:** Cross validation is needed to choose the best value.
 - **Small max_features:** Trees have too much diversity, individually garbage (splits are not optimized at all)
 - **Large max_features:** Trees are too similar, don't get benefits of “wisdom of the crowds”.

In-sample Performance

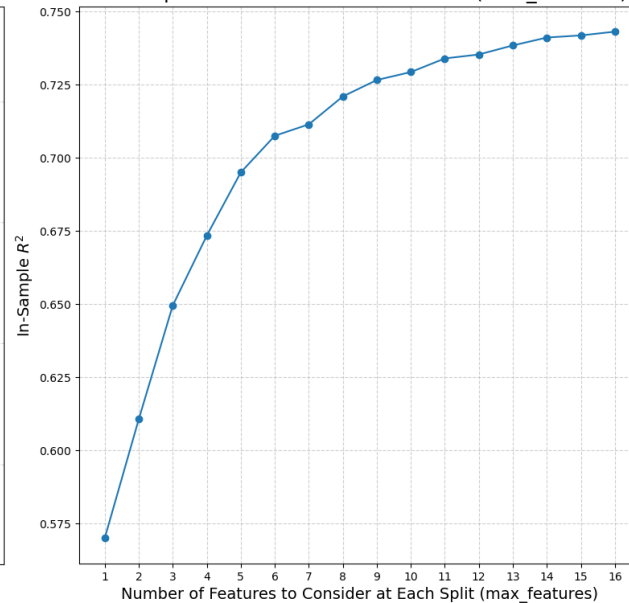
- In-sample performance improves with model complexity
- High in-sample performance due to high number of trees

In-Sample R^2 vs. Number of Trees ($n_{\text{estimators}}$)



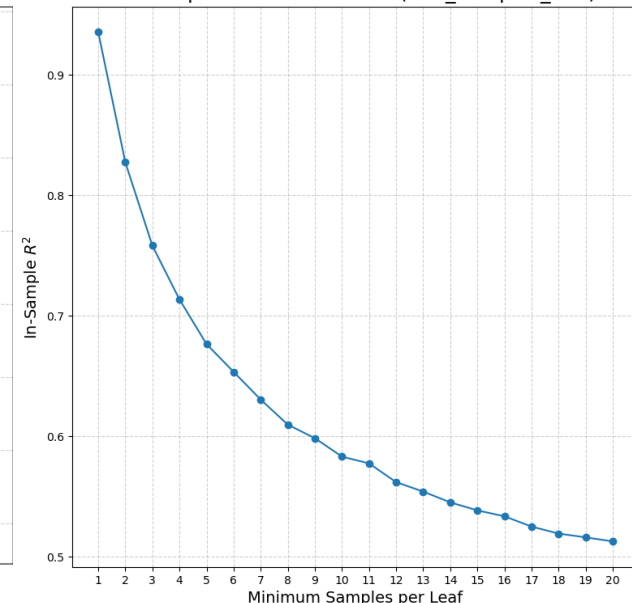
impact of $n_{\text{estimators}}$

In-Sample R^2 vs. Number of Features (max_features)



impact of max_features

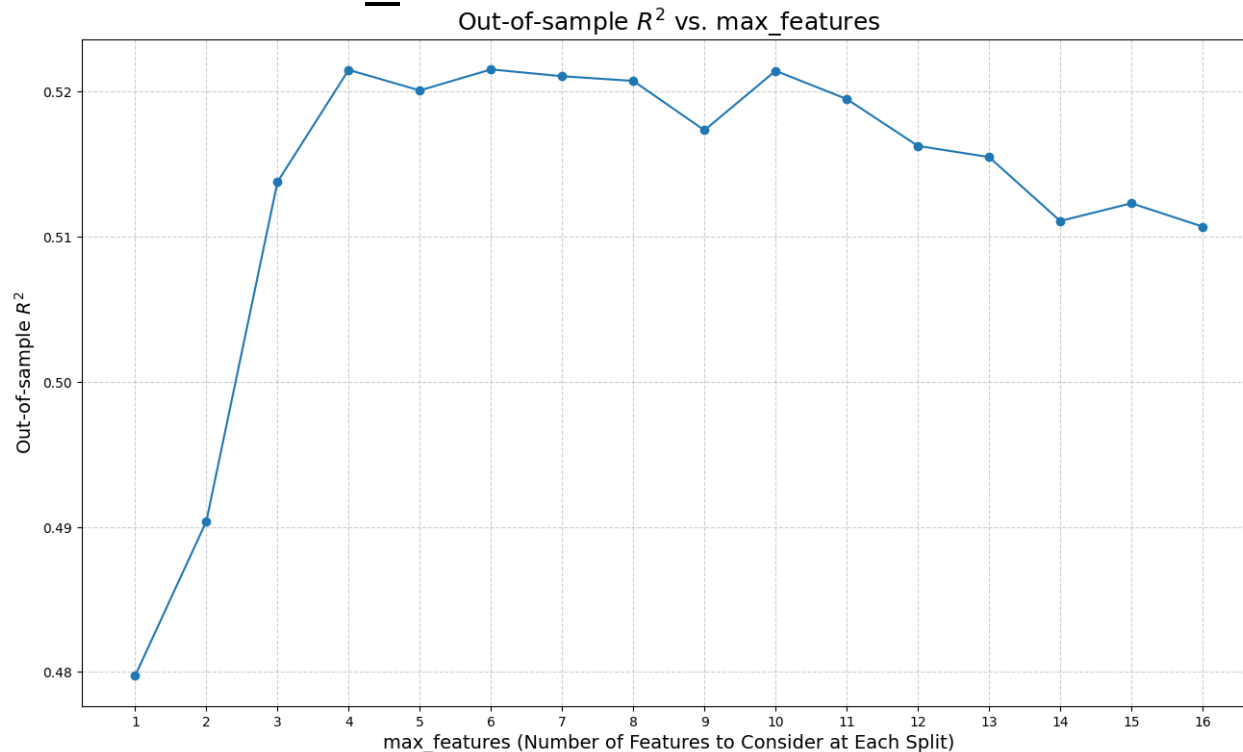
In-Sample R^2 vs. Node Size (min_samples_leaf)



impact of $n_{\text{samples_leaf}}$

max_features and out-of-sample performance

- Predictive (out-of-sample) performance is usually quite sensitive to **max_features**



impact of max_features

Hmm, it would be nice to find the best value of **max_features** ...

Cross-Validation on max_features

- Use default hyper-parameter values for: `n_estimators = 100`, `min_samples_leaf = 1`.
- For `max_features = 1, 2 ..., 16`:
 - Do cross-validation with the current value of `max_features`
 - Record the average “out-of-sample” R^2 over the cross-validated folds (**Average R^2**)
 - Choose the value of `max_features` that yields the highest value of **Average R^2**

Tested values

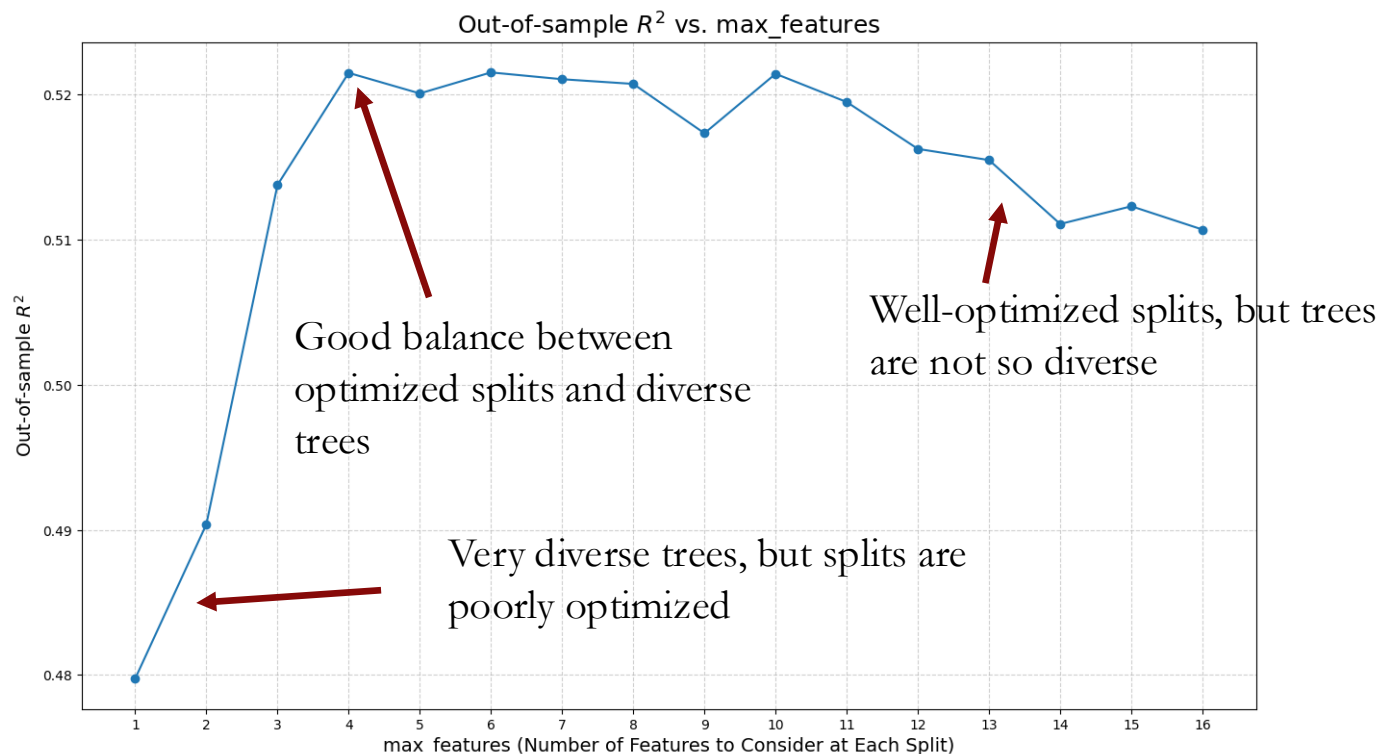
Average performance

Selected value

Optimal max_features based on Out-of-sample R^2 : 6

max_features	CV_RMSE	CV_R2	CV_MAE
1	0.05686846	0.47974634	0.03303685
2	0.05629023	0.49037711	0.03275158
3	0.05498121	0.51376365	0.03206813
4	0.05455010	0.52148579	0.03197125
5	0.05461888	0.52006767	0.03185977
6	0.05455348	0.52151672	0.03185957
7	0.05456461	0.52104752	0.03186136
8	0.05458457	0.52072551	0.03197906
9	0.05477675	0.51732449	0.03197170
10	0.05455394	0.52142082	0.03194125
11	0.05465199	0.51947636	0.03187482
12	0.05483519	0.51624585	0.03211161
13	0.05488208	0.51547648	0.03214342
14	0.05513954	0.51107239	0.03217064
15	0.05505429	0.51229425	0.03219512
16	0.05515126	0.51068888	0.03228553

Cross-Validation Results



- Cross-validation yields out-of-sample estimates within the overall training data
- Can you see any downside of using cross-validation to set the value of **max_features**?

Results

	Linear Regression	CART Model	Random Forest
OSR ²	0.3961	0.3377	0.4940
Out-of-Sample MAE	0.0331	0.0340	0.0304
Out-of-Sample RMSE	0.0551	0.0577	0.0504

- The predictive performance of the random forest model is significantly better than that of the other two models
- But the choice of the “right” model balances different objectives:
 - Predictive performance
 - Interpretability
 - Computational efficiency
- Let’s talk about the computational efficiency and interpretability of Random Forest.

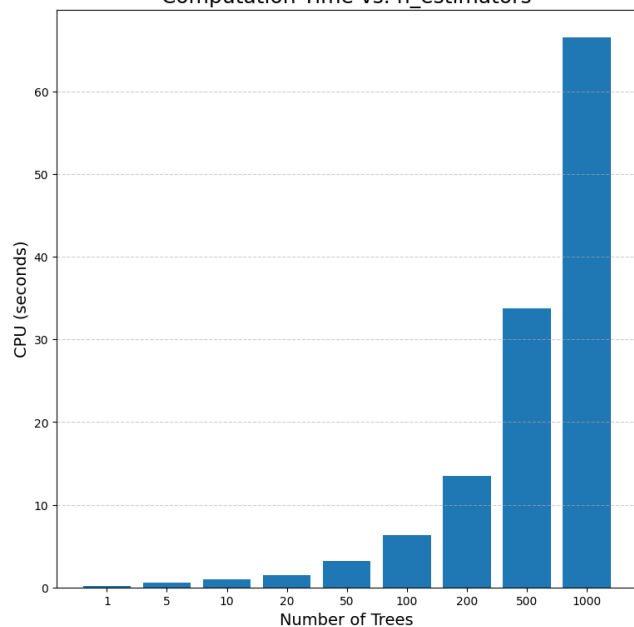
2.3 Computation Time and Interpretability

What are the downsides of Random Forest?

Computation Time

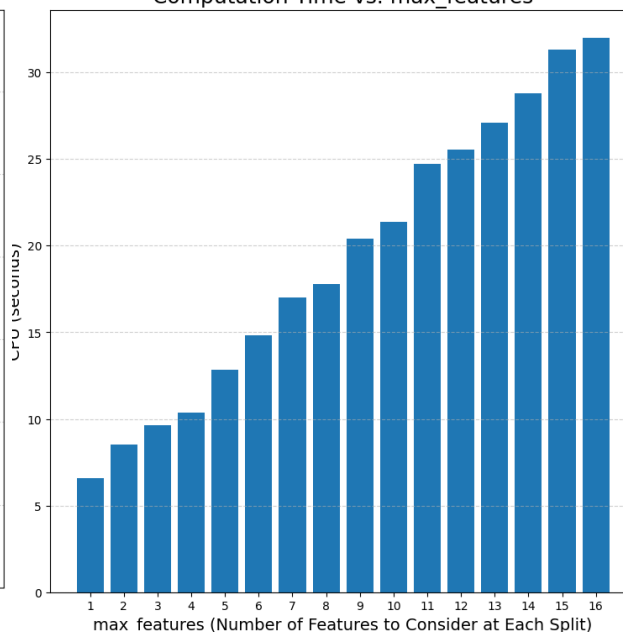
- Random forests can be computationally intensive to build
- Model runtimes increase significantly with model complexity

Computation Time vs. n_estimators



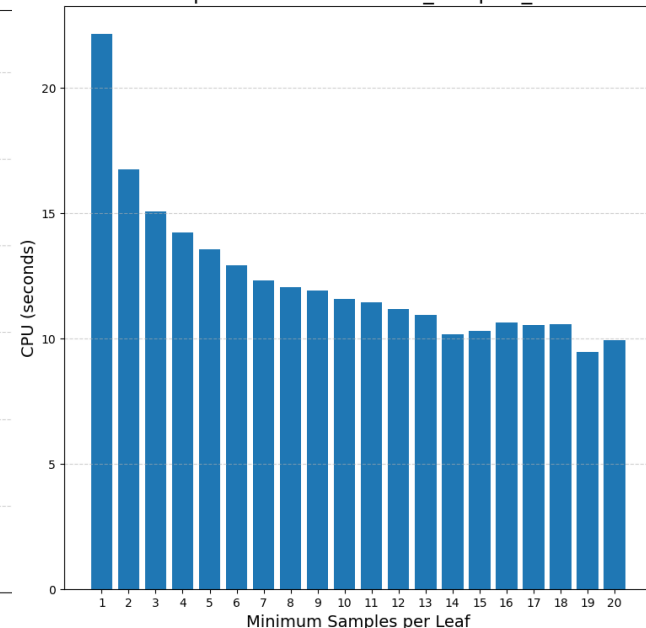
impact of n_estimators

Computation Time vs. max_features



impact of max_features

Computation Time vs. min_samples_leaf



impact of min_samples_leaf

Computation Time

- Random forests can be computationally intensive to build
- Model runtimes increase significantly with model complexity (e.g., number of trees, depth of each tree)
- However, they have one very important advantage: they can easily be computed using **parallel architecture**
 - Each tree in the forest can be built in parallel
 - If you need to build 500 trees, spin up 500 servers in the cloud and build one tree on each server, or 2 trees on 250 servers, or ...
 - The time it takes to build a random forest using parallel architectures is proportional to the time it takes to build a single CART tree

Computational Effort

- We performed 10-fold cross validation on 10 values of `max_features` (1, 2, ..., 10). Each random forest model was trained with `n_estimators` = 100.
- **Questions:**
 - How many CART models did we build in total?
- **Answer:** Consider `max_features` = 1, and a particular fold (let's say fold 1, which uses the last 10% of the data as testing). We must produce 100 bootstrapped datasets, and each one generates a CART model. 100 CART models and 100 bootstrapped samples just for 1 `max_features` and 1 fold, so $100 * 10 * 10 = 10,000$ trees and bootstrapped datasets in total.

Computational Effort of Cross-Validation on Random Forests

	CART (10-Fold Cross-Validation)	Random Forest (10-Fold Cross-Validation)
Time to fit a CART tree (mseconds)	~ 10.0	~ 10.0
CART trees per model fit	1	500
Model fits per parameter tested	10	10
Parameter values tested	10	10
Computation time	1 second	8 minutes

- The computational effort can be very significant because:
 - K-Fold cross-validation replicates the fitting/testing procedure many times
 - Trees in random forests are more complex than CART (no `ccp_alpha`)

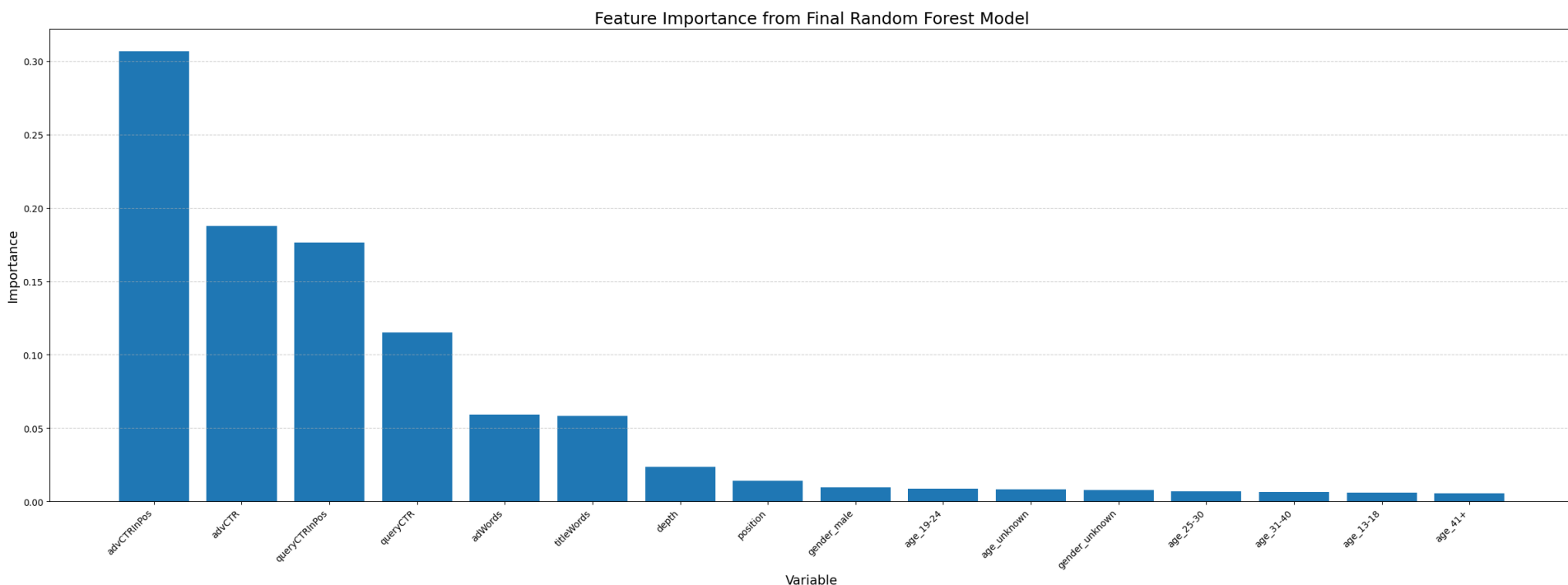
Interpreting Random Forest Models

Variable Importance/Influence

- CART trees are very interpretable
 - In contrast, by combining many trees (each of which is quite large), random forests are typically not interpretable at all
 - Random forest can sort-of be interpreted using **variable importance** metrics
 - For regression, we look at the improvement in the sum of squared errors from splitting with the variable, averaged across all trees
 - For classification, we look at the improvement in the weighted loss, averaged across all trees
- This provides an ordering of variables by importance (although the actual values cannot be readily interpreted)

Variable Importance in CTR Model

- The most important variables identify the main drivers of CTR
- The main drivers appear to be historical click-thru rates of the advertiser's ad impressions and of the query (both in general and in the position)



Comparison of CTR Prediction Models

Let us Compare all Models Discussed So Far

	Linear Regression	Logistic Regression	CART	Random Forest
Can be used to predict numerical outcomes	✓	✗		
Can be used for classification problems	✗	✓		
Captures nonlinear relationships in the data	~	~		
Simple and interpretable	~	~		
Yields best out-of-sample prediction quality	~	~		
Computational Efficiency	✓	✓		

Let us Compare all Models Discussed So Far

	Linear Regression	Logistic Regression	CART	Random Forest
Can be used to predict numerical outcomes	✓	✗	✓	✓
Can be used for classification problems	✗	✓	✓	✓
Captures nonlinear relationships in the data	~	~	✓	✓
Simple and interpretable	~	~	✓	✗
Yields best out-of-sample prediction quality	~	~	~	✓
Computational Efficiency	✓	✓	✓	~

Random Forest Summary

- Builds a collection of CART models in a way that promotes diversity among the trees.
 - Bagging for diversity in the training datasets
 - `max_features` for diversity in the splits chosen
- Cross-validation is used to select optimal parameters (`max_features`, `n_estimators`, `min_samples_leaf`).
- Random Forest models work very well in practice, but are computationally intensive to train (can be parallelized) especially when cross validation is performed too.
- Random Forest models are difficult to interpret. Some notion of feature importance can be obtained, but it is not clear how changing the feature impacts the prediction of the model.

Advanced Materials: Reducing Computation Time using “Out-of-bag” Predictions

An alternative to K-Fold Cross Validation specifically for Random Forest

Out-of-bag Predictions: Introduction

- Random forests are built by sampling data from the training set
 - In each sample, some rows appear multiple times, while others do not appear at all
- Using the training data, how might we estimate how well the tree fitted with this dataset generalizes — without using cross-validation?

	titleWords	adWords	depth	position	advCTR	advCTRInPos	
→ → →	2	12	14	1	1	0.0254	0.031
→	3	5	19	3	2	0.0178	0.0076
→ → →	4	11	17	2	2	0.0096	0.0069
	5	15	28	2	2	0.0307	0.0157
	6	10	20	3	1	0.0283	0.0338
	7	9	23	3	1	0.0054	0.0084
	9	11	21	2	1	0.018	0.0314
	...	...	...	...	...	...	...
→ →	6051	10	15	3	2	0.0292	0.0303
	6052	8	16	1	1	0.0273	0.0356
→	6054	9	26	3	1	0.0496	0.0523

	titleWords	adWords	depth	position	advCTR	advCTRInPos
2	12	14	1	1	0.0254	0.031
2	12	14	1	1	0.0254	0.031
2	12	14	1	1	0.0254	0.031
3	5	19	3	2	0.0178	0.0076
4	11	17	2	2	0.0096	0.0069
4	11	17	2	2	0.0096	0.0069
4	11	17	2	2	0.0096	0.0069
...	...	...	...	...	...	...
6051	10	15	3	2	0.0292	0.0303
6051	10	15	3	2	0.0292	0.0303
6054	9	26	3	1	0.0496	0.0523

Out-of-bag Predictions: Principles

- Each training set observation (row) appears in roughly 63% of the bootstrap (bag) samples*
- **Out-of-bag prediction:** for each row, we use all the trees that did not use that row in the fitting process ($\sim 37\%$ of the trees)
 - Regression: predict the average over all of these trees
 - Classification: predict the most common class among all of these trees
- Estimates the out-of-sample performance (R^2 , RMSE, MAE, etc.) using only the training data
- An efficient way to estimate how well random forests generalize to “new” data, without having to do k -fold cross-validation

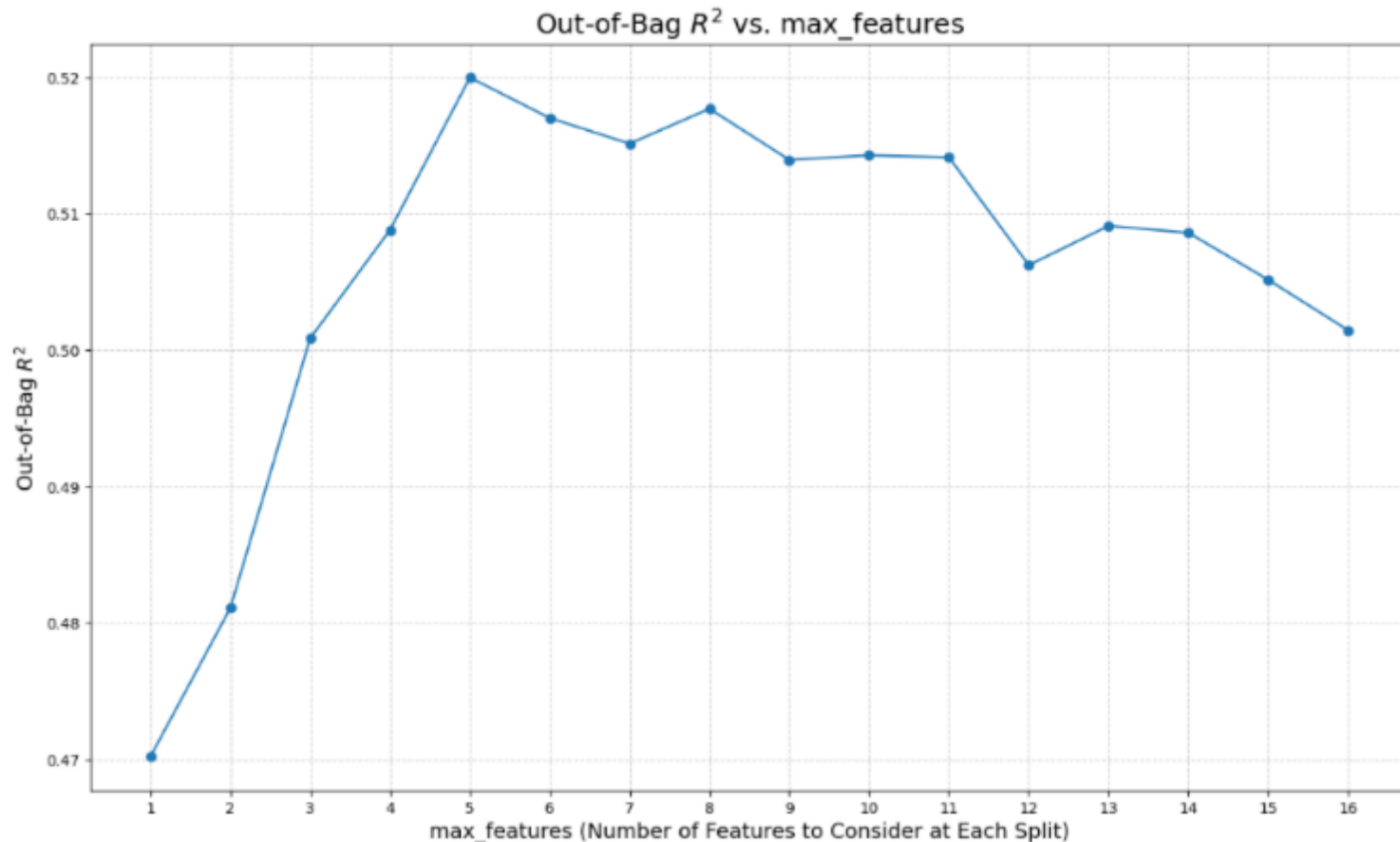
Out-of-bag Predictions

Illustration for $n_{\text{tree}} = 500$

Bootstrap Sample	Row i included?	Model	Prediction for row i
Sample 1	YES	Model 1	—
Sample 2	NO	Model 2	Prediction 2
Sample 3	YES	Model 3	—
Sample 4	NO	Model 4	Prediction 4
$\vdots$	$\vdots$	$\vdots$	$\vdots$
Sample 500	YES	Model 500	—

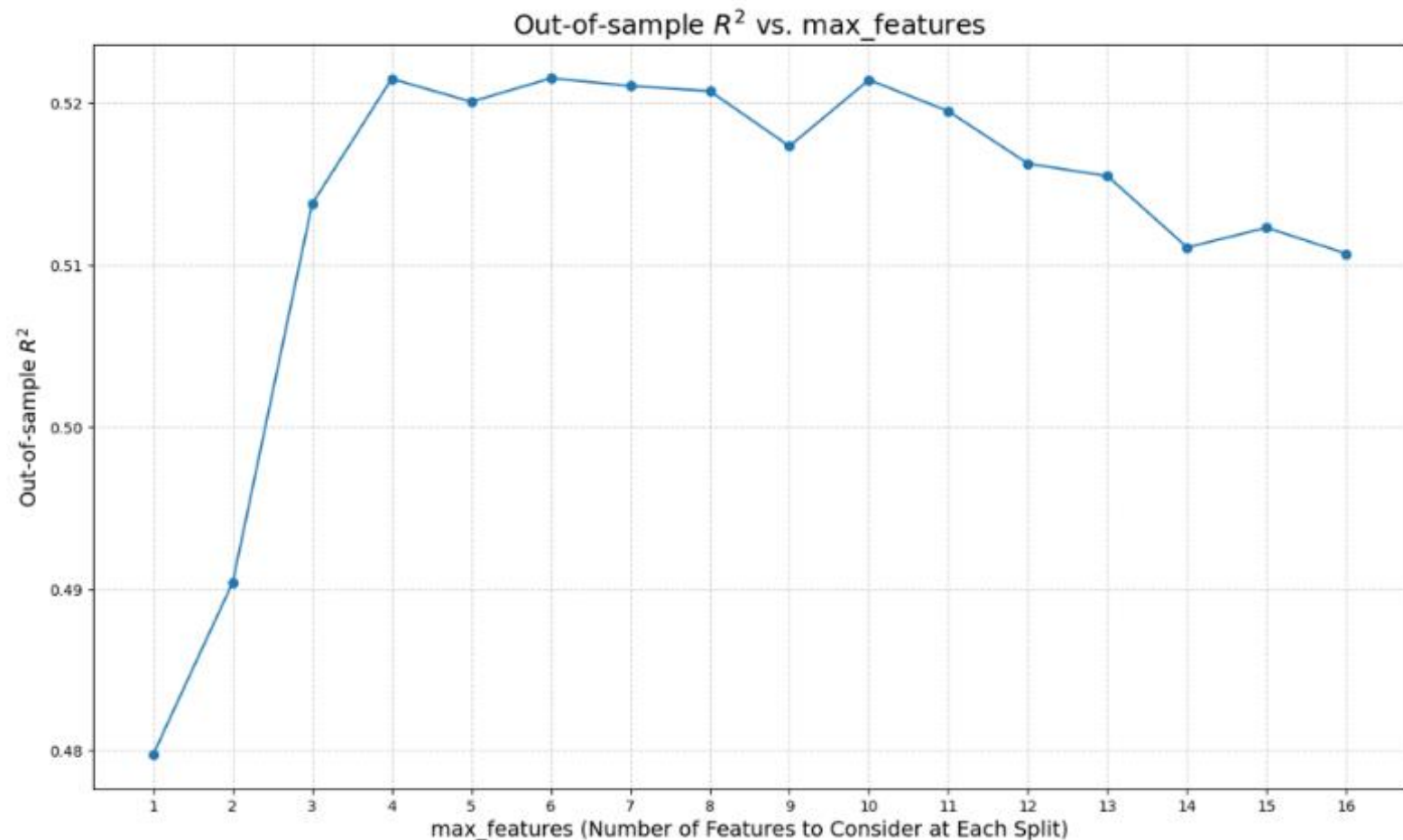
- Average prediction for row i : $(\text{Pred. 2} + \text{Pred. 4} + \dots) / (\text{number of samples where row } i \text{ is NOT included})$
- Repeat for all parameter values and compute the performance metrics, and then choose the value that optimizes the performance metric

Out-of-bag Predictions: Results



These results are quite similar to what we obtained using cross-validation

Cross-Validation and Out-of-bag Prediction yield similar results



Computational Burden

	CART (10-Fold CV)	Random Forest (10-Fold CV)	Random Forest (out-of-bag)
Time to fit a CART tree (milliseconds)	~ 10.0	~ 10.0	~ 10.0
CART trees per model fit	1	500	500
Model fits per parameter tested	10	10	1
Parameter values tested	10	10	10
Computational time	1 second	8 minutes	1 minute

- In summary, out-of-bag predictions:
 - avoids repeating the fitting/testing procedure multiple times, and is thus more computationally efficient than cross-validation
 - is oftentimes quite accurate, leading to similar parameter value choices (e.g. **max_features**)



PART II: BOOSTING

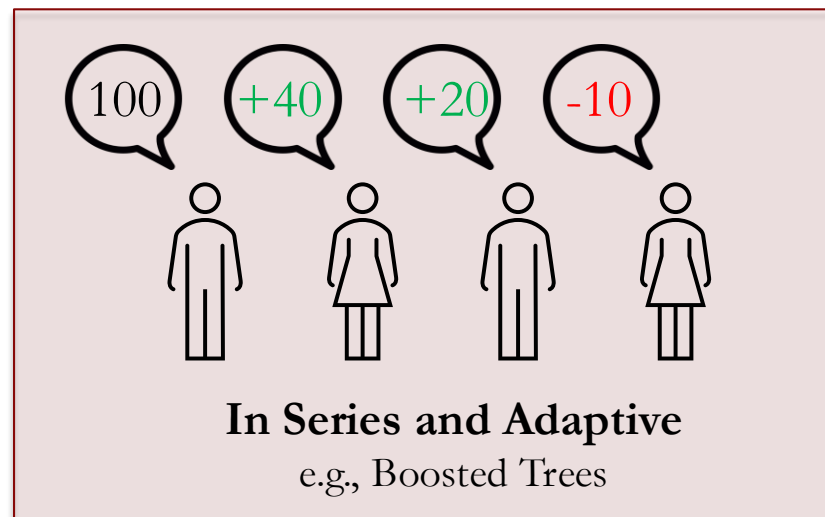
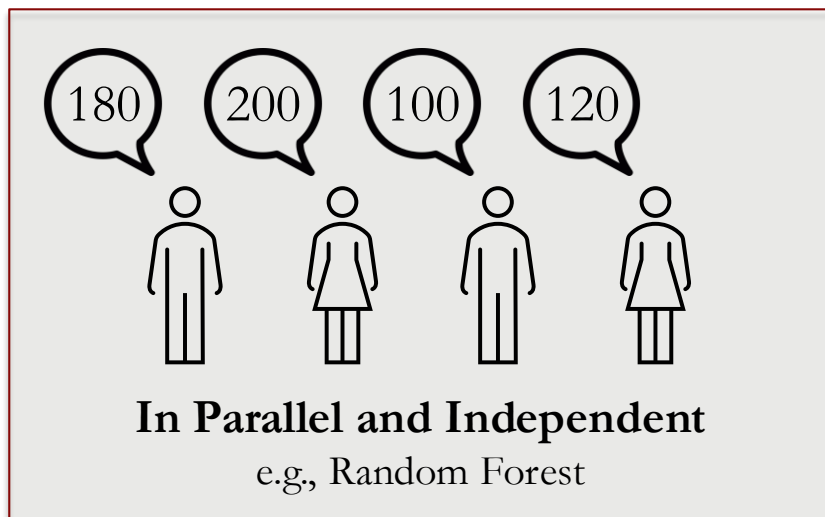
Predicting Click-Through Rates of Ad Impressions

Ensemble Learning Paradigms

- **Models in Parallel:** Build models independently, then make predictions by taking an average or majority vote.
- **Models in Series:** Build models adaptively, where each model is trained to fix the mistakes of the previous models.



Number of candies?



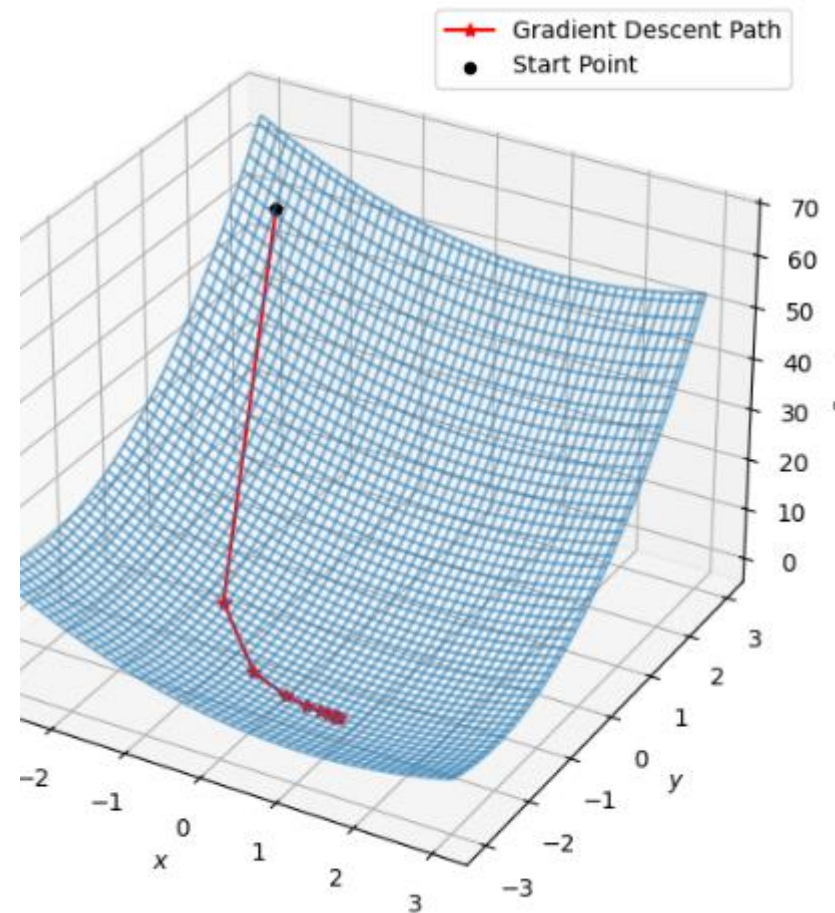
Boosted Trees

(also known as “Gradient Boosted Trees”, “Gradient Boosting Machine”)

Sequential predictions

Aside: trying to minimize a function $f(\mathbf{x})$

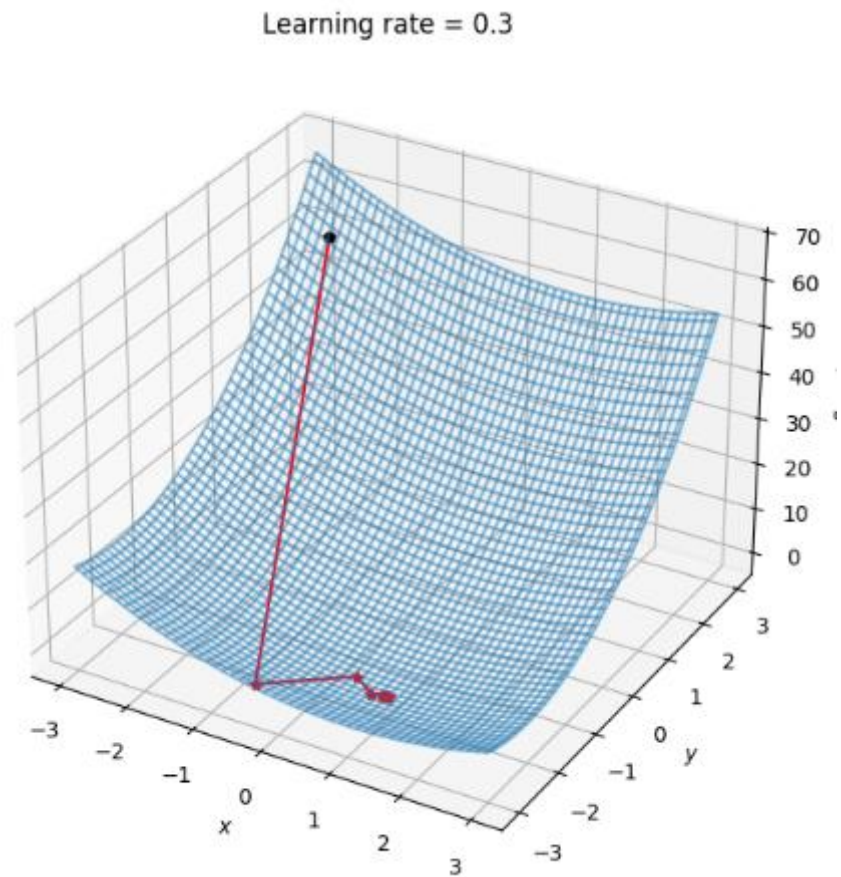
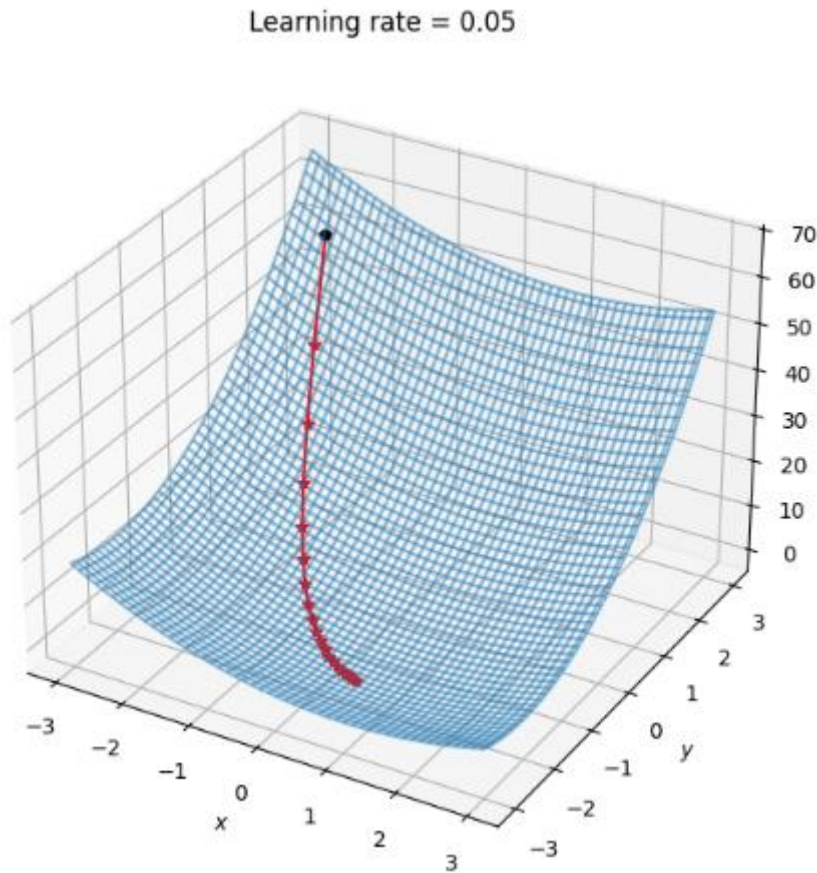
- Suppose we want to find $\mathbf{x}$ that minimizes $f(\mathbf{x})$
- Suppose we can't “see” the function, but we can compute $f'(\mathbf{x})$ for any given value of $\mathbf{x}$
- “Gradient Descent”



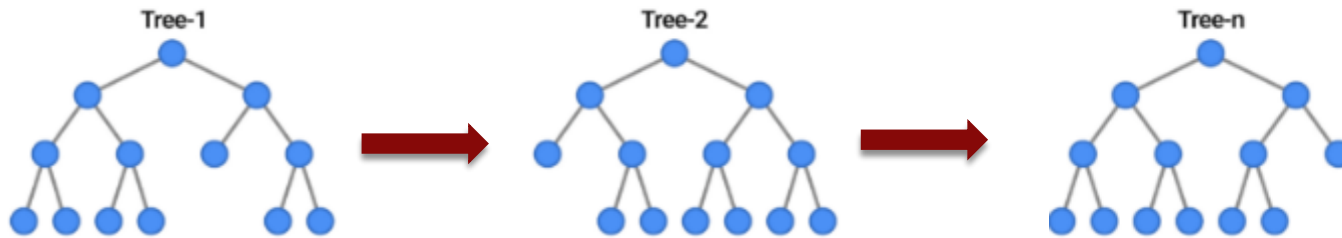
Gradient Descent Algorithm

- Algorithm:
 1. Current point is x
 2. Compute $f'(x)$
 3. Next point is $x \leftarrow x - \eta f'(x)$
- η is called the “step-size” among mathematicians, but in Machine Learning it is called the “learning rate” or the “shrinkage parameter”
- A larger learning rate allows us to make initial progress fast, at the expense of less accuracy as we get nearer to the actual solution
- A smaller learning rate will take us longer to get near to the solution, but is likely to result in a higher accuracy solution

Step-size/learning rate



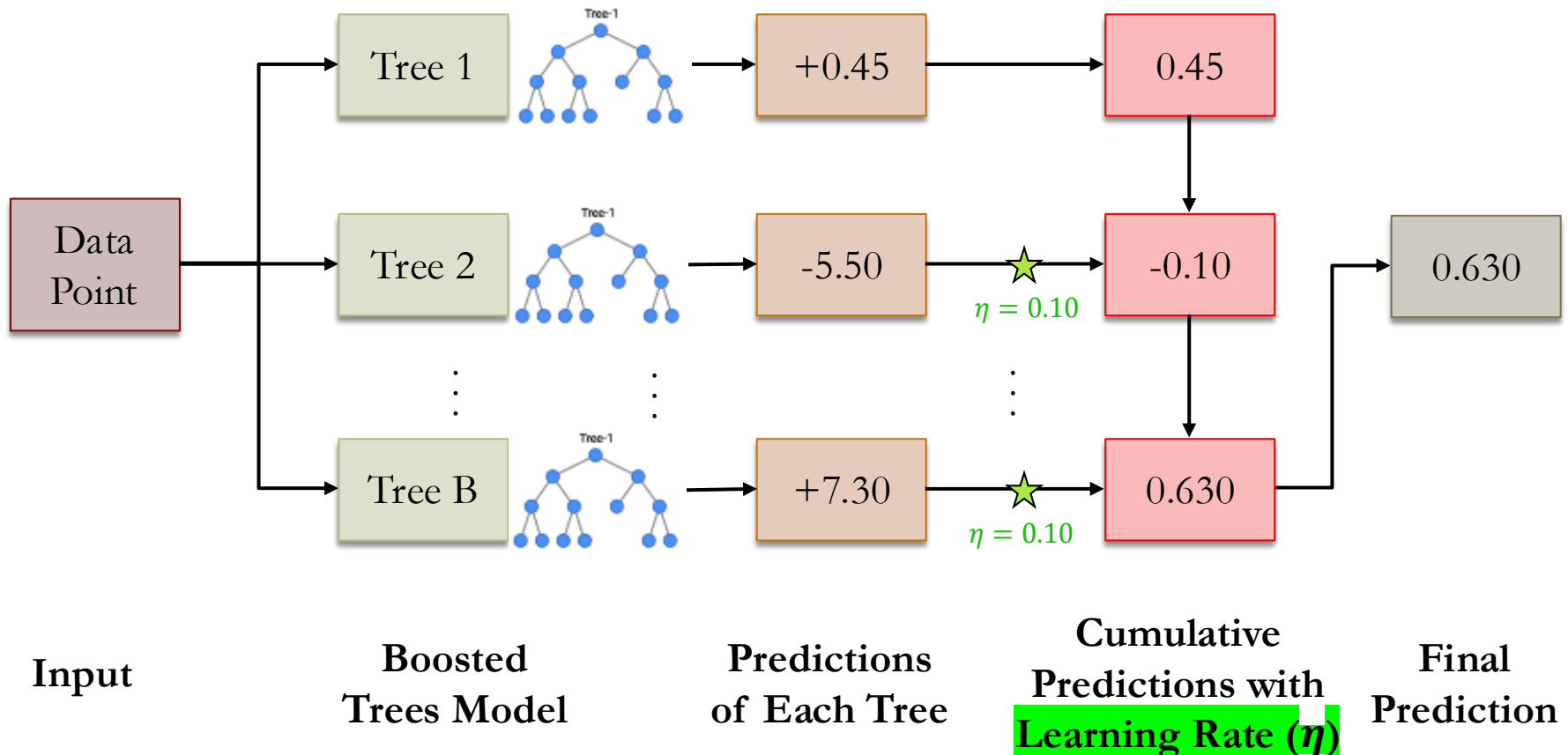
Boosted Trees Overview



- (Gradient) Boosted trees is an ensemble learning model consisting of a sequence of trees.
- **Making Predictions.** Start with the predictions of the first tree, then having each subsequent tree adjust that prediction slightly.
- **Building the Trees.** The trees are built sequentially: each tree is built to correct the “mistakes” of the previous trees.

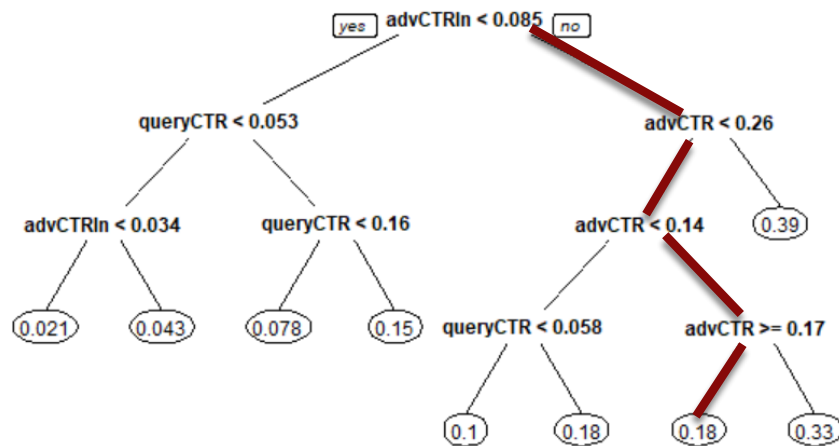
Making Predictions

Making Predictions



Notation

- Function notation for trees: we can think of a given tree, say tree number 7 in our ensemble, as a function $f_7(x_1, \dots, x_k)$ that takes values of our k independent variables $x_1, \dots, x_k$, and assigns a value to the dependent variable y , namely the prediction $y = f_7(x_1, \dots, x_k)$



For this tree we have:

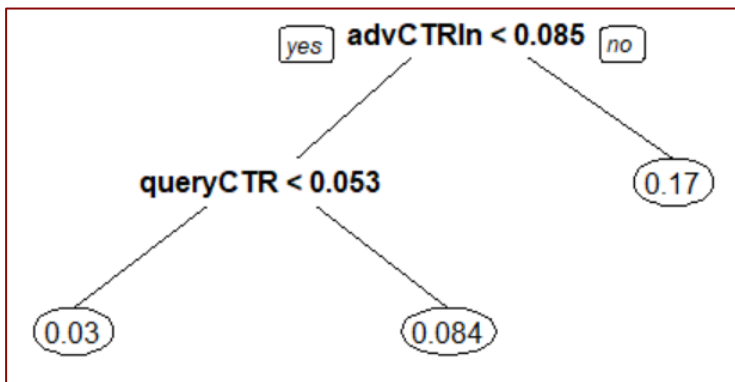
$$f_7(x_1, \dots, x_k) = f_7(\text{advCTRln}, \text{advCTR}, \dots, \text{queryCTR})$$

$$f_7(0.1, 0.2, \dots, 0.05) = 0.18$$

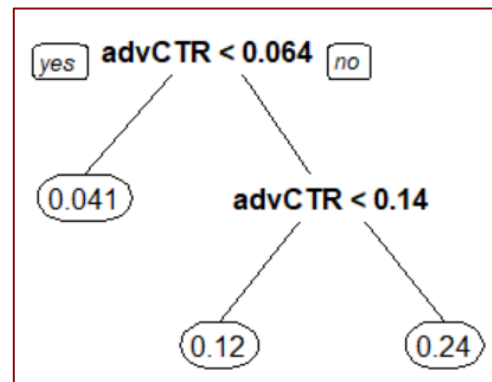
Additively Combining trees

Using this notation, we can define additive combinations of trees:

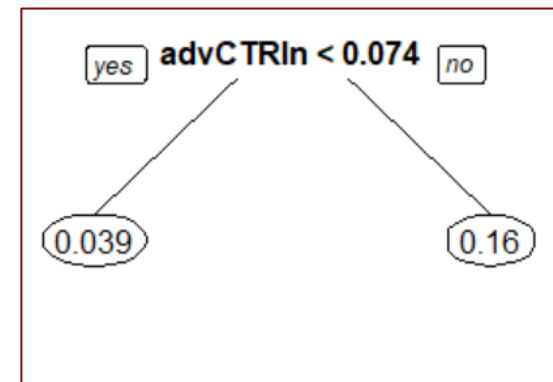
$$y = \text{Tree}_1 + 0.10 \times \text{Tree}_2 + 0.10 \times \text{Tree}_3$$



Tree₁



Tree₂

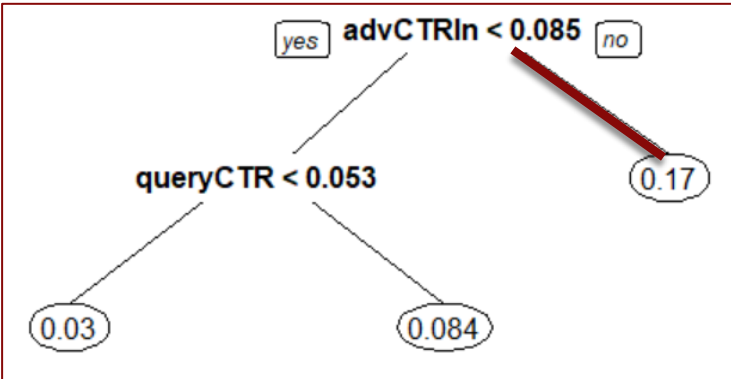


Tree₃

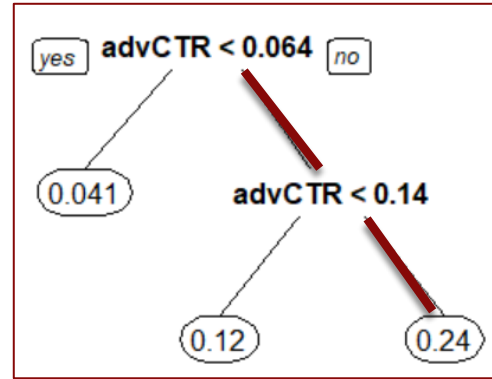
This is also:

$$y = f_1(x_1, \dots, x_k) + 0.10 f_2(x_1, \dots, x_k) + 0.10 f_3(x_1, \dots, x_k)$$

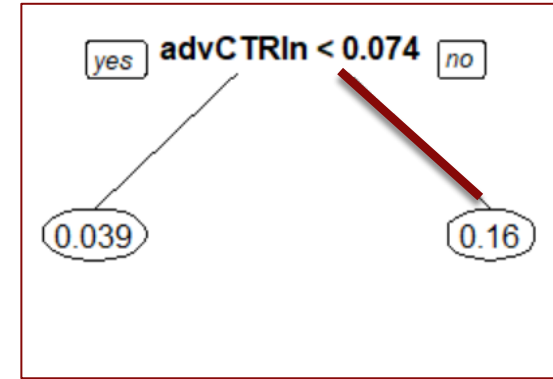
Additively Combining trees, cont.



Tree₁



Tree₂



Tree₃

$$y = f_1(x_1, \dots, x_k) + .10 f_2(x_1, \dots, x_k) + .10 f_3(x_1, \dots, x_k)$$

Suppose (advCTRIn, advCTR, queryCTR) = (0.1, 0.2, 0.05)

$$\text{Then } y = 0.17 + .10 * 0.24 + .10 * 0.16 = 0.21$$

Making Predictions

- Let $f_1, f_2, \dots, f_B$ be the function notation for each of the B trees.
- For a regression problem, the final prediction is made by:

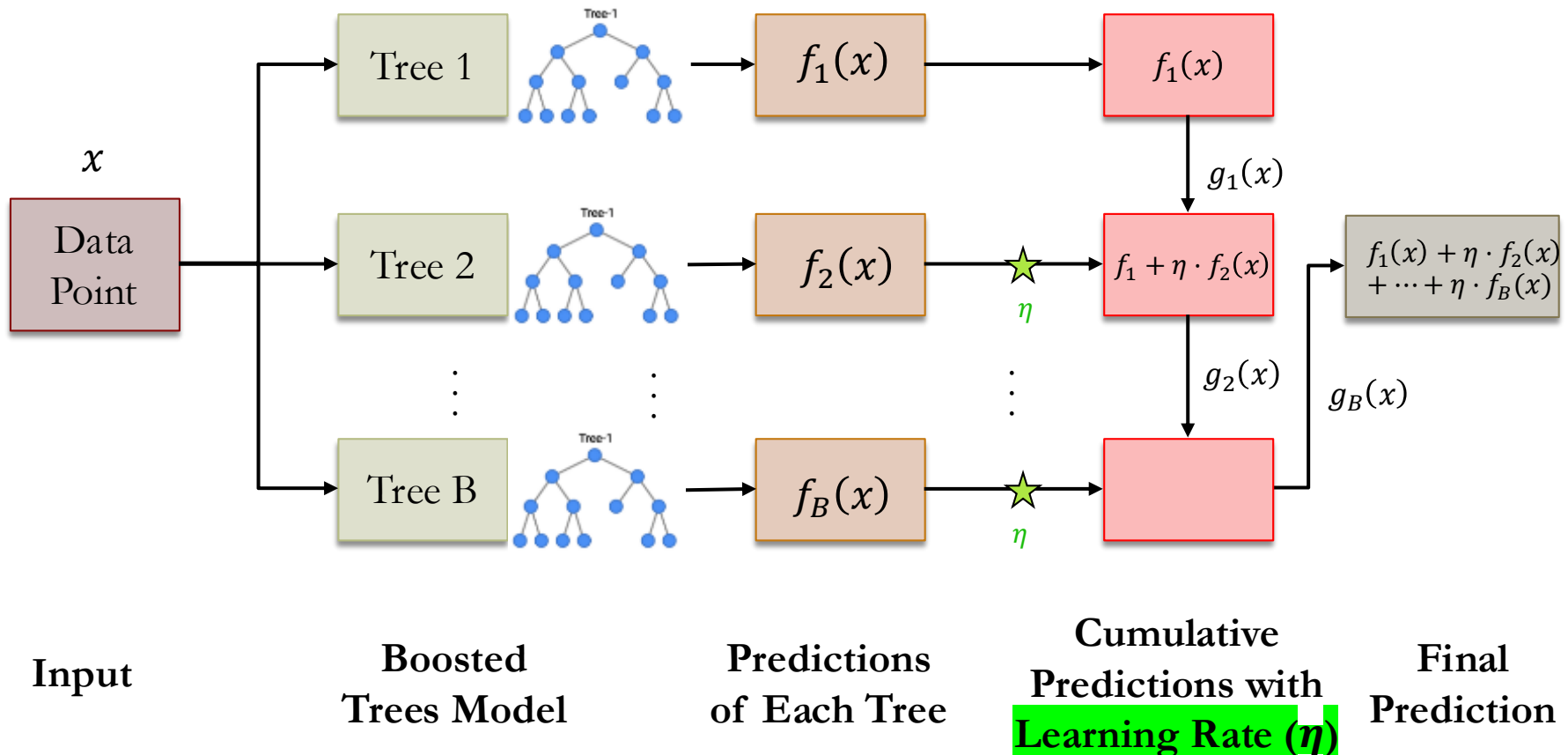
$$f_1(x) + \eta \cdot f_2(x) + \eta \cdot f_3(x) + \dots + \eta \cdot f_B(x)$$

Here, I'm using x to represent a vector of features $x = (x_1, x_2, \dots, x_k)$

- For a classification problem using logistic regression, the predicted probability is given by:

$$\frac{1}{1 + e^{-[f_1(x) + \eta \cdot f_2(x) + \eta \cdot f_3(x) + \dots + \eta \cdot f_B(x)]}}$$

Making Predictions



Building the Trees

Adaptively building trees based on residuals.

A (very) Simplified Boosting Example

Suppose we have the following very simple dataset:

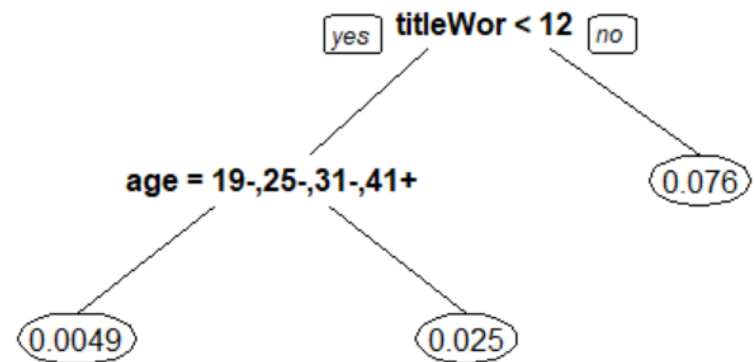
Data

CTR	titleWords	age
0.0000	10	41+
0.0761	13	13-18
0.0068	11	19-24
0.0383	10	13-18
0.0125	11	13-18
0.0128	9	31-40
0.0000	9	25-30

A (very) Simplified Boosting Example, cont.

We fit an initial simple tree model to the data. Cthis model g_1 :

Data			Initial model g_1
CTR	titleWords	age	
0.0000	10	41+	0.0049
0.0761	13	13-18	0.0761
0.0068	11	19-24	0.0049
0.0383	10	13-18	0.0254
0.0125	11	13-18	0.0254
0.0128	9	31-40	0.0049
0.0000	9	25-30	0.0049



A (very) Simplified Boosting Example, cont.

Let us compute the residuals (errors) of the model g_2 :

Data			Initial model	Residuals
CTR	titleWords	age	g_1	r_1
0.0000	10	41+	0.0049	-0.0049
0.0761	13	13-18	0.0761	0.0000
0.0068	11	19-24	0.0049	0.0019
0.0383	10	13-18	0.0254	0.0129
0.0125	11	13-18	0.0254	-0.0129
0.0128	9	31-40	0.0049	0.0079
0.0000	9	25-30	0.0049	-0.0049

A (very) Simplified Boosting Example, cont.

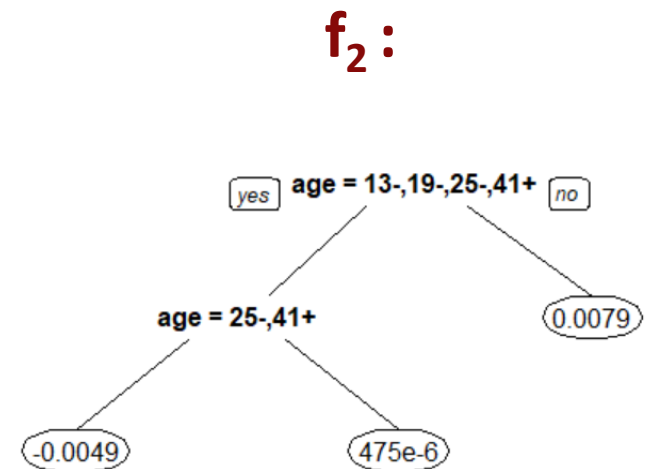
We now try to fit the residuals using a second simple tree model:

Data			Model 1	Residuals
CTR	titleWords	age	g_1	r_1
0.0000	10	41+	0.0049	-0.0049
0.0761	13	13-18	0.0761	0.0000
0.0068	11	19-24	0.0049	0.0019
0.0383	10	13-18	0.0254	0.0129
0.0125	11	13-18	0.0254	-0.0129
0.0128	9	31-40	0.0049	0.0079
0.0000	9	25-30	0.0049	-0.0049

A (very) Simplified Boosting Example, cont.

We now try to fit the residuals using a second simple tree model:

Data			Model 1	Residuals	Tree to fit residuals
CTR	titleWords	age	g_1	r_1	f_2
0.0000	10	41+	0.0049	-0.0049	-0.0049
0.0761	13	13-18	0.0761	0.0000	0.0005
0.0068	11	19-24	0.0049	0.0019	0.0005
0.0383	10	13-18	0.0254	0.0129	0.0005
0.0125	11	13-18	0.0254	-0.0129	0.0005
0.0128	9	31-40	0.0049	0.0079	0.0079
0.0000	9	25-30	0.0049	-0.0049	-0.0049



A (very) Simplified Boosting Example, cont.

We will now “boost” the initial model a little with the tree that fit the residuals:

New model

$$\mathbf{g}_2 = \mathbf{g}_1 + 0.10 \mathbf{f}_2$$

Data			Model 1	Residuals	Tree to fit residuals	Model 2
CTR	titleWords	age	$\mathbf{g}_1$	$\mathbf{r}_1$	$\mathbf{f}_2$	$\mathbf{g}_2$
0.0000	10	41+	0.0049	-0.0049	-0.0049	0.0044
0.0761	13	13-18	0.0761	0.0000	0.0005	0.0761
0.0068	11	19-24	0.0049	0.0019	0.0005	0.0049
0.0383	10	13-18	0.0254	0.0129	0.0005	0.0254
0.0125	11	13-18	0.0254	-0.0129	0.0005	0.0254
0.0128	9	31-40	0.0049	0.0079	0.0079	0.0057
0.0000	9	25-30	0.0049	-0.0049	-0.0049	0.0044

A (very) Simplified Boosting Example, cont.

Then we repeat the process, and repeat again, ..., etc.

Data			Model 1	Residuals	Tree to fit residuals	Model 2	Residuals		
CTR	titleWords	age	g_1	r_1	f_2	g_2	r_2		
0.0000	10	41+	0.0049	-0.0049	-0.0049	0.0044	-0.0044		
0.0761	13	13-18	0.0761	0.0000	0.0005	0.0761	0.0000		
0.0068	11	19-24	0.0049	0.0019	0.0005	0.0049	0.0019		
0.0383	10	13-18	0.0254	0.0129	0.0005	0.0254	0.0129		
0.0125	11	13-18	0.0254	-0.0129	0.0005	0.0254	-0.0129		
0.0128	9	31-40	0.0049	0.0079	0.0079	0.0057	0.0071		
0.0000	9	25-30	0.0049	-0.0049	-0.0049	0.0044	-0.0044		

A (very) Simplified Boosting Example, cont.

Then we repeat the process, and repeat again, ..., etc.

Data			Model 1	Residuals	Tree to fit residuals	Model 2	Residuals	Tree to fit residuals
CTR	titleWords	age	g_1	r_1	f_2	g_2	r_2	f_3
0.0000	10	41+	0.0049	-0.0049	-0.0049	0.0044	-0.0044	-0.0044
0.0761	13	13-18	0.0761	0.0000	0.0005	0.0761	0.0000	-0.0065
0.0068	11	19-24	0.0049	0.0019	0.0005	0.0049	0.0019	0.0019
0.0383	10	13-18	0.0254	0.0129	0.0005	0.0254	0.0129	0.0100
0.0125	11	13-18	0.0254	-0.0129	0.0005	0.0254	-0.0129	-0.0065
0.0128	9	31-40	0.0049	0.0079	0.0079	0.0057	0.0071	0.0100
0.0000	9	25-30	0.0049	-0.0049	-0.0049	0.0044	-0.0044	-0.0044

A (very) Simplified Boosting Example, cont.

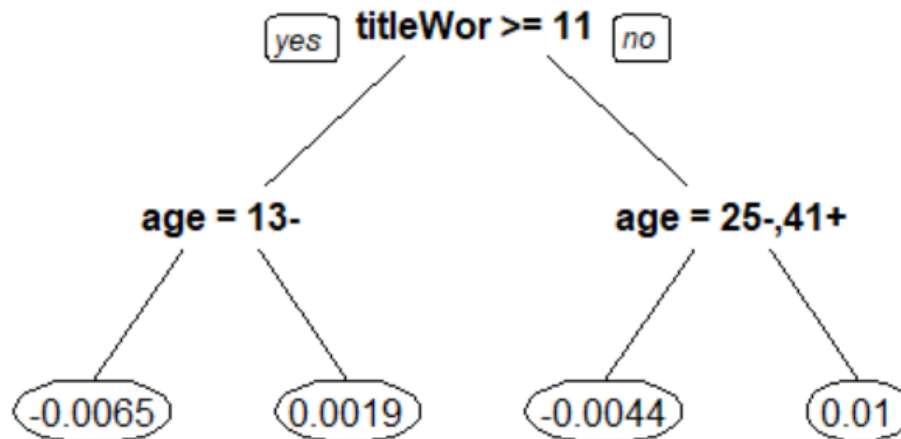
Then we repeat the process, and repeat again, ..., etc.

Data			Model 1	Residuals	Tree to fit residuals	Model 2	Residuals	Tree to fit residuals	Model 3
CTR	titleWords	age	g_1	r_1	f_2	g_2	r_2	f_3	g_3
0.0000	10	41+	0.0049	-0.0049	-0.0049	0.0044	-0.0044	-0.0044	0.0040
0.0761	13	13-18	0.0761	0.0000	0.0005	0.0761	0.0000	-0.0065	0.0755
0.0068	11	19-24	0.0049	0.0019	0.0005	0.0049	0.0019	0.0019	0.0051
0.0383	10	13-18	0.0254	0.0129	0.0005	0.0254	0.0129	0.0100	0.0264
0.0125	11	13-18	0.0254	-0.0129	0.0005	0.0254	-0.0129	-0.0065	0.0248
0.0128	9	31-40	0.0049	0.0079	0.0079	0.0057	0.0071	0.0100	0.0067
0.0000	9	25-30	0.0049	-0.0049	-0.0049	0.0044	-0.0044	-0.0044	0.0040

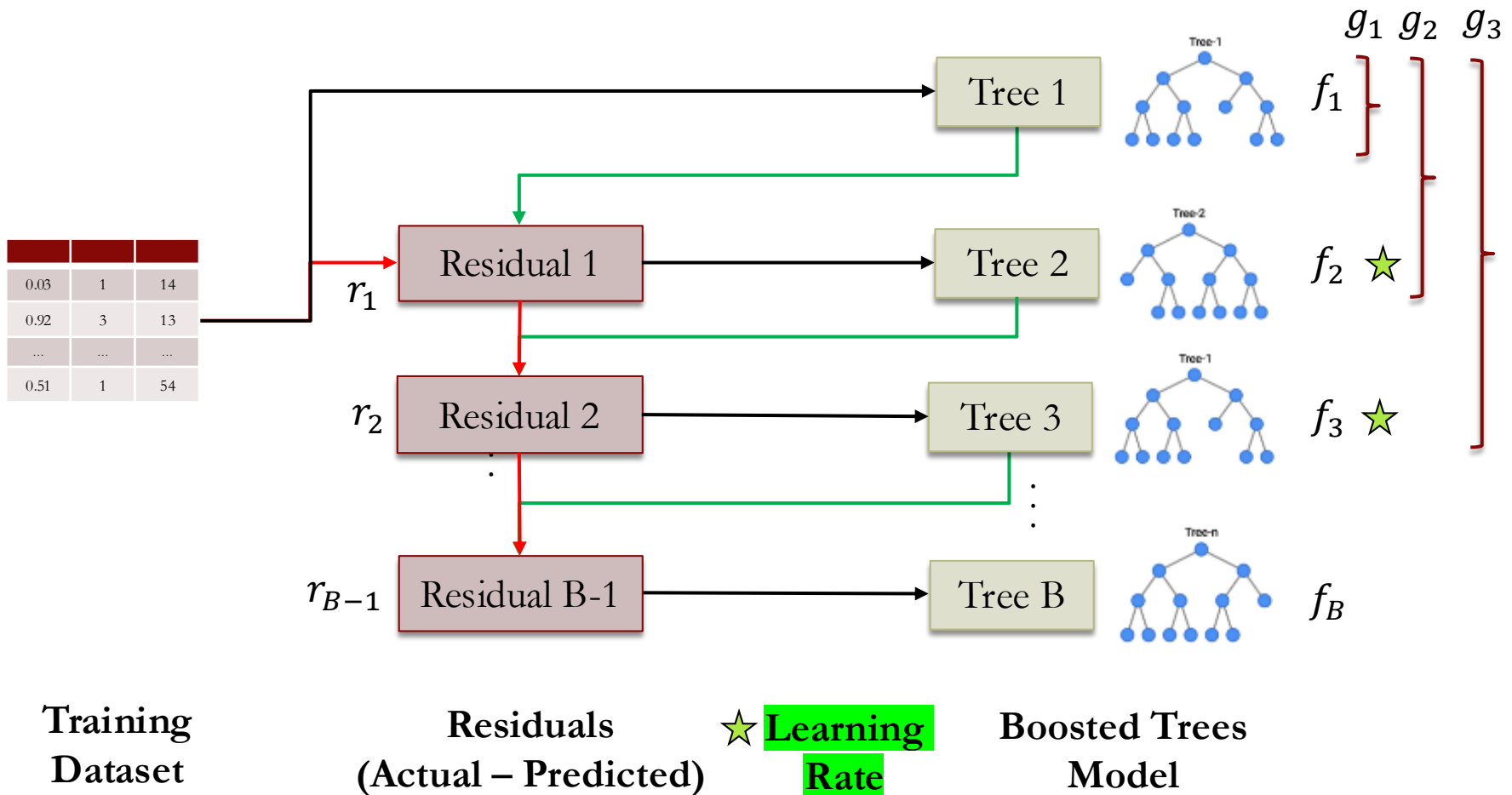
$$g_3 = g_2 + 0.10 f_3 (= g_1 + 0.10 f_2 + 0.10 f_3)$$

A (very) Simplified Boosting Example, cont.

Here is tree $\mathbf{f}_3$:



Building the Boosted Trees



Building Boosted Trees

- Start with an initial tree g_1 and calculate its residual r_1 .
- For each iteration $i = 2, 3, \dots, B$
 - Fit a CART model f_i to the residuals r_{i-1}
 - Calculate $g_i = g_{i-1} + \eta \cdot f_{i-1}$
 - Calculate new residuals r_i as $y - g_i$
- g_i is the prediction using the CART models $1, 2, \dots, i$ out of B .
- The residuals r_i are the “errors” between the actual value (y) and the prediction g_i . Each CART model f_i tries adjust the prediction g_{i-1} based on the residuals r_{i-1} .
- We often set g_1 (the initial model) to be a tree with 1 leaf that predicts classical baseline model $\bar{y}_{train}$ (the average of the observations in the training set). But this is not required.

Building Boosted Trees

- Each trees $f_1, f_2, \dots, f_B$ are intentionally limited to be shallow/weak trees (very few splits, not very deep trees).
 - These types of trees are sometimes called “**tree stumps**” or just “**stumps**”
 - Each tree is limited in its ability to fix residuals, but each tree certainly does not overfit.
- η is the **learning rate**, also sometimes called the **shrinkage parameter**.
 - The learning rate η is usually set to be a very small number ($\eta = 0.01$ is fairly common)
 - Many small changes lead to more stable models than a lot of big changes.

Tuning Parameters for Boosting

The class **GradientBoostingRegressor** in sklearn implements boosting for trees, and has four main hyper-parameters:

- **n_estimators:** Total number of boosting iterations B .
- **max_depth:** total number of splits performed in each tree
 - Setting to 1 allows only for a single predictor variable; setting to 2 allows for pairwise interactions
 - Such small trees are sometimes called “tree stumps” or just “stumps”
- **learning_rate:** (η) controls the extent to which each new tree contributes to the additive model:

$$g_{j+1}(x_1, \dots, x_k) = g_j(x_1, \dots, x_k) + \eta f_{j+1}(x_1, \dots, x_k)$$

- **min_samples_leaf:** minimum number of samples in a leaf (same as in CART).

Boosting Parameter Insights from Practice

The class **GradientBoostingRegressor** implements boosted trees, and **GridSearchCV(...)** implements cross-validation of the hyper-parameters.

- Boosting works best with *slow learning*, namely the **learning_rate** η is small
- The slower the learning rate, the more trees (**n_estimators**) will be needed to optimize performance...
- But setting **n_estimators** too large can result in overfitting (this does not happen with Random Forests)
- The best value of **max_depth** is often between 4 and 8, but can be problem dependent
- **min_samples_leaf** should be kept at a reasonable value, but is less important in practice (the default value in Python is **min_samples_leaf** = 1)
- But wait, there is even more...

Stochastic version of Boosting

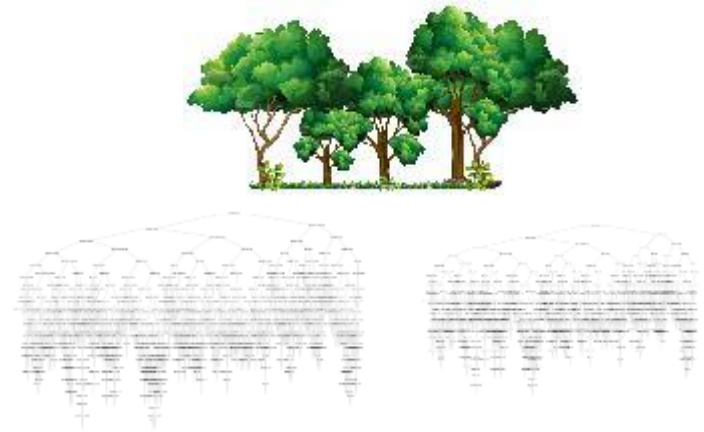
- In Random Forests, we have seen the power of introducing randomness into our fitting algorithm through the use of bootstrapped samples (bagging)
- For Boosting, a slight variant of Boosted Trees which induces randomness also provides improvements in performance
- In each iteration, instead of using the entire training set to fit the next tree, we choose a random subset of the training set, where each observation is chosen to be in the random subset with probability p .
- The rest of the Boosting method is exactly as described earlier
- It is typical to use $p = 0.5$. This is controlled by the **subsample** parameter in **GradientBoostingRegressor**, and the default is $p = 1.0$. It is sometimes set smaller when n is very large.

Random Forest vs. Boosted Trees

Comparing Boosting and Random Forests

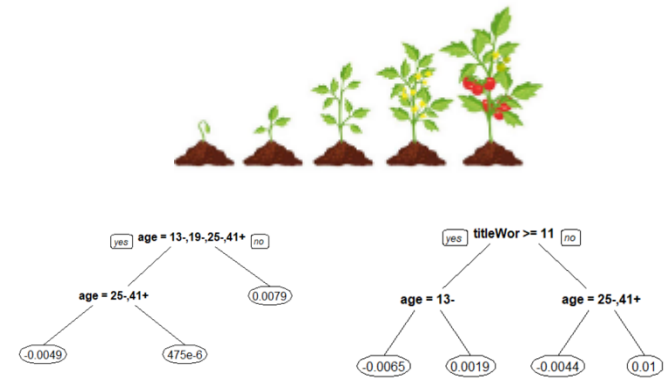
In Random Forests:

- Deep trees are trained **independently** in different bootstrap samples and combined by average/majority vote.
- Each tree is designed to be diverse using randomness.

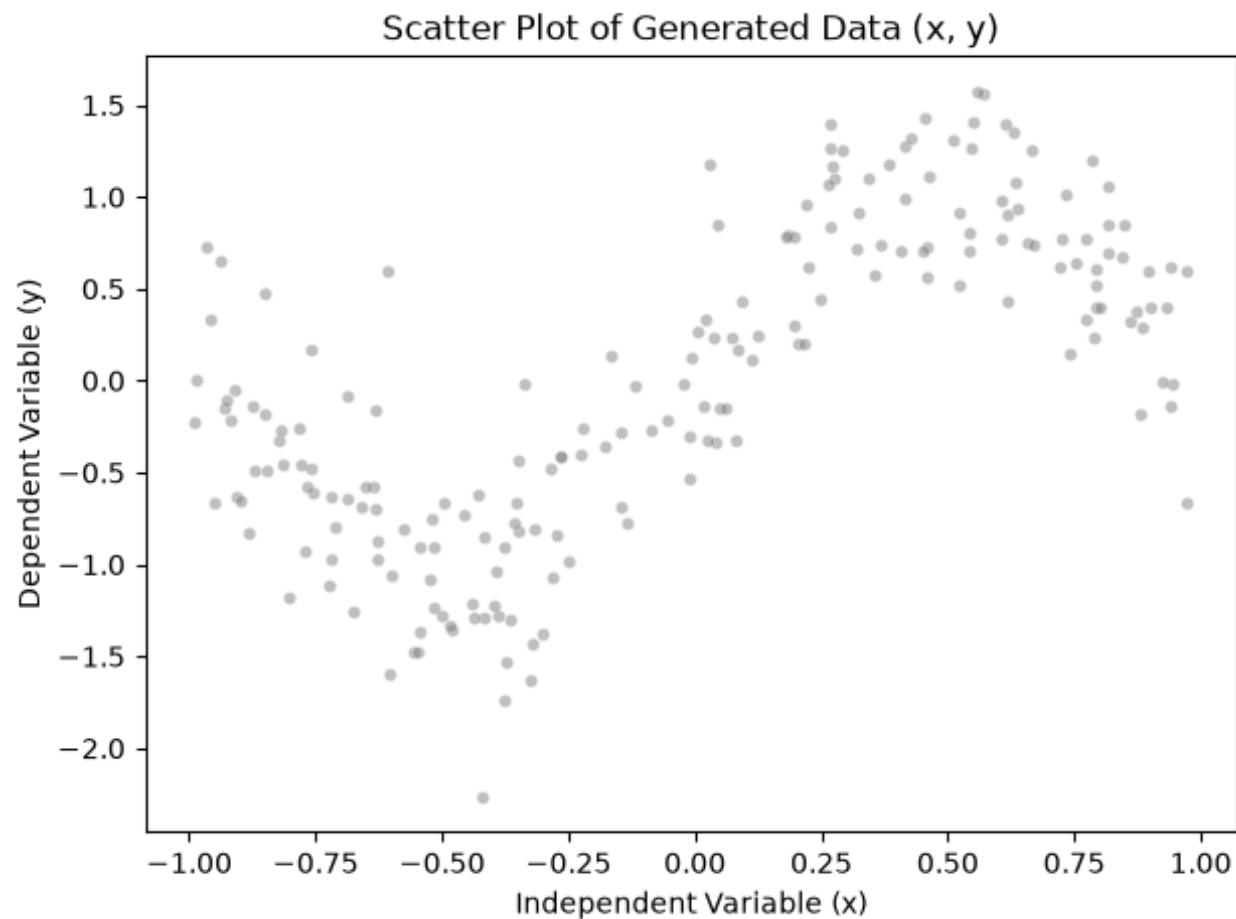


In Boosted Trees:

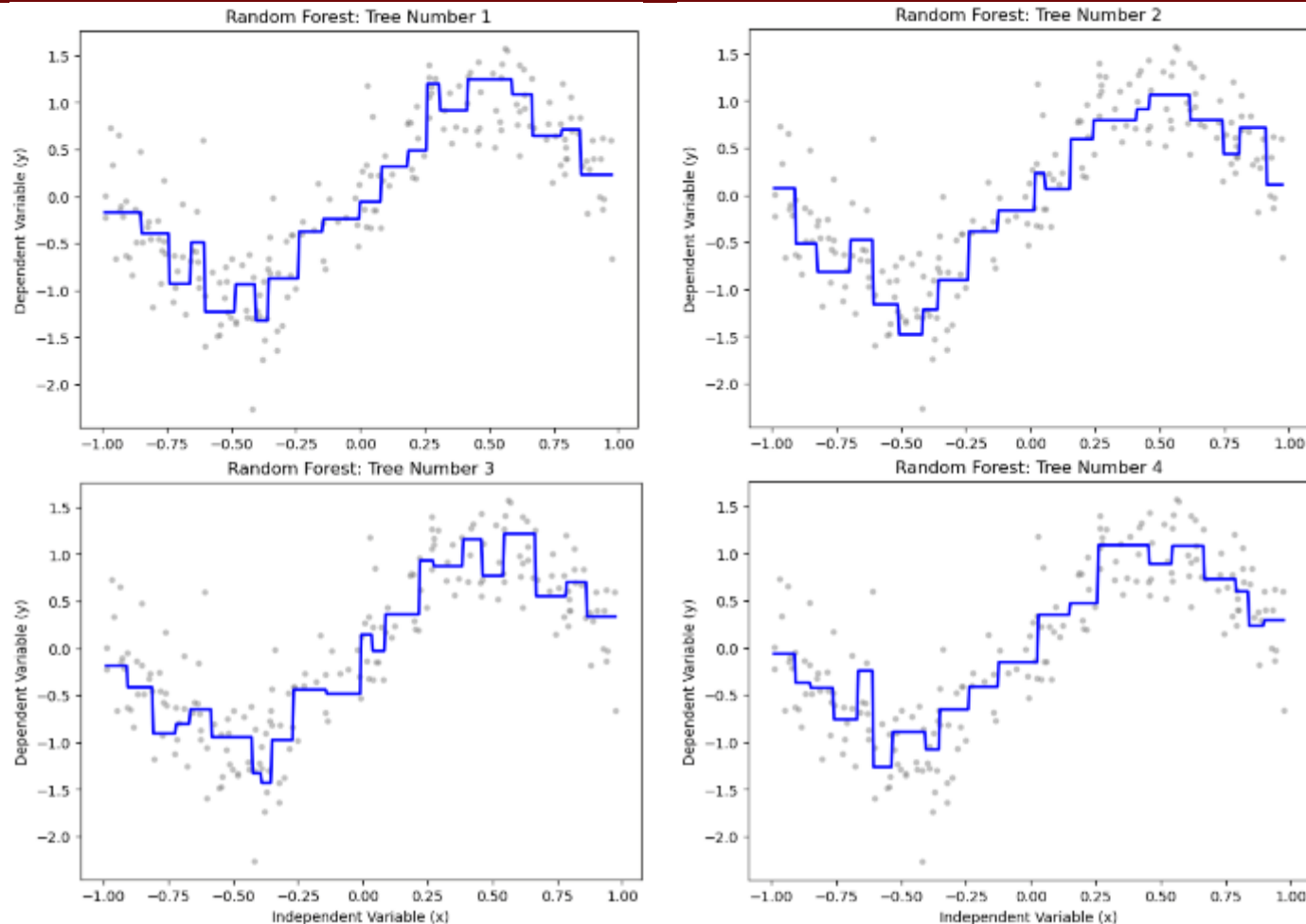
- Shallow trees are trained **adaptively** and combined sequentially.
- Each new tree is designed to tackle the residuals (errors) from the previous additive tree model



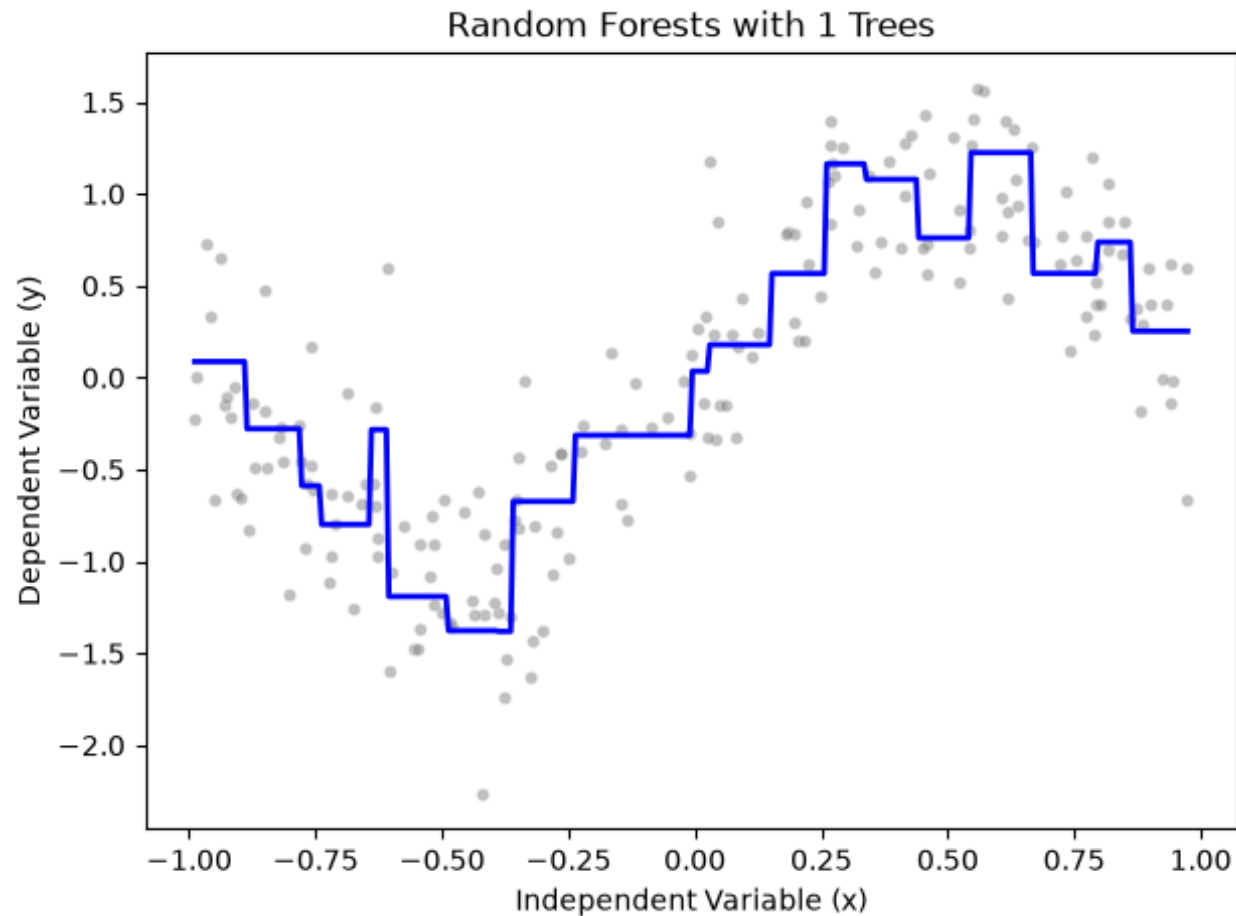
Demonstration



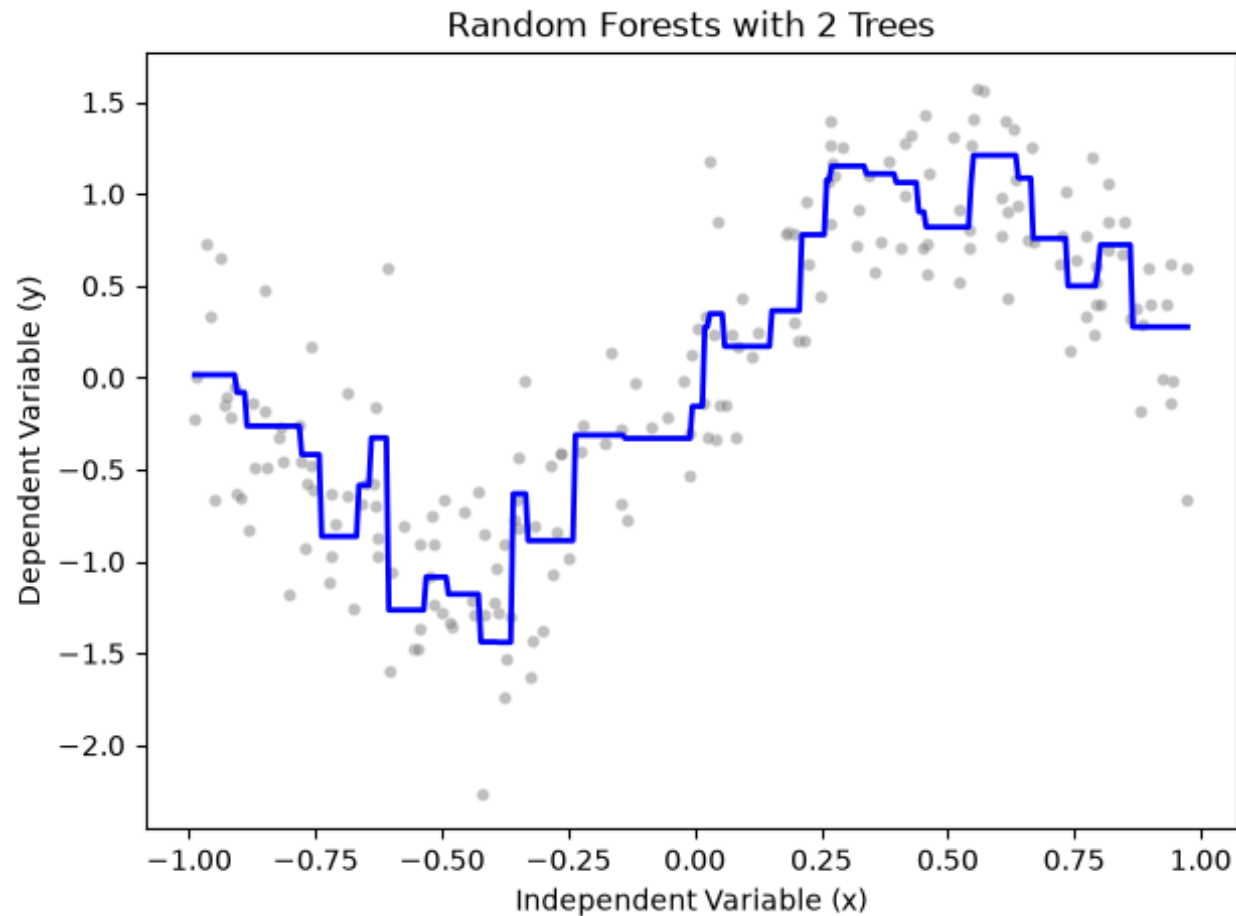
Random Forest: Individual Trees



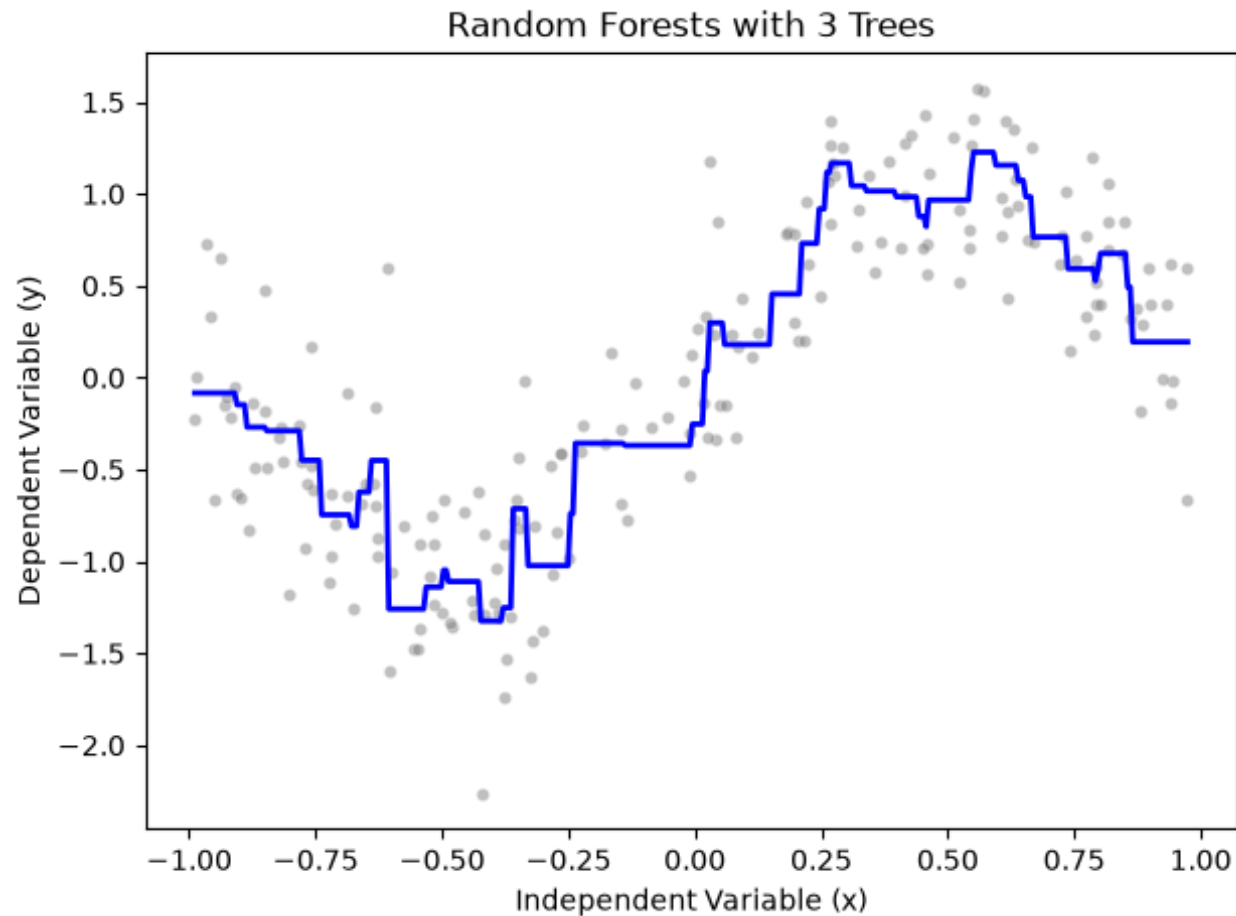
Random Forest: Tree 1 Only



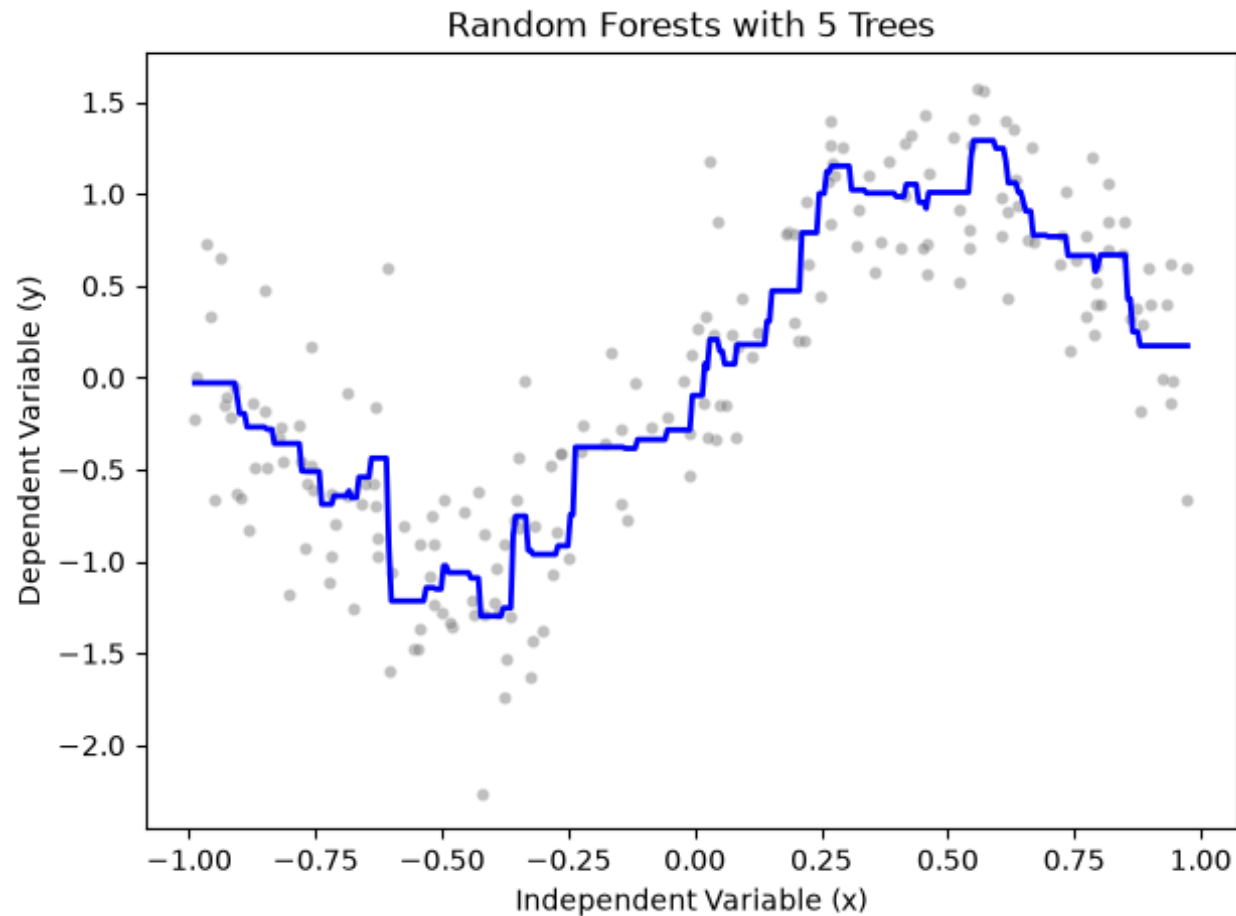
Random Forest: 2 Trees



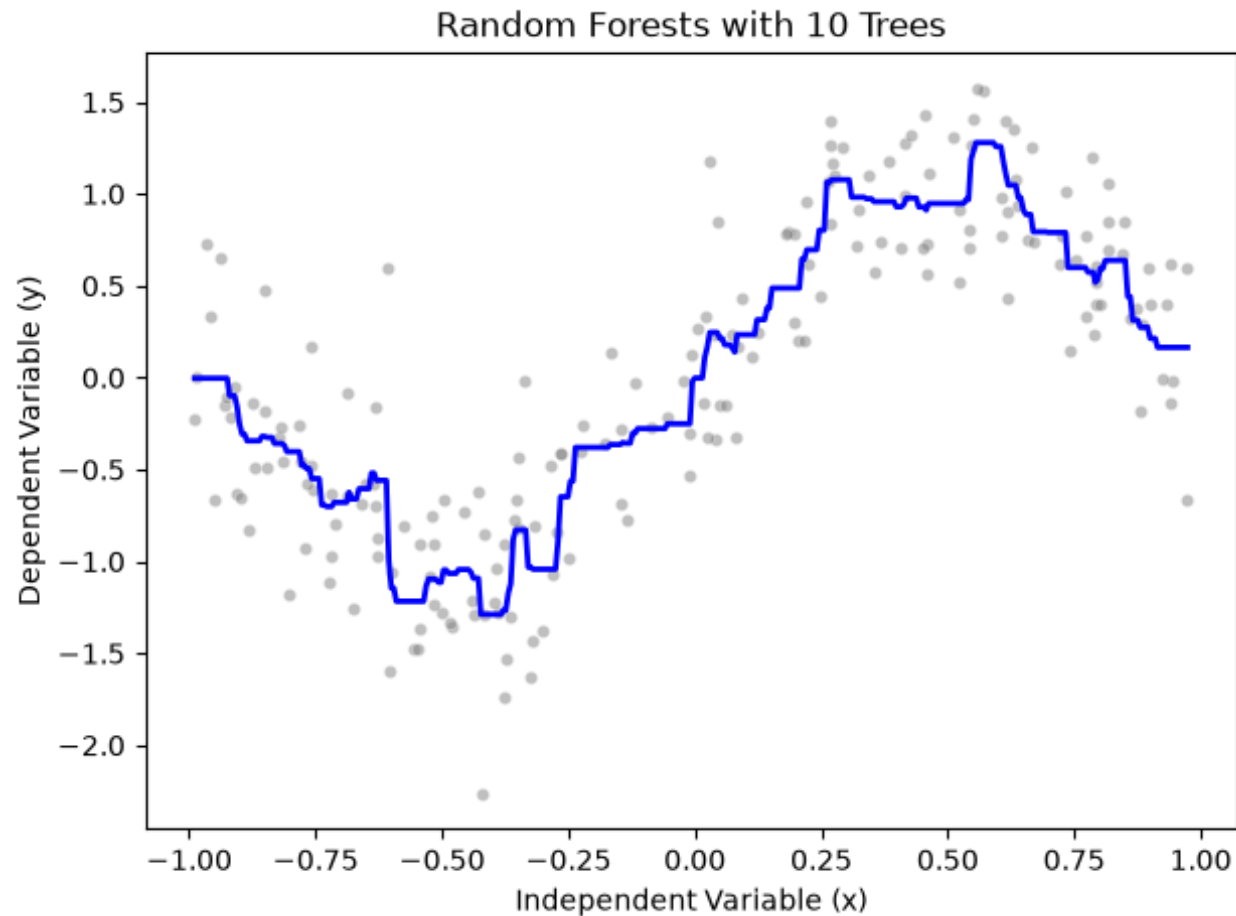
Random Forest: 3 Trees



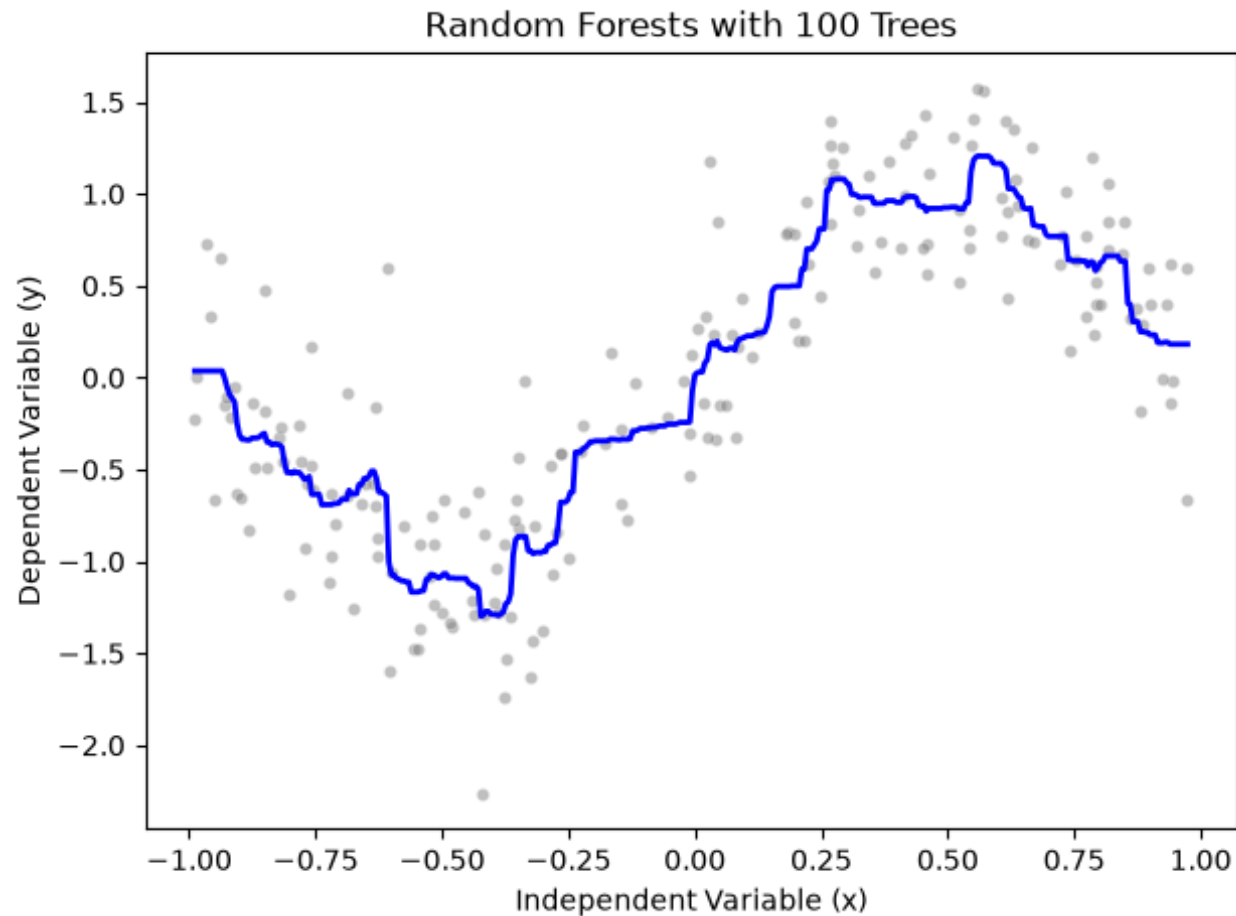
Random Forest: 5 Trees



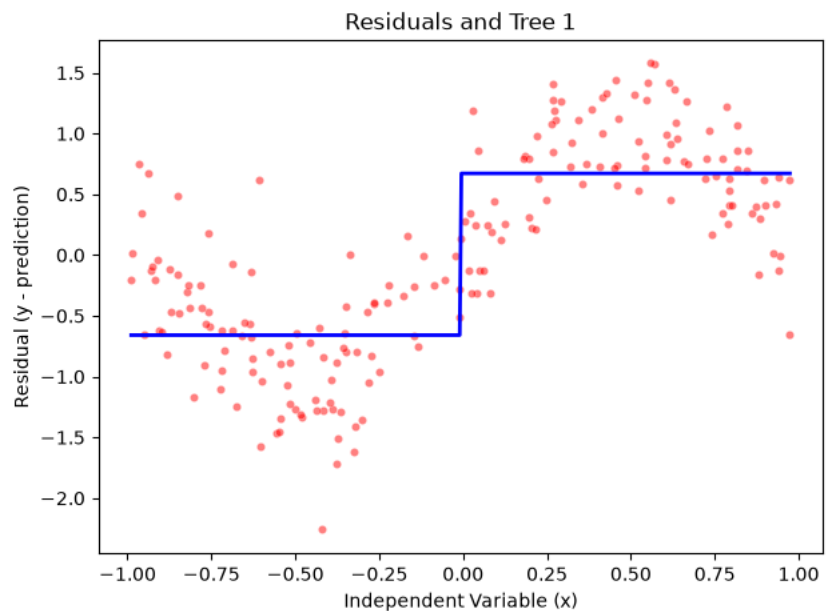
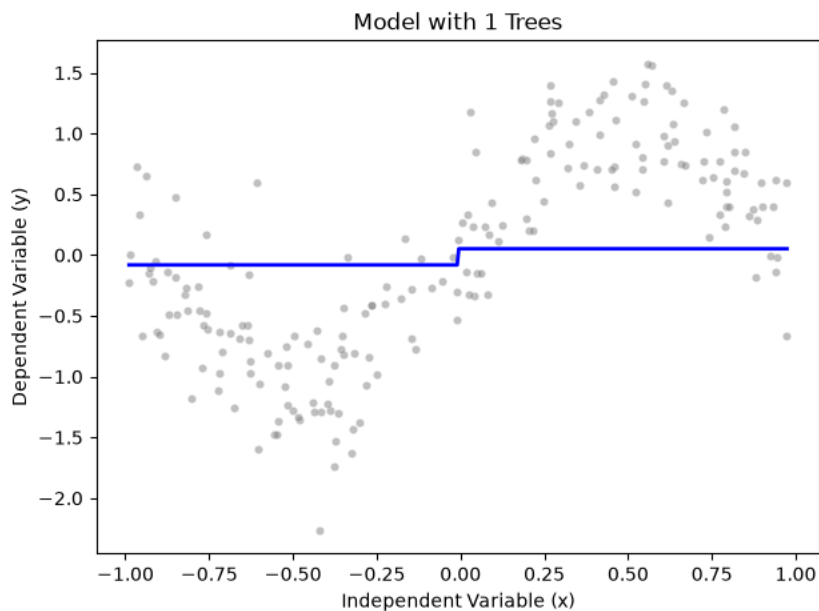
Random Forest: 10 Trees



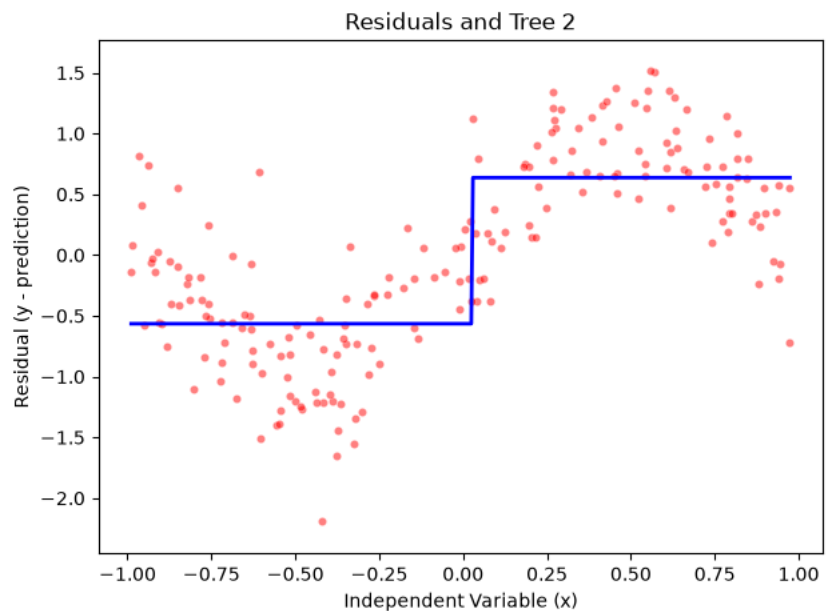
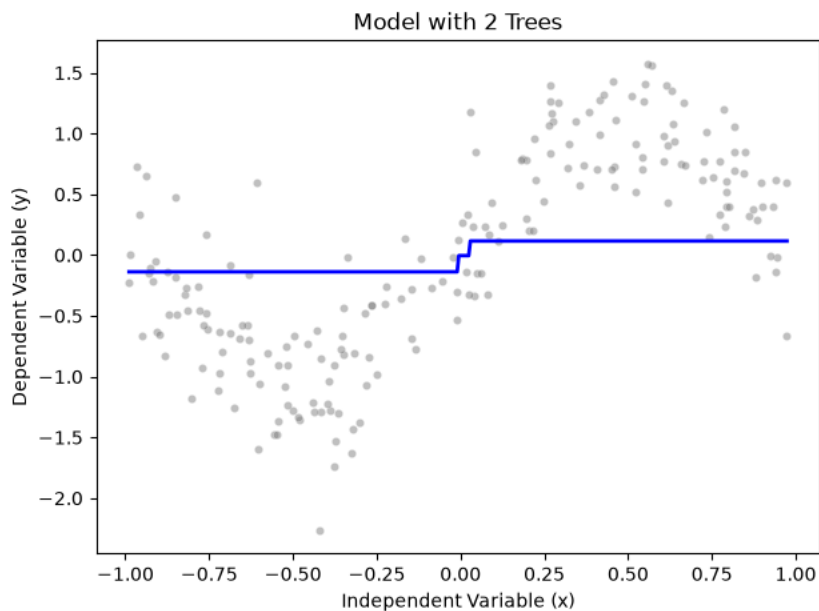
Random Forest: 100 Trees



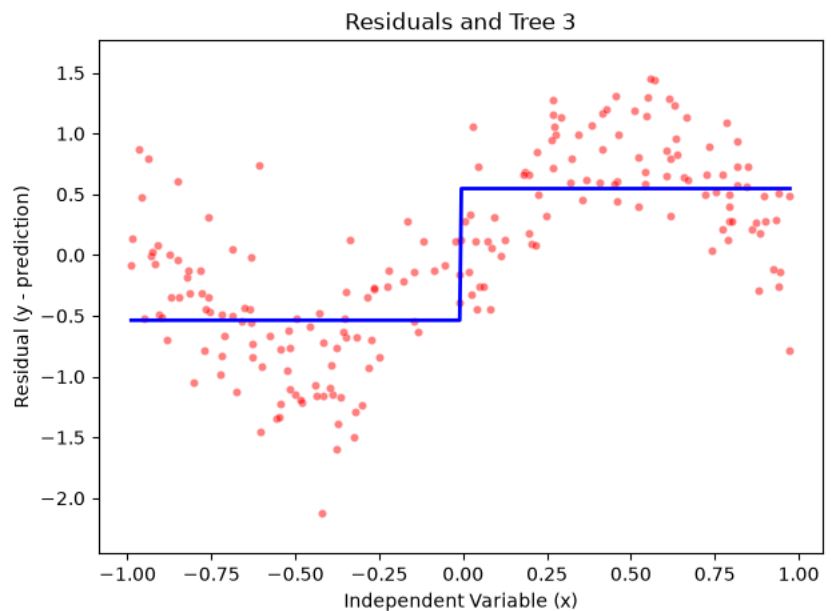
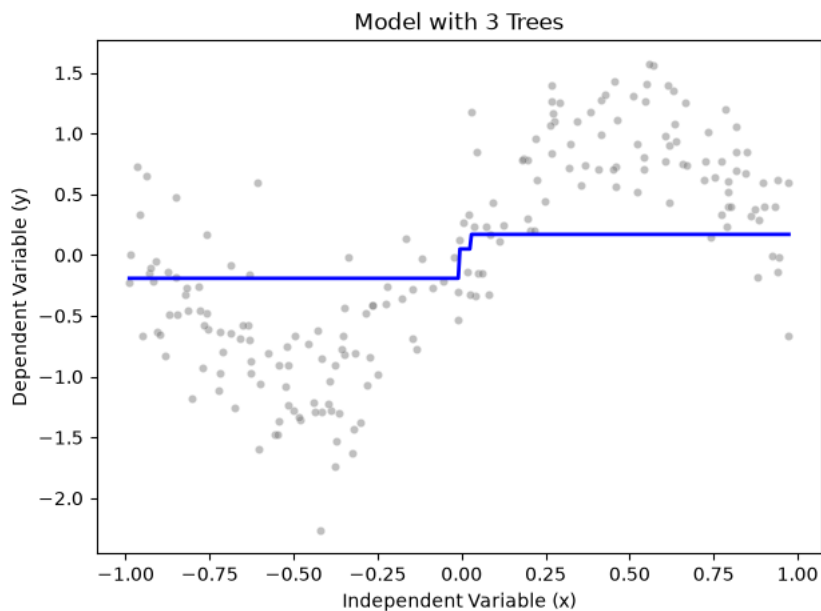
Boosted Trees: 1 Tree



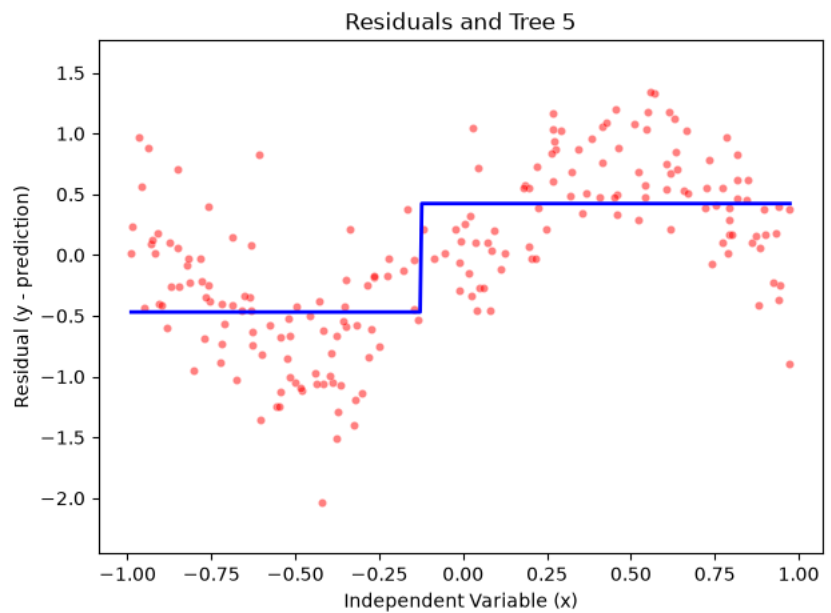
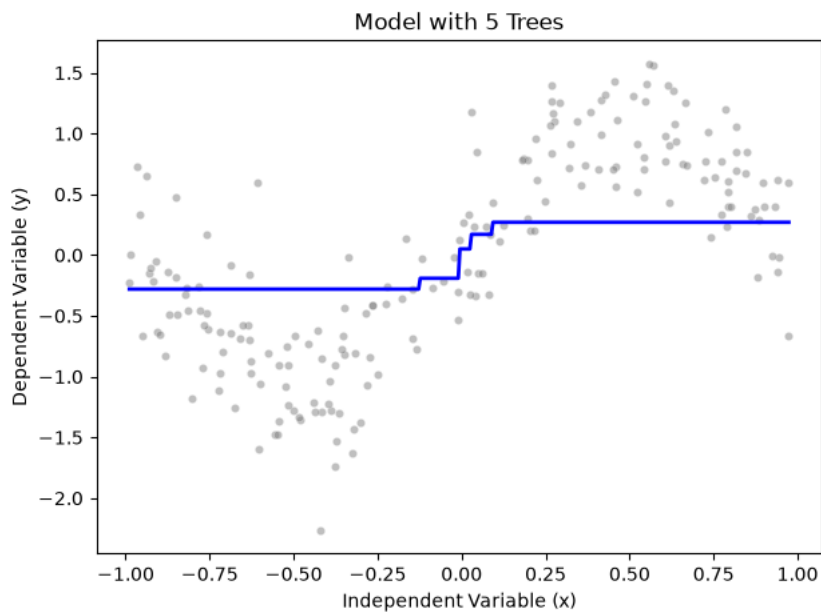
Boosted Trees: 2 Trees



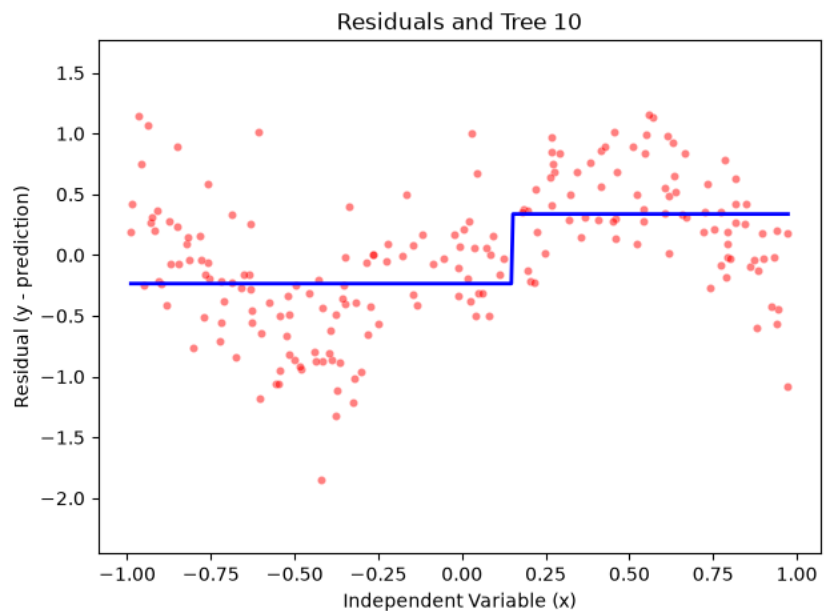
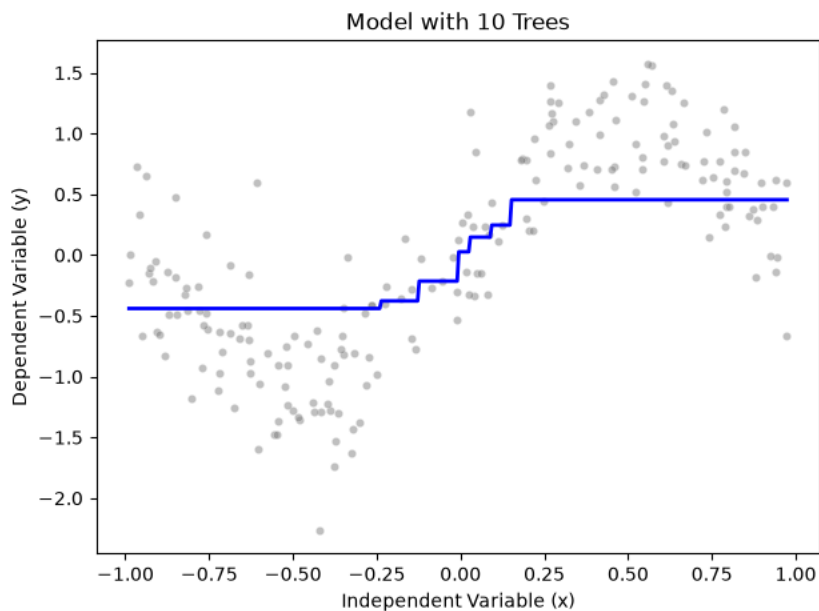
Boosted Trees: 3 Trees



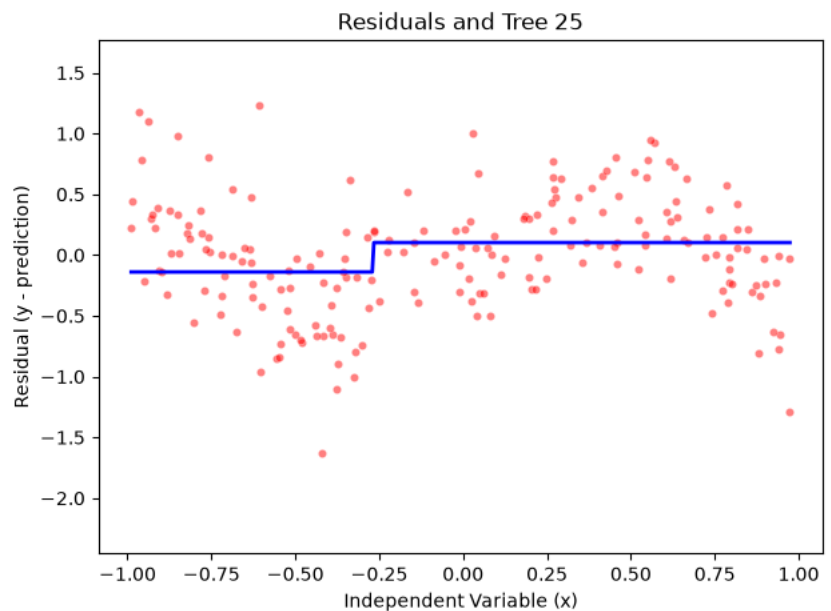
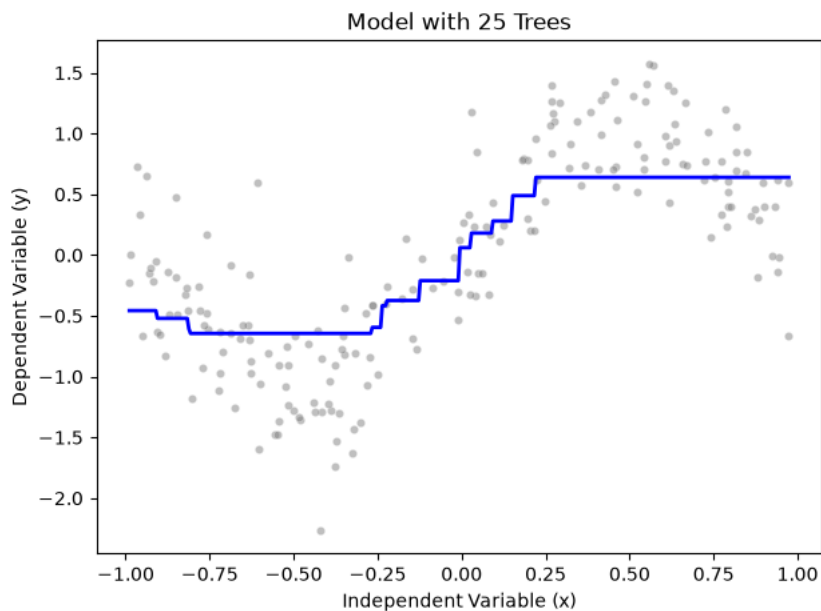
Boosted Trees: 5 Trees



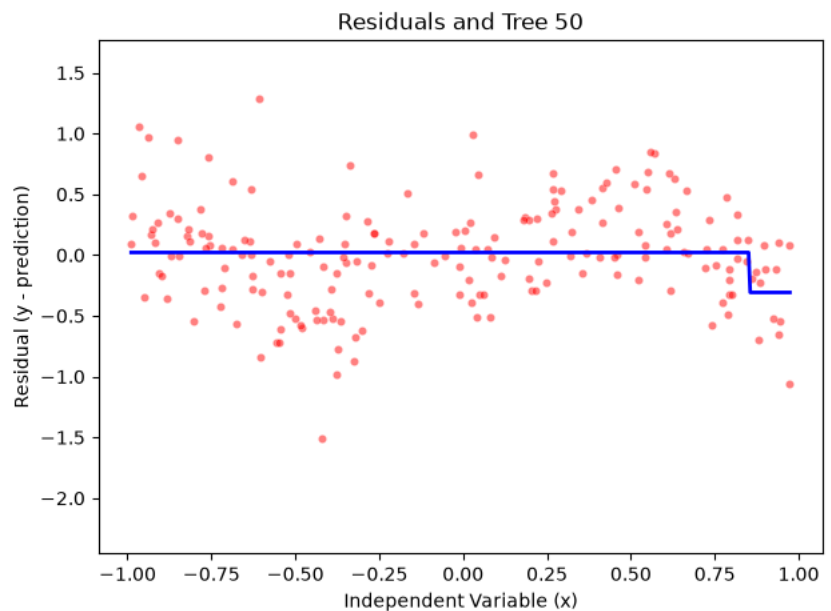
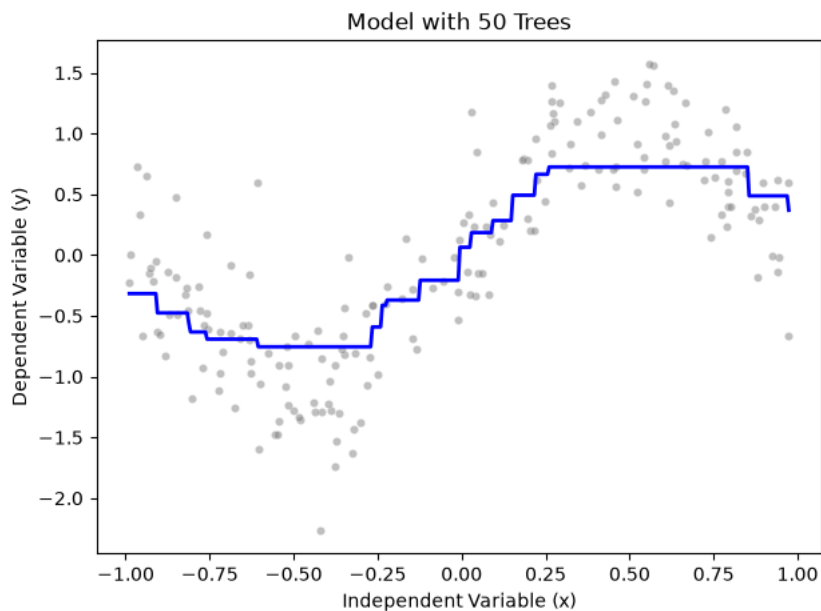
Boosted Trees: 10 Trees



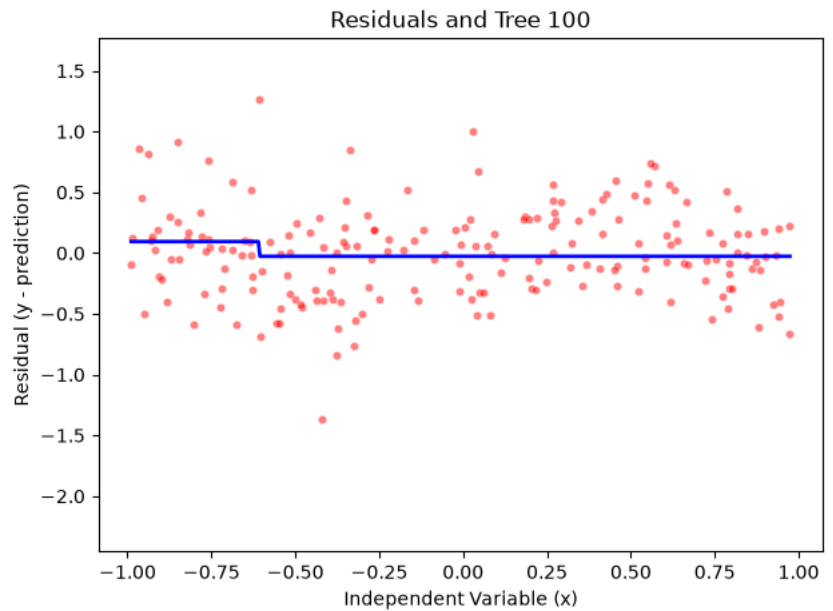
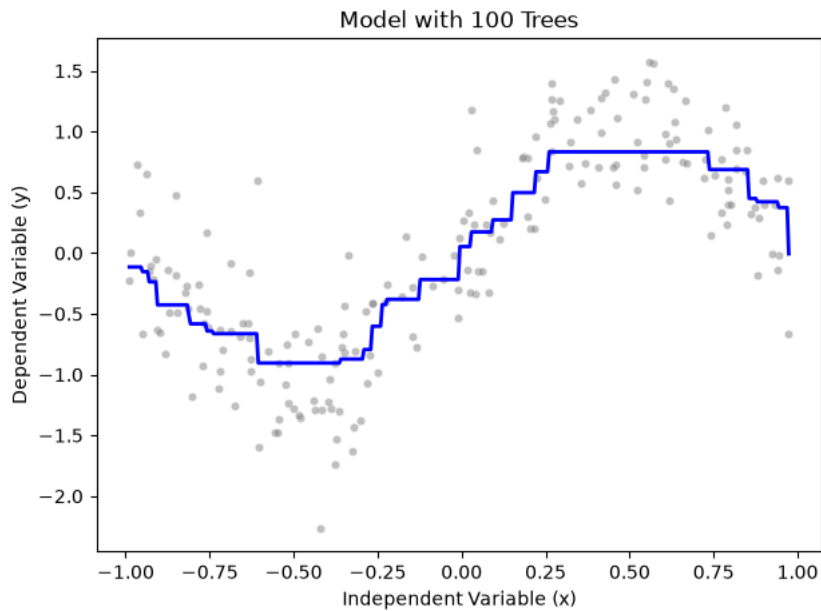
Boosted Trees: 25 Trees



Boosted Trees: 50 Trees



Boosted Trees: 100 Trees



The Punchline

Out-of-Sample Error = In-Sample Error + Tendency to Overfit

- Random Forest
 - Each tree has great in-sample fit, but overfits significantly.
 - **Solution:** Averaging many trees keeps the in-sample performance, but reduces the overfitting.
- Boosted Trees
 - Each tree is simple (little overfitting) but cannot fit the in-sample data well.
 - **Solution:** Creating trees sequentially keeps the low overfitting tendency, but gradually improves the in-sample fit.

For those interested, look up the bias-variance tradeoff in statistics.

Results on CTR Dataset

Click-Through Rate (CTR) Data

k : # of independent variables ($k = 10$); n : # of observations ($n = 6,057$)

	titleWords	adWords	depth	position	advCTR	advCTRInPos	queryCTR	queryCTRInPos	gender	age	CTR
0	10	17	3	2	0.0136	0.0146	0.0	0.0	male	41+	0.0
1	13	30	2	1	0.0373	0.0465	0.0382	0.0581	male	13-18	0.0761
2	12	14	1	1	0.0254	0.031	0.0255	0.0323	male	13-18	0.0426
3	5	19	3	2	0.0178	0.0076	0.0035	0.0017	female	25-30	0.0
4	11	17	2	2	0.0096	0.0069	0.0294	0.0171	female	19-24	0.0068
5	15	28	2	2	0.0307	0.0157	0.0324	0.0137	female	31-40	0.0
6	10	20	3	1	0.0283	0.0338	0.0219	0.0353	female	13-18	0.0383
7	9	23	3	1	0.0054	0.0084	0.0183	0.0208	male	25-30	0.0172
8	8	20	2	1	0.0153	0.0282	0.0281	0.0562	male	13-18	0.1015
9	11	21	2	1	0.018	0.0314	0.0216	0.0296	female	13-18	0.0125
...	...	...	...	...	...	...	...	...	...	...	...
6052	8	16	1	1	0.0273	0.0356	0.004	0.0	unknown	unknown	0.0182
6053	6	20	1	1	0.0919	0.0982	0.0	0.0	female	19-24	0.0536
6054	9	26	3	1	0.0496	0.0523	0.0467	0.0668	unknown	unknown	0.0141
6055	9	16	2	1	0.0566	0.061	0.0618	0.1012	unknown	unknown	0.1339
6056	12	23	2	1	0.043	0.062	0.0215	0.0287	unknown	unknown	0.1322

We use Cross-Validation to set parameter values

We fix **learning_rate** = 0.001 and **min_samples_leaf** = 10

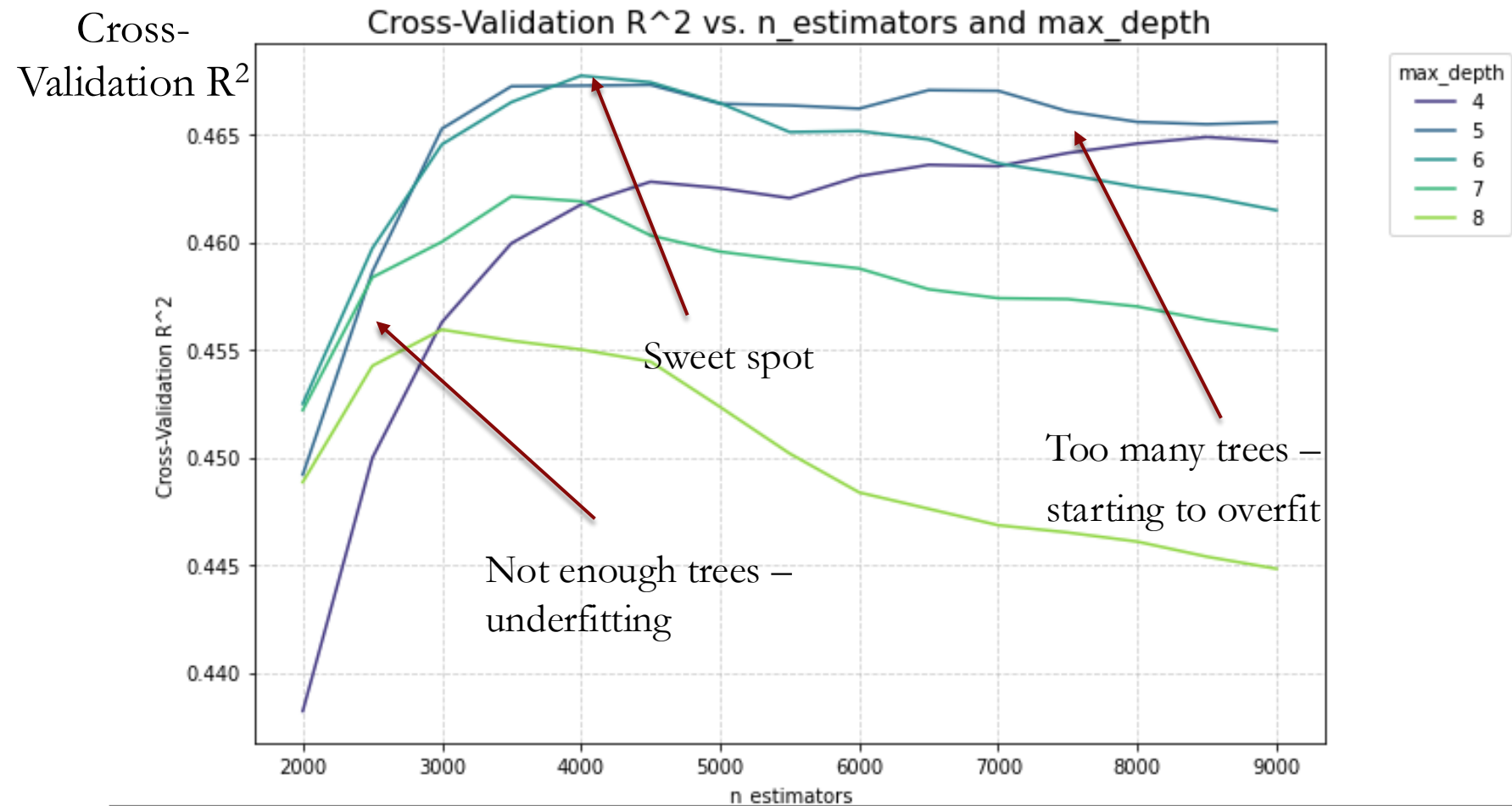
We perform a grid search over **max_depth** and **n_estimators**

For **max_depth** = 4, 6, 8, 10

For **n_estimators** = 100, 1000, 5000, 10000, 15000, 20000, 25000, 30000

- Do cross-validation with the current values of **(max_depth, n_estimators)**
- Record the averaged cross-validated R^2 (over the 2 folds) using the current **(max_depth, n_estimators)** values
- Then set the **(max_depth, n_estimators)** values to be those values that achieved the highest average R^2 value
- This yielded **max_depth** = 6; **n_estimators** = 5000

Selecting `n_estimators` and `max_depth` hyper-parameters using cross-validation (CV)



Model Comparison

	Linear Regression	CART Model	Random Forest	Boosting
OSR ²	0.3961	0.3377	0.4940	0.4718
Out-of-Sample MAE	0.0331	0.0340	0.0304	0.0307
Out-of-Sample RMSE	0.0551	0.0577	0.0504	0.0515

- For this example, Boosting and Random Forests have comparable performance.
- Parameter tuning took about 16 minutes. (Unfortunately, there is no analogue of “out-of-bag tuning” for Boosting)
- If you are only going to train an algorithm once, having to wait a few hours is not too bad
- If you need to frequently update predictions as new data arrives, computation time is a huge consideration
- Unlike Random Forests, Boosting is not easily amenable to parallel computation

Boosting versus Random Forests

- Both models are very powerful
- Conventional wisdom is that Boosting typically outperforms Random Forests
- However, many times this difference is very small, and sometimes Random Forests is better
 - But in some contexts (think Google or Meta) miniscule improvements in predictive performance can equate to many millions of dollars of additional revenue
- Random Forests is easier to tune, which is a major advantage
 - Fewer tuning parameters for Random Forests than for Boosting
 - Random Forests is generally more robust (less sensitive) with respect to parameter choices (but some care should still be taken)

Model Comparison Exercise

	Linear Regression	Logistic Regression	CART	Random Forest	Boosting
Can be used to predict numerical outcomes					
Can be used for classification problems					
Captures non-linear structures in the data					
Are Simple and interpretable					
Yield best out-of-sample prediction quality					
Is computationally efficient					

Model Comparison Exercise

	Linear Regression	Logistic Regression	CART	Random Forest	Boosting
Can be used to predict numerical outcomes	✓	✗	✓	✓	✓
Can be used for classification problems	✗	✓	✓	✓	✓
Captures non-linear structures in the data	✗	✗	✓	✓	✓
Are Simple and interpretable	~	~	✓	✗	✗
Yield best out-of-sample prediction quality	~	~	~	✓	✓
Is computationally efficient	✓	✓	✓	~	✗

Optional: XGBoost

XGBoost: background

- For a while, Boosting and Random Forests were essentially tied for first place in a host of Machine Learning competitions
- Tianqi Chen developed XGBoost while he was a PhD student at the University of Washington. (He finished his Ph.D. in 2019 and is now Associate Professor at Carnegie Mellon University and Distinguished Engineer at NVIDIA.)
- XGBoost has been a dominant prediction method ever since
- XGBoost stands for Extreme Gradient Boosting



<https://tqchen.com/>

Comparison of XGBoost and other models to predict Click-Through Rates (CTRs)

	Linear Regression	CART Model	Random Forest	Boosting	XGBoost
OSR ²	0.3961	0.3377	0.4940	0.4718	0.4930
Out-of-Sample MAE	0.0331	0.0340	0.0304	0.0307	0.0304
Out-of-Sample RMSE	0.0551	0.0577	0.0504	0.0515	0.0505

- XGBoost yields a (slight? is it real?) improvement in our out-of-sample metrics over Boosting without regularization
- Computationally very expensive to train -- we only explored a few hyperparameter settings through cross-validation
- Indeed, some better hyperparameter combinations may have been left unexplored

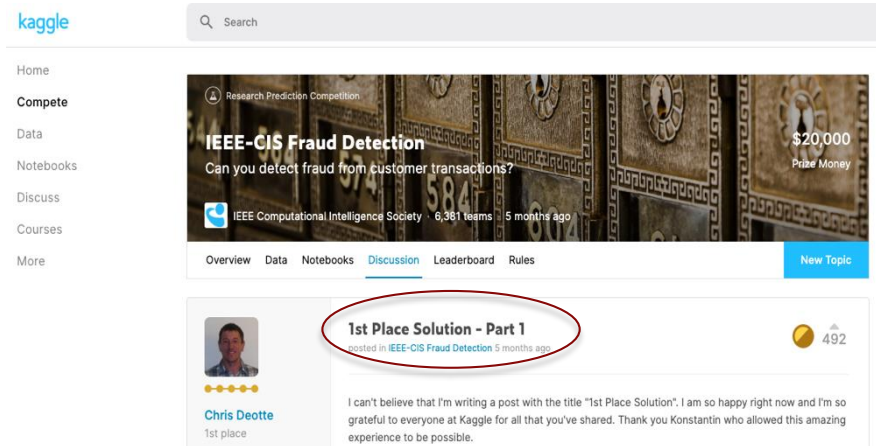
“No free lunch”, but...

- If you have a prediction problem with structured data where predictive performance is much more important than interpretability, try XGBoost
- There is not a single machine learning algorithm that will perform best in all settings, but XGBoost rarely performs poorly, and is often at or near the top
- Of course, if the underlying “true model” is a linear model, nothing can beat linear regression
- A key idea behind XGBoost is “regularization,” a topic that we cover next week.

Other Applications of Ensemble Learning

- Other variants of Boosting: LightGBM and CatBoost are similar and may be worth trying too

- Better still, ensemble the algorithms themselves together. (**Ensemble the ensemble learners...**)



Final Model

Our final model was a combination of 3 high scoring single models. CatBoost (Public/Private LB of 0.9639/0.9408), LGBM (0.9617/0.9384), and XGB (0.9602/0.9324). These models were diversified because Konstantin built the CAT and LGB while I built the XGB and NN. And we engineered features independently. (In the end we didn't use the NN which had LB 0.9432). XGB notebook posted [here](#).

